

AP Computer Science A

APCSA 国际考备考手册

2025 新版考纲 • 全四章合订本

第一章 Using Objects and Methods

第二章 Selection and Iteration

第三章 Class Creation

第四章 Data Collections

2026 年 9 月

总 目 录

第一章 Using Objects and Methods	1
一、本章定位与考情速览.....	2
1.1 新大纲四大单元与考试权重.....	2
1.2 考试结构.....	3
1.3 本章与 FRQ 的关联（备考重点提示）	3
二、知识点与考点梳理（Topic 1.1 - 1.15）	4
板块 A：Java 程序基础（Topic 1.1 - 1.2）	4
板块 B：表达式与运算（Topic 1.3 - 1.6）	5
板块 C：方法（Topic 1.7 - 1.10）	7
板块 D：Math 类（Topic 1.11）	9
板块 E：对象与 String（Topic 1.12 - 1.15）	10
三、重难点与易错点精讲.....	12
易错点 1：整数除法悄悄丢小数（失分率最高）	12
易错点 2：cast 的位置——太早、太晚都会错	12
易错点 3：截断 $\neq$ 四舍五入，负数向零截断.....	13
易错点 4：运算优先级——% 与 * / 同级	13
易错点 5：Math.random 区间公式记错端点	13
易错点 6：void 方法混入表达式	13
易错点 7：static 与实例方法调错对象	14
易错点 8：null 引用调用方法	14
易错点 9：String 不可变，忘记接住返回值.....	14
易错点 10：substring 含头不含尾 + 越界.....	14
易错点 11：equals / compareTo 的返回值	14
易错点 12：构造器重载的匹配.....	15
四、考纲对应的国际考真题与解析.....	15

4.1 MCQ 真题精选与解析	15
4.2 FRQ 真题精选与解析	22
五、备考策略与复习清单	39
5.1 复习优先级（按投入产出比排序）	39
5.2 考前自查清单	39
5.3 教师使用建议	40
参考文献	40
第二章 Selection and Iteration	41
一、本章定位与考情速览	42
1.1 新大纲四大单元与考试权重	42
1.2 考试结构	43
1.3 本章与 FRQ 的关联（备考重点提示）	43
1.4 本章学习目标（对应官方 12 个 Topic）	44
二、知识点与考点梳理（Topic 2.1 - 2.12）	44
板块 S：算法与控制结构总览（Topic 2.1 / 2.9）	44
板块 B：if / Nested if 选择结构（Topic 2.3 / 2.4）	48
板块 C：while 循环（Topic 2.7）	49
板块 D：for 循环与字符串算法（Topic 2.8 / 2.10 / 2.11）	50
板块 R：Informal Run-Time Analysis（Topic 2.12）	52
板块 F：易错点专题（综合）	53
三、本章重难点与易错点清单（按出错频率排序）	54
高频易错点 TOP 10	54
学习建议	54
四、MCQ 真题精选与详解	55
MCQ-1（&&/ 短路、De Morgan）	55
MCQ-2（混合条件运算符优先级）	56
MCQ-3（for 循环 + 字符串反向遍历）	57

MCQ-4 (独立 if 链 vs if-else if 链行为差异)	58
MCQ-5 (De Morgan / 表达式真值表)	60
MCQ-6 (while 循环输出追踪)	61
MCQ-7 (整数除法 + 双重嵌套循环计数)	62
MCQ-8 (String 比较 + 判别陷阱)	63
五、FRQ 真题精选与解析	64
FRQ-1 (AP Classroom Unit 2 FRQ Part A • BeadSequence)	65
FRQ-2 (AP Classroom Unit 2 FRQ Part B • colWidth 字符串填充)	68
FRQ-3 (AP Classroom Unit 2 FRQ Part B • 最长公共前缀)	70
FRQ-4 (AP Classroom Unit 2 FRQ Part B • 个位数恰好出现 1 次)	71
FRQ-5 (AP Classroom Unit 2 FRQ Part B • songInfo 提取)	73
六、应试技巧清单	74
MCQ 限时训练策略 (建议节奏)	75
FRQ 写作策略	75
代码追踪 (Tracing) 口诀	75
易错点自检清单	75
七、章节总结	75
第三章 Class Creation	79
一、本章定位与考情速览	80
1.1 新大纲四大单元与考试权重	80
1.2 考试结构	81
1.3 本章与 FRQ 的关联 (备考重点提示)	81
1.4 本章学习目标 (对应官方 9 个 Topic)	82
二、知识点与考点梳理 (Topic 3.1 - 3.9)	83
板块 A: 类设计基础 (Topic 3.1 / 3.2)	83
板块 B: Anatomy of a Class 与 Constructors (Topic 3.3 / 3.4)	84

板块 C: Methods 与引用传递 (Topic 3.5 / 3.6)	86
板块 D: Static 与 Scope (Topic 3.7 / 3.8)	87
板块 E: this Keyword (Topic 3.9)	89
三、本章重难点与易错点清单 (按出错频率排序)	90
高频易错点 TOP 10	90
学习建议.....	90
四、MCQ 真题精选与详解	91
MCQ-1 (TopicQuestion #1: 实例变量与构造器的可见性)	91
MCQ-2 (TopicQuestion #2: static 变量与多次创建对象)	92
MCQ-3 (TopicQuestion #3: 方法内访问实例变量)	93
MCQ-4 (TopicQuestion #4: 实例变量 vs 参数命名)	94
MCQ-5 (MCQ Part B #2: 返回对象的字符串表示)	96
MCQ-6 (MCQ Part B #4: String 不可变性 + 参数传递)	97
MCQ-7 (MCQ Part B #7: static 变量与方法反复修改)	98
MCQ-8 (MCQ Part B #8: static 方法不能直接访问实例变量)	100
五、FRQ 真题精选与解析	101
FRQ-1 (2021 International Q2 • SingleTable / CombinedTable)	101
FRQ-2 (2023 International Q2 • Sign)	105
FRQ-3 (2024 International Q2 • Scoreboard)	111
FRQ-4 (2025 International Q2 • SignedText)	115
六、应试技巧清单.....	121
MCQ 限时训练策略 (建议节奏)	121
FRQ 写作策略.....	121
代码追踪 (Tracing) 口诀	121
易错点自检清单.....	121
七、章节总结.....	122

第四章 Data Collections.....	124
一、本章定位与考情速览.....	125
1.1 新大纲四大单元与考试权重.....	125
1.2 考试结构.....	126
1.3 本章与 FRQ 的关联（备考重点提示）	127
1.4 本章学习目标（对应官方 17 个 Topic）	127
二、知识点与考点梳理（Topic 4.1 - 4.17）	128
板块 A：数据基础与伦理（Topic 4.1 / 4.2）	128
板块 B：1D Array 三件套（Topic 4.3 / 4.4 / 4.5）	129
板块 C：文本文件 + 包装类（Topic 4.6 / 4.7）	133
板块 D：ArrayList 三件套（Topic 4.8 / 4.9 / 4.10）	136
板块 E：2D Array 三件套（Topic 4.11 / 4.12 / 4.13）	141
板块 F：搜索、排序与递归（Topic 4.14 / 4.15 / 4.16 / 4.17）	145
三、本章重难点与易错点清单.....	149
易错点 #1：数组下标越界 vs 长度访问错位	149
易错点 #2：增强 for 误以为能修改元素	150
易错点 #3：ArrayList 遍历中删除的下标错位	150
易错点 #4：ArrayList 与数组互相转换时的强转	150
易错点 #5：2D 数组的 length 含义混淆	151
易错点 #6：Scanner.nextInt 后立刻 nextLine 读到空字符串	151
易错点 #7：ArrayList 误用 == 比较	151
易错点 #8：递归缺 base case 导致栈溢出.....	152
易错点 #9：二分搜索忘记前提「数组已排序」	152
易错点 #10：归并排序的 merge 阶段数组下标错误.....	152
四、MCQ 真题精选与详解	153
MCQ-1：1D 数组初始化与默认值（Unit 4 TopicQuestion MCQ #1）	153

MCQ-2: ArrayList 插入与替换的区别 (Unit 4 TopicQuestion MCQ #2)	154
MCQ-3: 2D 数组初始化语法 (Unit 4 TopicQuestion MCQ #11)	155
MCQ-4: ArrayList remove 后下标变化 (Unit 4 TopicQuestion MCQ #10)	155
MCQ-5: 数组长度不可变 (Unit 4 TopicQuestion MCQ #9)	156
MCQ-6: 包装类的自动装箱 (Unit 4 TopicQuestion MCQ #12)	157
MCQ-7: 2D 数组遍历的边界条件 (Unit 4 TopicQuestion MCQ #13)	158
MCQ-8: 增强 for 循环遍历 ArrayList (Unit 4 TopicQuestion MCQ #8)	159
MCQ-9: 2D 数组列数检查 (Unit 4 TopicQuestion MCQ #7)	160
MCQ-10: ArrayList 创建语法 (Unit 4 TopicQuestion MCQ #4)	161
MCQ-11: 增强 for 与传统 for 的差异 (Unit 4 TopicQuestion MCQ #5)	162
MCQ-12: ArrayList 搜索与 findValue 方法 (Unit 4 TopicQuestion MCQ #6)	163
五、FRQ 真题精选与解析	164
FRQ-1 (2021 International Q4 • ArrayResizer)	164
FRQ-2 (2022 International Q3 • ReviewAnalysis)	168
FRQ-3 (2023 International Q3 • WeatherData)	172
FRQ-4 (2024 International Q4 • GridPath)	174
FRQ-5 (2025 International Q4 • SumOrSameGame)	177
六、应试技巧清单	180
技巧 #1: 阅读题目时先标记「数组类型 + 遍历方向」	180
技巧 #2: ArrayList 遍历删除时优先用倒序	180
技巧 #3: 2D 数组遍历务必分清「行长度 vs 列长度」	181

技巧 #4: 二分搜索前必加注释「已排序」	181
技巧 #5: 递归题目先写 <code>base case</code> 再写 <code>recursive call</code>	181
技巧 #6: 写完方法后用题目给的示例验证.....	181
技巧 #7: <code>MCQ</code> 时间分配	181
七、章节总结.....	182

第一章 Using Objects and Methods

项目	内容
适用考试	AP Computer Science A 国际考（2025 秋新纲 • 2026 及以后）
课程章节	第一章 Using Objects and Methods（对应官方大纲 Unit 1）
考纲来源	College Board 《AP Computer Science A Course and Exam Description, Effective Fall 2025》
真题来源	2021 - 2025 年近五年 APCSA 国际考 FRQ 真题、2025 新版 CED 官方样题、AP Classroom 官方 Progress Check 试题

摘 要

本手册为 AP Computer Science A（APCSA）国际考备考系列的第一章，主题为 Using Objects and Methods，对应 College Board 2025 秋新版考纲 Unit 1（适用于 2026 年及以后考试）。手册按“考情速览—知识点梳理—易错点精讲—真题与解析—备考策略”五个部分组织：第一章系统梳理 15 个 Topic（Topic 1.1 - 1.15），覆盖 Java 程序基础、表达式与运算、方法、Math 类、对象与 String 五大板块，按 Learning Objective（LO）编号对照官方大纲；按失分频率排序 12 个高频易错点，给出正误对照与判断步骤；精选 15 道 MCQ（含 2025 新版 CED 官方样题与 AP Classroom 官方 Progress Check 试题）与 5 道 2021 - 2025 年国际考 FRQ 完整真题（每题含完整题面、官方参考解答与评分要点），覆盖考试结构 42 题 MCQ / 55% 与 4 题 FRQ / 45% 的最新分布。手册可作为学案随堂使用、考前冲刺复习与 AP Classroom 单元终结性测评的错题回炉对照资料。

关键词：AP Computer Science A；2025 新版考纲；Java 基础；String 方法；Math.random；FRQ 真题解析；国际考备考

目 录

[一、本章定位与考情速览](#)

[1.1 新大纲四大单元与考试权重](#)

1.2 考试结构
1.3 本章与 FRQ 的关联（备考重点提示）
二、知识点与考点梳理（Topic 1.1 - 1.15）
板块 A：Java 程序基础（Topic 1.1 - 1.2）
板块 B：表达式与运算（Topic 1.3 - 1.6）
板块 C：方法（Topic 1.7 - 1.10）
板块 D：Math 类（Topic 1.11）
板块 E：对象与 String（Topic 1.12 - 1.15）
三、重难点与易错点精讲
四、考纲对应的国际考真题与解析
4.1 MCQ 真题精选与解析
4.2 FRQ 真题精选与解析
五、备考策略与复习清单
5.1 复习优先级（按投入产出比排序）
5.2 考前自查清单
5.3 教师使用建议

一、本章定位与考情速览

本章是 APCS A 课程的开篇章节，对应官方大纲 Unit 1: Using Objects and Methods。内容覆盖 Java 程序的基本语法元素、表达式求值、方法的定义与调用、Math 类常用方法以及 String 类的核心 API。本章虽不涉及复杂的控制流与数据集合，但所学的“对象-方法-调用”思维模型贯穿全场考试，是后续 Unit 2（Selection and Iteration）、Unit 3（Class Creation）与 Unit 4（Data Collections）的基础。

1.1 新大纲四大单元与考试权重

2025 年秋起，APCS A 课程框架由旧的 10 个单元重组为 4 个单元，与本课程四个章节一一对应。选择题（MCQ）部分各单元权重如下：

单元（本课程章节）	MCQ 权重	建议课时
Unit 1：Using Objects and Methods（使用对象和方法）	15 - 25%	约 32 - 34 课时

Unit 2 : Selection and Iteration (选择与迭代)	25 - 35%	约 29 - 31 课时
Unit 3 : Class Creation (类的创建)	10 - 18%	约 20 - 22 课时
Unit 4 : Data Collections (数据集)	30 - 40%	约 50 - 52 课时

1.2 考试结构

考试整体由 Section I (MCQ) 与 Section II (FRQ) 两部分组成, 时长各 90 分钟, 分值占比与题量如下表所示。

部分	题量与时长	占比
Section I : 选择题 (MCQ, 含独立题与成组题)	42 题 / 90 分钟	55%
Section II : 自由作答题 (FRQ)	4 题 / 90 分钟	45%

1.3 本章与 FRQ 的关联 (备考重点提示)

根据新版考纲, FRQ 第 1 题 (Methods and Control Structures) 分为两部分:

- Part A (4 分): 要求编写含条件或循环的方法, 并调用题目所给类的方法;
- Part B (3 分): 明确要求调用 String 类的方法。

也就是说, 第一章的 String 方法 (length、substring、indexOf 等) 是 FRQ 的高频考点, 必须达到"闭卷熟练"的程度。2021 年国际考 FRQ1

(WordMatch) 的 scoreGuess / findBetterGuess 就是综合运用 String 方法的典型 (见本手册第四节 FRQ-5)。

格式说明

2021 - 2025 年国际考采用旧版 9 分制 FRQ1 (part (a) + part (b) 合计 9 分); 2026 年起改为新版 7 分制 (Part A 4 分 + Part B 3 分)。以下真题按年份倒序排列, 每题均给出完整题面、官方参考解答与评分要点。

此外, MCQ 中所有单元的题目都会大量阅读代码, 第一章培养的**代码追踪 (tracing) 能力**是全场考试的基础。

二、知识点与考点梳理 (Topic 1.1 - 1.15)

本章共 15 个 Topic，按知识结构分为五大板块。每个知识点均标注考纲 Learning Objective (LO) 编号，便于对照官方大纲复习。LO 编号格式为"单元号.Topic 号.知识点小标号"，例如 LO 1.1.A 表示 Unit 1、Topic 1.1 的第 A 小点。

板块 A：Java 程序基础 (Topic 1.1 - 1.2)

本板块聚焦程序设计的最基础概念：算法的描述方式、编译与执行流程、三类常见错误的区分，以及变量与三种基本数据类型的声明与使用。Topic 1.1 - 1.2 是整门课程的认知起点，MCQ 每年固定考查 1 - 2 道"哪种错误类型""哪种类型声明"的概念辨析题。

A1. 算法、编程与编译器 (Topic 1.1, LO 1.1.A - 1.1.C)

核心概念：

- **算法 (algorithm)**：完成任务或解决问题的逐步过程，可以用自然语言或图示表示 (LO 1.1.A)。
- **顺序执行 (sequencing)**：过程中的步骤按顺序、一次一步地完成。
- **编译与执行**：代码可在任何文本编辑器中编写；IDE 集成了编写、编译、运行工具。编译器会检查**部分**错误，编译器能发现的错误必须修复后程序才能运行 (LO 1.1.B)。
- **三类错误 + 异常** (LO 1.1.C，概念题高频考点)：

错误类型	英文	发现时机	特征
语法错误	syntax error	编译期	违反语言规则，编译器报错
逻辑错误	logic error	测试期	程序能运行但结果不符合预期
运行时错误	run-time error	运行期	通常导致程序异常终止
异常	exception	运行期	一类运行时错误，中断程序正常执行流程

记忆口诀：编译器抓语法，测试抓逻辑，运行抓异常。判断"哪种错误"是 MCQ 常见的概念题。

A2. 变量与数据类型 (Topic 1.2, LO 1.2.A - 1.2.B)

核心概念:

- **数据类型 (data type)** = 一组值 + 这组值上的操作。类型分为**基本类型** (primitive) 和**引用类型** (reference) 两大类 (LO 1.2.A)。
- 本课程只考三种基本类型 (LO 1.2.B.1): int (整数)、double (实数)、boolean (true/false)。
- **考纲排除项:** long、short、byte、float、char 不在考试范围内。
- 变量是存储位置, 有名字和类型; 基本类型变量直接存值。

```
int age = 16;           // 整数
double gpa = 3.85;      // 实数
boolean isPass = true;  // 布尔值
```

选择题考法: 给一个场景 (如"存储精确到 0.1 度的体温"), 选出正确的类型声明——精确到小数必须用 double, 用 int 会丢失精度。

板块 B: 表达式与运算 (Topic 1.3 - 1.6)

本板块是 MCQ 中输出值类题目的"重灾区", 每年固定考查 3 - 4 道。重点是理解 Java 的混合运算规则 (两个 int 运算仍为 int)、整数除法的截断、复合赋值运算符与自增运算符的语义, 以及强制类型转换的"截断而非四舍五入"特性。整除、cast 位置、运算优先级三者的组合是常见陷阱。

B1. 输出与字符串字面量 (Topic 1.3, LO 1.3.A - 1.3.B)

- System.out.print 输出后不换行; System.out.println 输出后换行 (LO 1.3.A.1)。
- **字面量 (literal):** 代码中固定值的表示。字符串字面量必须用双引号括起; 未加引号的 hello 是变量名而非字符串, 会引发编译错误。
- **转义序列:** \" (双引号)、\\ (反斜杠)、\n (换行) ——考试要求掌握这三种 (LO 1.3.B.3)。

```
System.out.print("cat ");
System.out.println("dog ");
System.out.println("horse ");
System.out.print("cow ");
// 输出:
// cat dog
```

```
// horse
// cow
```

B2. 算术表达式 (Topic 1.3, LO 1.3.C)

- 运算符：+、-、*、/、% (LO 1.3.C.2)。
- 混合运算规则 (本章第一高频考点)：
 - 两个 int 运算 → 结果仍是 int；
 - 至少一个 double 参与运算 → 结果为 double。
- 整数除法：两个 int 相除只保留商的整数部分，如 5 / 2 得 2，不是 2.5 (LO 1.3.C.3)。
- 取余 %：a % b 是 a 除以 b 的余数。考纲只需掌握 $a \geq 0$ 、 $b > 0$ 的情况 (LO 1.3.C.4)。
- 运算优先级 (LO 1.3.C.5)：*、/、% 高于 +、-；同级从左到右；括号可改变优先级。
- 整数除以 0 → ArithmeticException (LO 1.3.C.6)。double 参与的除零不在考试范围内。

```
System.out.println(7 / 2);           // 3    (int / int = int, 截断)
System.out.println(7.0 / 2);         // 3.5  (有 double 参与)
System.out.println(7 % 3);           // 1
System.out.println(1 + 2 * 3 % 4);   // 3    (先 2*3=6, 再 6%4=2, 再 1+2)
```

B3. 赋值语句与输入 (Topic 1.4, LO 1.4.A - 1.4.B)

本节重点是赋值与输入的边界：变量必须先声明再赋值；引用类型可赋值为 null；Scanner 是获取输入的一种方式，但"具体怎么读"不考。

- 变量必须先赋值才能使用；引用类型可赋值为对象或 null (LO 1.4.A.1)。
- = 是赋值：把右边表达式的值存入左边变量；右边先求值 (LO 1.4.A.2)。
- 表达式求值产生单一值，且该值有类型 (LO 1.4.A.3)。
- Scanner 类是获取键盘文本输入的一种方式；具体的输入方式不在考试范围内 (LO 1.4.B)。
- 考纲排除项：a = b = 4；这类"表达式内赋值"不考。

B4. 强制类型转换与数值范围 (Topic 1.5, LO 1.5.A - 1.5.C)

本节是 MCQ 中输出值类题目的另一高频来源。核心是 cast 的"截断"语义以及"四舍五入"必须靠手动加 0.5 实现的考点。

- (int) 和 (double) 转换符可实现类型转换 (LO 1.5.A.1)。
- **double 强转 int 是截断 (truncate)，不是四舍五入**：直接丢掉小数点右侧数字 (LO 1.5.A.2)。
- int 会自动拓宽 (widening) 为 double (LO 1.5.A.3)，如 double x = 3; 合法。
- **四舍五入技巧 (考纲明确要求) (LO 1.5.A.4)：**
 - 非负数：(int)(x + 0.5)
 - 负数：(int)(x - 0.5)
- int 用 4 字节存储，范围是 Integer.MIN_VALUE (约 -21.47 亿) 到 Integer.MAX_VALUE (约 21.47 亿)；超出范围发生**整数溢出 (overflow)**，结果仍在 int 范围内但不是期望值 (LO 1.5.B)。
- double 精度有限，超出会**舍入误差 (round-off error)**，例如 0.1 + 0.2 不精确等于 0.3；避免误差的方法是使用 int (LO 1.5.C)。

```
System.out.println((int) 3.99); // 3    (截断，不是 4)
System.out.println((int) -3.99); // -3   (向零截断)
System.out.println((int) (3.99 + 0.5)); // 4 (四舍五入技巧)
```

```
int big = Integer.MAX_VALUE;
System.out.println(big + 1); // -2147483648 (溢出环绕)
```

B5. 复合赋值运算符 (Topic 1.6, LO 1.6.A)

- +=、-=、*=、/=、%=：左值与右值做相应运算后赋回左值。
- x++、x-- (后缀形式)：加 1 或减 1 后赋回。
- **考纲排除项**：前缀 ++x 不考；arr[x++] 这类"在表达式内使用自增"不考。

```
int num = 5;
num *= 2; // num = 10
num %= 6; // num = 4
num++;    // num = 5
```

板块 C：方法 (Topic 1.7 - 1.10)

本板块是后续 FRQ 的"骨架"：写方法 = 读懂方法签名、确定参数列表、决定返回类型与 void 选择。MCQ 每年固定考查 1 - 2 道关于"哪个调用合法""void 与非 void 方法使用区别"的概念辨析题。

C1. API 与库 (Topic 1.7, LO 1.7.A)

本节是概念性的：理解 API 文档的作用、库（library）按包（package）组织，以及"属性 = 变量、行为 = 方法"在 FRQ 注释中反复出现的考点。

- 库（library）是类的集合；API 文档告诉程序员如何使用这些类。类库按包（package）组织。
- 属性（attributes）用变量存储数据；行为（behaviors）由方法定义（LO 1.7.A.2）——"属性 = 变量、行为 = 方法"是概念题考点。

C2. 注释与文档 (Topic 1.8, LO 1.8.A)

本节聚焦 Javadoc 中的前置/后置条件（precondition / postcondition）。FRQ 题干几乎必含 precondition 行，必须能正确读懂。

- 三种注释：// 单行、/* */ 多行块、/** */ Javadoc（生成 API 文档）。注释被编译器忽略。
- 前置条件（precondition）：方法执行前必须为真的条件，方法不负责检查前置条件是否满足。
- 后置条件（postcondition）：方法执行后必然为真的条件（返回值/对象状态）。
- 前置/后置条件的识别在 FRQ 题干中反复出现，务必能读懂注释。

C3. 方法签名 (Topic 1.9, LO 1.9.A - 1.9.B)

本节是 FRQ 写代码的入门：方法签名 = 方法名 + 有序参数类型列表；理解 void 与非 void 方法的使用差异；掌握方法重载。

- 方法签名 = 方法名 + 有序参数类型列表，如 `sqrt(double)`。无参方法签名含空参数列表。
- 参数（parameter）是方法头中声明的变量；实参（argument）是调用时传入的值，个数与类型必须与签名兼容，按值传递（call by value）传入副本。
- void 方法没有返回值，不能出现在表达式里，只能单独调用；非 void 方法返回一个值，必须存入变量或用于表达式中（LO 1.9.B.1 - 1.9.B.2）。
- 重载（overloading）：同名但签名不同的多个方法（LO 1.9.B.4）。
- 方法调用会中断顺序执行流程：先执行完方法体（或遇到 `return`），再回到调用点的下一条语句（LO 1.9.B.5）。
- 过程抽象（procedural abstraction）：知道方法做什么即可使用，无需知道如何实现（LO 1.9.A.1）。

C4. 调用类方法 (Topic 1.10, LO 1.10.A)

本节是 Math 类与 String 类的"调用语法"基础: static 方法用类名调用, 与对象无关。

- 类方法 (static 方法) 属于类本身而非对象, 方法头含 static 关键字。
- 调用格式: 类名.方法名(实参), 如 Math.sqrt(2.0)。在定义类内部调用时类名可省略。

板块 D: Math 类 (Topic 1.11)

本板块是 MCQ 与 FRQ 双线考查的核心。其中 Math.random() 区间公式是本章第二高频考点, 2024 年 FRQ1 (Feeder) 即直接考查 [10, 50] 区间的随机数生成。

D1. Math 类方法 (LO 1.11.A)

Math 类位于 java.lang 包, 默认可用, 只含类方法 (static)。以下五个方法在考试时提供 Java Quick Reference, 但必须会用:

方法	功能	示例
static int abs(int x)	int 绝对值	Math.abs(-5) → 5
static double abs(double x)	double 绝对值	Math.abs(-2.5) → 2.5
static double pow(double b, double e)	b 的 e 次方	Math.pow(2, 10) → 1024.0
static double sqrt(double x)	非负平方根	Math.sqrt(9.0) → 3.0
static double random()	返回 [0.0, 1.0) 随机数	见下方公式

D2. 随机数区间公式 (本章第二高频考点, LO 1.11.A.3)

Math.random() 返回大于等于 0.0 且小于 1.0 的 double。通过算术运算与强制转换可生成指定区间的随机整数:

[min, max] (双闭区间):

```
(int) (Math.random() * (max - min + 1)) + min
```

[min, max) (左闭右开):

```
(int)(Math.random() * (max - min)) + min
```

示例：生成 10 到 50（含）之间的随机整数：

```
(int)(Math.random() * 41) + 10    // 41 = 50 - 10 + 1
```

记忆要点：区间内整数的个数就是乘的系数；+ min 平移起点。MCQ 常给四个表达式让你判断"哪个能产生指定区间"。

板块 E：对象与 String（Topic 1.12 - 1.15）

本板块是 FRQ 重点考查的内容。对象（实例）的创建与方法是 MCQ 高频概念题；String 类的六个核心方法（length、substring、indexOf、equals、compareTo、+）是本章第三高频考点，也是 FRQ1 Part B 的必考内容。

E1. 对象与类的关系（Topic 1.12, LO 1.12.A - 1.12.B）

- **类是蓝图（blueprint），对象是类的实例（instance），有具体的属性值（LO 1.12.A.1）。**
- 所有类都是 Object 类的子类（LO 1.12.A.3）。继承关系的设计与实现不在考试范围，但要理解"子类可以复用超类属性和行为"。
- 引用类型变量存的是**对象引用（reference）**，可理解为对象的内存地址（LO 1.12.B.1）。
- 概念题考法："story 是 Book 类的一个对象"的正确表述是——**story 是 Book 类的实例，isHardcover 是 story 对象的属性。**

E2. 对象创建与存储（Topic 1.13, LO 1.13.A - 1.13.C）

本节是 MCQ 概念题的高频区。理解 new + 构造器调用的语法，以及"只声明不创建"时引用变量为 null 的关键事实。

- **构造器（constructor）：**与类同名、用于创建对象；构造器签名 = 类名 + 有序参数类型列表；构造器同样可以**重载**。
- 创建对象的语法：new + 构造器调用（LO 1.13.C.1）：

```
Date birthday = new Date("September", 5);  
//      ^类型      ^new      ^构造器("September", 5)为实参
```

- 只声明不创建时，引用变量为 null：Date d; 或 Date d = null;。
- 构造器实参必须与参数列表**类型兼容、顺序一致、个数相同**（LO 1.13.C.3）。
- 构造器参数用来初始化对象的属性（LO 1.13.C.2）。

E3. 调用实例方法 (Topic 1.14, LO 1.14.A)

- 实例方法必须通过对象调用：对象名.方法名(实参)（点运算符）。
- 在 `null` 引用上调用方法 → `NullPointerException` (LO 1.14.A.2, 异常类高频考点)。
- 区分记忆：static 方法用类名调用，实例方法用对象名调用。用类名调实例方法、或用对象名调 static 方法以外的错误形式都会导致编译错误。

E4. String 类 (Topic 1.15, LO 1.15.A - 1.15.B) ——本章第三高频考点

本节是 FRQ1 Part B 的必考内容，MCQ 每年固定考查 3 - 5 题。核心是 String 的不可变性（所有方法返回新串，不修改原串）以及六个必会方法的签名与陷阱。

创建与拼接 (LO 1.15.A)：

- String 可用字面量或构造器创建；位于 `java.lang` 包。
- String 是不可变的 (immutable)：所有 String 方法都返回新对象，绝不修改原串 (LO 1.15.A.3, 最易错点)。
- `+` / `+=` 拼接产生新 String；任何类型与 String 用 `+` 连接会被隐式转为 String；对象连接时会隐式调用其 `toString` 方法 (LO 1.15.A.4 - 1.15.A.5)。

六个必会方法 (LO 1.15.B.2, 均在 Java Quick Reference 中)：

方法	功能	易错点
<code>int length()</code>	字符个数	空串长度为 0
<code>String substring(int from, int to)</code>	返回 <code>[from, to-1]</code> 的子串	含头不含尾， <code>to</code> 本身不含
<code>String substring(int from)</code>	返回 <code>substring(from, length())</code>	等价于取到末尾
<code>int indexOf(String str)</code>	第一次出现 <code>str</code> 的下标，找不到返回 <code>-1</code>	返回 <code>-1</code> 不是异常
<code>boolean equals(Object other)</code>	内容相同返回 <code>true</code>	不能用 <code>==</code> 比较内容
<code>int compareTo(String other)</code>	按字典序：小于返回负数，相等返回 0，大于返回正数	只看符号，不看具体数值

- 下标范围：0 到 `length()-1`；越界访问 → `IndexOutOfBoundsException` (LO 1.15.B.1)。
- 取下标 `index` 处的单个字符：`substring(index, index + 1)` (LO 1.15.B.3)。

```
String str1 = "LMNOP";
System.out.println(str1.length());           // 5
System.out.println(str1.substring(3));        // "OP"      (下标 3 到末尾)
System.out.println(str1.substring(2, 3));      // "N"        (只含下标 2)
System.out.println(str1.indexOf("NO"));        // 2
System.out.println(str1.indexOf("X"));         // -1
System.out.println("apple".compareTo("banana")); // 负数
// 不可变性演示
String s = "abc";
s.substring(1);                               // 返回 "bc", 但 s 仍是 "abc"!
s = s.substring(1);                           // 必须接住返回值, s 才变成 "bc"
```

三、重难点与易错点精讲

以下 12 个易错点按"历年学生失分频率"排序，每条给出正误对照。**建议考前逐条自查**。易错点 1、2、5 是 MCQ 每年必考的"输出值类"题目；易错点 6、7、8 是"代码行为判断"类题目的常考点；易错点 9、10、11 则是 FRQ 中"接住返回值"和"子串边界"扣分的元凶。

易错点 1：整数除法悄悄丢小数（失分率最高）

```
double ans = 5 / 10;           // ✗ 结果是 0.0 ! (5/10 先按 int 算出 0)
double ans = (double) 5 / 10; // ✓ 0.5 (先把 5 转成 double)
```

规则：除法两侧都是 `int` → 结果一定是 `int`。变量已经声明为 `double` 也救不了右侧的表达式。修法：**cast 任意一个操作数**即可（两个都 `cast` 也可以，但 `cast` 整个商太晚）。

易错点 2：cast 的位置——太早、太晚都会错

```
int b = (int) (a / 2); // a=7.0 → (int)3.5 = 3    先除后截断
int b = (int) a / 2;   // a=7.0 → 7 / 2 = 3      先截断再除 (此处结果相同纯属巧合)
```

`double avg = (double) (sum / 2);` // ✕ 太晚! `sum/2` 已按 `int` 截断, `cast` 只是换壳
`double avg = (double) sum / 2;` // ✓ 先转 `double` 再除

判断步骤: 先按运算优先级确定每一步的类型, 再看 `cast` 作用在哪一步。

易错点 3: 截断 ≠ 四舍五入, 负数向零截断

`(int) 3.99` → 3
`(int) -3.99` → -3 // 向零截断, 不是 -4

四舍五入必须手动加 0.5: 非负数 `(int) (x + 0.5)`, 负数 `(int) (x - 0.5)`。注意 `(int) (-0.2 - 0.5) = (int) (-0.7) = 0`, 不要想当然写 -1。

易错点 4: 运算优先级——% 与 * / 同级

`1 + 2 * 3 % 4` 的计算顺序: `2*3=6` → `6%4=2` → `1+2=3`。% 的优先级与 *、/ 相同, 从左到右。写成"先加后模"必错。

易错点 5: Math.random 区间公式记错端点

- `[0.0, 1.0): Math.random()`
- `[10, 50]: (int) (Math.random() * 41) + 10` (乘"个数"41, 加"起点"10)
- `[10, 50): (int) (Math.random() * 40) + 10`

MCQ 惯用陷阱: 给的选项里有一个端点多乘了 1 或少乘了 1, 用"区间内整数个数"验证。

易错点 6: void 方法混入表达式

```
public void totalAmount() {  
    System.out.print(purchase + tax);  
}  
  
System.out.print(p.totalAmount()); // ✕ 编译错误: void 不能用于表达式  
p.totalAmount(); // ✓ void 方法单独调用
```

反向陷阱: 非 `void` 方法只调用不接收, 返回值被丢弃——编译能过, 但输出往往是空 (题目问"打印什么"时容易漏看)。

易错点 7: static 与实例方法调错对象

```
MagicNumber n1 = new MagicNumber();  
MagicNumber.add_2(); // ✗ 实例方法用类名调用 → 编译错误  
n1.add_2();          // ✓
```

判断标准：方法头有没有 `static`。有 → 类名调用（如 `Math.abs`）；没有 → 必须先 `new` 对象再用对象名调用。

易错点 8: null 引用调用方法

```
Thing t = null;  
t.setColor("Green"); // 运行时 NullPointerException
```

`NullPointerException` 是运行时异常，不是编译错误——"这段代码会怎样"的选项里两者要分清。

易错点 9: String 不可变，忘记接住返回值

所有 `String` 方法都返回新串。`s.substring(1)`；单独一行对 `s` 毫无影响；必须 `s = s.substring(1)`；FRQ 中忘记赋回是常见扣分点。

易错点 10: substring 含头不含尾 + 越界

- `substring(2, 5)` 取下标 2、3、4，共 3 个字符。
- 下标有效范围 `0 ~ length()-1`；`substring` 的第二个参数最大到 `length()`（取到末尾），再大就越界。
- 越界 → `IndexOutOfBoundsException`（运行时异常）。

易错点 11: equals / compareTo 的返回值

- 比较内容必须 `s1.equals(s2)`；`==` 比较的是引用，内容相同也可能为 `false`（考试默认用 `equals`）。
- `compareTo` 返回值只看符号：负数、0、正数分别对应小于、等于、大于；不要假设返回 `-1/1`。

易错点 12：构造器重载的匹配

构造器按"参数类型序列"匹配，允许 int 自动拓宽为 double（new Purchase(1.5, 2) 中 $2 \rightarrow 2.0$ 合法），但 double 不能自动变 int。判断"哪个新构造器头合法"时，先写下已有构造器的签名，**签名完全相同的才是非法重载**。

四、考纲对应的国际考真题与解析

以下 MCQ 精选 15 题：第 1 - 4 题为 2025 新版 CED 官方样题（答案经官方 Answer Key 核对）；第 5 - 15 题为 College Board 官方 Progress Check 试题。FRQ 精选 5 题：2021 - 2025 年 APCSA 国际考 Question 1 完整真题，按年份倒序排列（FRQ-1 = 2025，FRQ-5 = 2021）。每题标注考点（对应第二节知识点编号），解析给出完整推理过程。

4.1 MCQ 真题精选与解析

本节 15 道 MCQ 覆盖 A、B、C、D、E 五大板块的高频考点：整除与 cast、随机数区间、构造器调用、String 方法、static/实例方法、void 与非 void 方法签名、equals/compareTo 等。建议限时训练（15 题控制在 20 分钟内）。

MCQ-1 CED 官方样题 • 考点：B4 强制转换

考虑以下代码段。

```
double q = 15.0;
int r = 2;

double x = (int) (q / r);
double y = q / r;
System.out.println(x + " " + y);
```

执行该代码段后打印的结果是什么？

- (A) 7.0 7.0
- (B) 7.0 7.5
- (C) 7.5 7.0
- (D) 7.5 7.5

解析： q / r ：double 与 int 相除 $\rightarrow 15.0 / 2 = 7.5$ 。x：(int) (7.5) 截断为 7，再存入 double 得 7.0。y：无 cast，直接 7.5。输出 7.0 7.5。答案：**B**

MCQ-2 CED 官方样题 • 考点：D2 随机数区间

考虑以下代码段。

```
int x = ((int) (Math.random() * 10)) + 5;
int y = ((int) (Math.random() * 5)) + 10;
System.out.println(x + " " + y);
```

执行该代码段后，以下哪项可能被打印？

- (A) 8 15
- (B) 9 12
- (C) 10 7
- (D) 15 11

解析：x 的取值范围： $(\text{int})([0,10)) + 5 = [5, 14]$ ；y 的范围： $(\text{int})([0,5)) + 10 = [10, 14]$ 。逐项检查：(A) y=15 越界；(C) y=7 越界；(D) x=15 越界；(B) x=9、y=12 都在范围内。答案：**B**

MCQ-3 CED 官方样题 · 考点：E2 构造器调用

考虑以下 Date 类。

```
public class Date{
    private String month;
    private int day;

    public Date(String m, int d)
    { /* implementation not shown */ }

    /* Class contains no other constructors.
       Other methods are not shown. */
}
```

在 Date 以外的类中，以下哪段代码能正确创建一个 Date 对象的实例？

- (A) `Date birthday = new Date("September", "5th");`
- (B) `Date birthday = new Date("September", 5);`
- (C) `Date birthday = new Date("September 5");`
- (D) `Date birthday = new Date();`
`birthday.month = "September";`
`birthday.day = 5;`

解析：类中唯一的构造器签名是 `Date(String, int)`，实参必须"个数 2、类型依次 String、int"。(A) 第二个实参是 String；(C) 只有一个 String 参数；(D) 无参构造器不存在，且从类外直接访问 private 属性非法。答案：**B**

MCQ-4 CED 官方样题 • 考点: E4 String 方法

考虑以下代码段。

```
String str1 = "LMNOP";  
String str2 = str1.substring(3);  
str2 += str1.substring(2, 3);
```

执行该代码段后 str2 的值是什么?

- (A) "OPN"
- (B) "OPNO"
- (C) "NOPM"
- (D) "NOPMN"

解析: str1.substring(3) 从下标 3 取到末尾 → "OP"。str1.substring(2, 3) 含头不含尾, 只有下标 2 一个字符 → "N"。拼接: "OP" + "N" = "OPN"。答案: **A**

MCQ-5 Progress Check 官方题 • 考点: B2 运算优先级

考虑以下代码段。

```
int a = 1;  
int b = 2;  
int c = 3;  
int d = 4;  
double x = a + b * c % d;
```

代码段执行后 x 的值是多少?

- (A) 1.0 (B) 2.5 (C) 3.0 (D) 7.0 (E) 9.0

解析: * 与 % 同级, 从左到右: $b * c = 6 \rightarrow 6 \% d = 2 \rightarrow a + 2 = 3$ 。int 结果 3 自动拓宽存入 double, $x = 3.0$ 。答案: **C**

MCQ-6 Progress Check 官方题 • 考点: B2 整数除法

考虑以下代码段。

```
int x = 10;  
int y = 20;  
System.out.print(y + x / y);
```

打印结果是什么?

- (A) 1 (B) 1.5 (C) 3 (D) 20 (E) 20.5

解析：先算 x / y : `int` 除 `int` $\rightarrow 10 / 20 = 0$ (截断)。再算 $y + 0 = 20$ 。
注意不是从左到右先算 $y + x$ 。答案：**D**

MCQ-7 Progress Check 官方题 • 考点: B2 整数除法与取余

以下哪些算术表达式的值为 1?

- I. $2 / 5 \% 3$
- II. $2 / (5 \% 3)$
- III. $2 / 5 + 1$

- (A) 只有 I (B) 只有 II (C) I 和 II (D) II 和 III (E) I、II 和 III

解析：I: $2/5=0$, $0\%3=0$, 不是 1。II: $5\%3=2$, $2/2=1$ ✓。III: $2/5=0$, $0+1=1$ ✓。
答案：**D**

MCQ-8 Progress Check 官方题 • 考点: B4/B5 复合赋值

考虑以下代码段。

```
double avg = 15 + 20;  
avg /= 2;
```

以下哪项最好地描述了代码段的行为?

- (A) 将 17 存入 `avg`, 因为 17 是 35 除以 2 整数除法的结果
- (B) 将 18 存入 `avg`, 因为 18 是 35 除以 2 整数除法的结果
- (C) 将 17.5 存入 `avg`, 因为 17.5 是 35.0 除以 2 浮点除法的结果
- (D) 代码不编译, 因为 `int` 值不能加到 `double` 变量
- (E) 代码不编译, 因为 `double` 不能除以 `int`

解析： $15 + 20$ 赋给 `double avg` 时自动拓宽为 35.0; $avg /= 2$ 即 $35.0 / 2$, 浮点除法得 17.5。答案：**C**

MCQ-9 Progress Check 官方题 • 考点: B4 截断与四舍五入技巧

考虑以下代码段。

```
double p = 10.6;  
double n = -0.2;  
System.out.println((int) (p + 0.5));  
System.out.print((int) (n - 0.5));
```

打印结果是什么?

- (A) 10 换行 -1 (B) 10 换行 0 (C) 11 换行 -1 (D) 11 换行 0 (E) 11 换行 1

解析：第一行: $p+0.5 = 11.1$, $(int)11.1$ 截断 $\rightarrow 11$ 。第二行: $n-0.5 = -0.7$, 向零截断 $\rightarrow 0$, 不是 -1。答案：**D**

MCQ-10 Progress Check 官方题 • 考点: B4 整数除法排错

以下代码段想显示 0.5。

```
int num1 = 5;
int num2 = 10;
double ans = num1 / num2;
System.out.print(ans);
```

以下哪项最好地描述了代码段中的错误（如果有）？

- (A) 没有错误，代码按预期工作
- (B) 应该把表达式 `num1 / num2` 整体 cast 为 `double`
- (C) 应该把 `num1` 或 `num2` 其中一个 cast 为 `double`
- (D) 应该把 `ans` 声明为 `int`
- (E) 应该把 `num1` 和 `num2` 初始化为 5.0 和 10.0

解析：`num1 / num2` 两侧都是 `int`，先算出 0，再存入 `double` 得 0.0。(B)

(`double`) (`num1/num2`) cast 太晚，无用。(C) cast 任一操作数即可让除法按 `double` 进行。答案：C

MCQ-11 Progress Check 官方题 • 考点: E1 对象与引用

考虑以下类声明。

```
public class Thing{
    private String color;
    public Thing()
    { color = "Blue"; }
    public Thing(String setColor)
    { color = setColor; }
}
```

在 `Thing` 以外的类中，以下哪段代码会创建一个值为 `null` 的 `Thing` 类型引用？

- (A) `Thing something = new Thing("Green");`
- (B) `Thing something = new Thing("");`
- (C) `Thing something = new Thing();`
- (D) `Thing something;`
- (E) `Thing("Green");`

解析：只声明不初始化的引用变量默认为 `null`。(A)(C) 用构造器创建了对象；

(B) 空串不是 `null`；(E) 缺少变量声明和 `new`，语法错误。答案：D

MCQ-12 Progress Check 官方题 • 考点: E3 实例方法调用

考虑以下类。

```
public class MagicNumber{
    private int num;
    public MagicNumber()
    { num = 10; }
    public void displayNumber()
    { System.out.println(num); }
    public void add_2()
    { num = num + 2; }
}
```

在其他类的方法中，以下哪段代码能编译通过？

- I. MagicNumber.add_2();
 MagicNumber.displayNumber();
- II. MagicNumber n1 = new MagicNumber();
 n1.add_2();
 n1.displayNumber();
- III. n2.add_2();
 n2.displayNumber();

- (A) 只有 I (B) 只有 II (C) 只有 III (D) II 和 III (E) 都不能编译

解析：add_2 和 displayNumber 是**实例方法**（无 static），必须用对象名调用。

I 用类名调实例方法，编译错误；II 先 new 对象再调用，正确；III 中 n2 从未声明，编译错误。答案：**B**

MCQ-13 Progress Check 官方题 • 考点: C3 方法签名与类型兼容

考虑以下方法。

```
public double secret(int x, double y)
{ return x / 2.0; }
```

在与 secret 同一个类的方法中，以下哪行代码能编译通过？

- (A) int result = secret(4, 4);
- (B) int result = secret(4, 4.0);
- (C) double result = secret(4, 4.0);
- (D) double result = secret(4.0, 4);
- (E) double result = secret(4.0, 4.0);

解析：形参 (int, double)：第一个实参必须是 int 兼容，第二个是 double (int 可自动拓宽)。调用侧没有问题的是 (A)(B)(C)，但返回类型是 double——(A)(B) 把 double 赋给 int 变量，编译错误。(D)(E) 第一个实参 4.0 是 double，不能自动变 int。答案：C

MCQ-14 Progress Check 官方题 • 考点：C3 void 方法与实参传递

考虑以下方法。

```
public void printSomething(int num, boolean val){
    num--;
    System.out.print(val);
    System.out.print(num);
}
```

在与 printSomething 同一个类的方法中执行以下代码段，打印结果是什么？

```
printSomething(1, true);
printSomething(2, true);
```

- (A) 0true1true
- (B) 1true2true
- (C) true0true1
- (D) true1true0
- (E) true1true2

解析：每次调用 num 是实参的副本 (call by value)。第一次：num=1 减 1 得 0，先打印 val 再打印 num → "true0"；第二次同理 → "true1"。拼接为 "true0true1"。
答案：C

MCQ-15 Progress Check 官方题 • 考点：C3 非 void 方法返回值的使用

考虑以下方法，它把华氏温度转换为摄氏温度。

```
public double fahrenheitToCelsius(double f){
    double c = (f - 32) * 5 / 9;
    return c;
}
```

在与 fahrenheitToCelsius 同一个类的方法中，以下哪段代码能打印 32 华氏度对应的摄氏温度？

- (A) double f = 32.0; fahrenheitToCelsius(); System.out.println(f);
- (B) double f = 32.0; f = fahrenheitToCelsius();
System.out.println(f);

- (C) `double f = 32.0; fahrenheitToCelsius(f);`
`System.out.println(f);`
- (D) `double f = 32.0; double c = fahrenheitToCelsius();`
`System.out.println(c);`
- (E) `double f = 32.0; double c = fahrenheitToCelsius(f);`
`System.out.println(c);`

解析：方法需要一个 **double 实参**且返回值要**接住**。(A)(D) 无实参调用，编译错误；(B) 无实参；(C) 调用正确但返回值被丢弃，打印的仍是 32.0（华氏）。(E) 实参正确、返回值存入 c 再打印。答案：**E**

4.2 FRQ 真题精选与解析

本节 5 道 FRQ 覆盖 2021 - 2025 年近五年国际考 Question 1 的完整真题，题面、官方参考解答、评分要点齐全。建议手写后对照评分要点互评，模拟真实阅卷视角。

FRQ-1 【2025 年国际考真题 · Question 1】DogWalker

考点：E2 对象创建与交互、E3 调用实例方法、C3 方法调用流程、第二章条件与循环

完整题面：

本题涉及通过遛狗公司付费遛狗服务。一家遛狗公司在一天中的每个小时都有不同数量的狗需要遛。DogWalkCompany 类定义如下。

```
public class DogWalkCompany{
    /**
     * Returns the number of dogs, always greater than 0, that
are available
     * for a walk during the time specified by hour
     * Precondition: 0 <= hour <= 23
     */
    public int numAvailableDogs(int hour)
    { /* implementation not shown */ }

    /**
     * Decreases, by numberDogsWalked, the number of dogs
available for a walk
     * during the time specified by hour
     * Preconditions: 0 <= hour <= 23
     *                  numberDogsWalked > 0
     */
}
```

```

    public void updateDogs(int hour, int numberDogsWalked)
    { /* implementation not shown */ }

    /* There may be instance variables, constructors,
       and methods that are not shown. */
}

```

一名遛狗员隶属于一家遛狗公司，用 `DogWalker` 类表示。你需要完成该类中的两个方法。

```

public class DogWalker
{
    /** The maximum number of dogs this walker can walk
    simultaneously per hour */
    private int maxDogs;
    /** The dog-walking company this dog walker is associated
    with */
    private DogWalkCompany company;

    /**
     * Assigns max to maxDogs and comp to company
     * Precondition: max > 0
     */
    public DogWalker(int max, DogWalkCompany comp)
    { /* implementation not shown */ }

    /**
     * Takes at least one dog for a walk during the time
    specified by hour,
     * as described in part (a)
     * Preconditions: 0 <= hour <= 23
     *                 maxDogs > 0
     */
    public int walkDogs(int hour)
    { /* to be implemented in part (a) */ }

    /**
     * Performs an entire dog-walking shift and returns the
    amount earned,
     * in dollars, as described in part (b)

```

```

    * Preconditions: 0 <= startHour <= endHour <= 23
    *
    *               maxDogs > 0
    */
public int dogWalkShift(int startHour, int endHour)
{ /* to be implemented in part (b) */ }

/* There may be instance variables, constructors,
   and methods that are not shown. */
}

```

(a)

编写 `walkDogs` 方法：更新并返回该遛狗员在 `hour` 时段遛的狗的数量。`hour` 取值范围是 0 到 23（含）。

`DogWalkCompany` 类已提供辅助方法 `numAvailableDogs`，返回指定小时可遛的狗数。遛狗员总是尽可能多遛——只要公司可用狗数不超过 `maxDogs`，就全遛；超过则只遛 `maxDogs` 只。`DogWalkCompany` 类还提供 `updateDogs` 方法，用于将指定小时的可遛数量减少相应值，以避免多名遛狗员重复认领同一批狗。

例如，若某小时公司有 10 只狗需遛，而 `maxDogs` 为 4，则应调用 `updateDogs` 表明该遛狗员遛 4 只；若公司有 3 只而 `maxDogs` 为 4，则表明遛 3 只。方法返回该遛狗员本次遛的狗数。

(b)

编写 `dogWalkShift` 方法：完成从 `startHour` 到 `endHour`（含）的一整个遛狗班次，并返回总收入（美元）。例如，从 14 时到 16 时的班次包含 14、15、16 三个小时的遛狗。

每小时的底薪是 5 美元/只，外加 3 美元奖金，当且仅当满足以下至少一个条件：

- 该小时遛了 `maxDogs` 只狗；
- 该小时在 **peak hours** 9 - 17（含）之间。

下表展示了从 7 时到 10 时（含）的一个示例。

Hour	Maximum Number of Dogs	Number of Dogs Walked	Amount Earned, in Dollars
7	3	3	$3 \times 5 + 3 = 18$
8	3	2	$2 \times 5 = 10$

9	3	2	$2 \times 5 + 3 =$
			13
10	3	3	$3 \times 5 + 3 =$
			18
Total			59

官方参考解答：

```
// (a)
public int walkDogs(int hour)
{
    int dogsToWalk = company.numAvailableDogs(hour);
    if (dogsToWalk > maxDogs)
    {
        dogsToWalk = maxDogs;
    }
    company.updateDogs(hour, dogsToWalk);
    return dogsToWalk;
}

// (b)
public int dogWalkShift(int startHour, int endHour)
{
    int totalPay = 0;
    for (int hour = startHour; hour <= endHour; hour++)
    {
        int dogs = walkDogs(hour);
        int hourPay = 5 * dogs;
        if (dogs == maxDogs || (hour >= 9 && hour <= 17))
        {
            hourPay += 3;
        }
        totalPay += hourPay;
    }
    return totalPay;
}
```

解析与评分要点：

- Part (a) 共 4 分：调用 company 的辅助方法；比较可用狗数与 maxDogs；以正确参数调用 updateDogs；返回计算值。
- Part (b) 共 5 分：正确遍历 startHour 到 endHour；在循环中调用 walkDogs 并使用返回值；计算每小时底薪；正确判断奖金条件（注意两个条件只要满足其一即可，不能要求同时满足）；累加并返回总收入。
- 关键易错点：updateDogs 是 void 方法，不能赋值；walkDogs 返回的 dogs 必须用于计算薪酬，不要重复调用导致公司数据被重复扣减。

FRQ-2【2024 年国际考真题 · Question 1】Feeder

考点：D2 Math.random 区间公式、B4 强制转换、第二章 if/循环

完整题面：

本题模拟鸟或熊在鸟食器取食的过程。Feeder 类声明如下。

```
public class Feeder
{
    /**
     * The amount of food, in grams, currently in the bird feeder;
     * initialized in the constructor and always greater than or
    equal to zero
     */
    private int currentFood;

    /**
     * Simulates one day with numBirds birds or possibly a bear
    at the bird feeder,
     * as described in part (a)
     * Precondition: numBirds > 0
     */
    public void simulateOneDay(int numBirds)
    { /* to be implemented in part (a) */ }

    /**
     * Returns the number of days birds or a bear found food to
    eat at the feeder
     * in this simulation, as described in part (b)
     * Preconditions: numBirds > 0, numDays > 0
     */
    public int simulateManyDays(int numBirds, int numDays)
```

```

    { /* to be implemented in part (b) */ }

    // There may be instance variables, constructors, or methods
    that are not shown.
}

```

(a)

编写 `simulateOneDay` 方法：模拟 `numBirds` 只鸟或一只熊在一天内取食，并更新 `currentFood`。

正常情况占 95%：只有鸟来觅食，每只鸟当天吃同样数量的食物，该数量在 10 到 50 克之间（含端点，以 1 克为增量，等概率随机选取）。若当天总共要吃掉的食物超过 `currentFood`，则鸟食器被吃空，`currentFood` 置 0。

异常情况占 5%：一只熊清空食器，`currentFood` 置 0。

(b)

编写 `simulateManyDays` 方法：调用 `simulateOneDay` 模拟最多连续 `numDays` 天。每天只要鸟或熊能找到食物（即调用前 `currentFood > 0`）就算“找到食物”一天。返回实际找到食物的天数。

示例：

- `currentFood` 为 2400，调用 `simulateManyDays(10, 4)`，四天后都还有食物，返回 4。
- `currentFood` 为 250，调用 `simulateManyDays(10, 5)`，第二天结束后 `currentFood` 变为 0，返回 2。
- `currentFood` 为 0，调用 `simulateManyDays(5, 10)`，直接返回 0。

官方参考解答：

```

// (a)
public void simulateOneDay(int numBirds)
{
    double condition = Math.random();
    if (condition < 0.05)
    {
        currentFood = 0;
    }
    else
    {
        int eachBirdEats = (int) (Math.random() * 41) + 10;

```

```

        int totalEaten = numBirds * eachBirdEats;
        if (totalEaten > currentFood)
        {
            currentFood = 0;
        }
        else
        {
            currentFood -= totalEaten;
        }
    }
}

// (b)
public int simulateManyDays(int numBirds, int numDays)
{
    for (int daysSoFar = 0; daysSoFar < numDays; daysSoFar++)
    {
        if (currentFood == 0)
        {
            return daysSoFar;
        }
        simulateOneDay(numBirds);
    }
    return numDays;
}

```

解析与评分要点:

- **Part (a)** 共 4 分: 调用 `Math.random` 生成随机值; 用比较实现 5% / 95% 分支; 生成 [10, 50] 均匀随机整数; 按鸟数缩放并正确更新 `currentFood` (包括吃空归 0)。
- **Part (b)** 共 5 分: 在循环中调用 `simulateOneDay(numBirds)`; 控制循环最多执行 `numDays` 次; 统计找到食物的天数; 在 `currentFood == 0` 时正确提前退出; 返回天数。
- 关键易错点: 随机区间 `(int)(Math.random() * 41) + 10` 中的 41 是 50 - 10 + 1; 比较 `totalEaten > currentFood` 与 `>=` 在本题中均不影响归 0, 但官方样例使用 `>`。

FRQ-3【2023 年国际考真题 · Question 1】AppointmentBook

考点：C3 方法调用、第二章循环与条件、过程抽象

完整题面：

本题涉及 AppointmentBook 类，用于学生预约与老师见面。预约只能在第 1 到第 8 节课之间进行，每节课 60 分钟，分钟编号为 0 到 59。要成功预约，老师必须在指定节课中有至少连续 duration 分钟的空闲时间。预约必须开始并结束于同一节课内。

```
public class AppointmentBook
{
    /**
     * Returns true if minute in period is available for an
appointment and returns
     * false otherwise
     * Preconditions: 1 <= period <= 8; 0 <= minute <= 59
     */
    private boolean isMinuteFree(int period, int minute)
    { /* implementation not shown */ }

    /**
     * Marks the block of minutes that starts at startMinute in
period and
     * is duration minutes long as reserved for an appointment
     * Preconditions: 1 <= period <= 8; 0 <= startMinute <= 59;
     *                  1 <= duration <= 60
     */
    private void reserveBlock(int period, int startMinute, int
duration)
    { /* implementation not shown */ }

    /**
     * Searches for the first block of duration free minutes
during period, as described in
     * part (a). Returns the first minute in the block if such a
block is found or returns -1 if no
     * such block is found.
     * Preconditions: 1 <= period <= 8; 1 <= duration <= 60
     */
    public int findFreeBlock(int period, int duration)
```

```

{ /* to be implemented in part (a) */ }

/**
 * Searches periods from startPeriod to endPeriod, inclusive,
for a block
 * of duration free minutes, as described in part (b). If
such a block is found,
 * calls reserveBlock to reserve the block of minutes and
returns true; otherwise
 * returns false.
 * Preconditions: 1 <= startPeriod <= endPeriod <= 8; 1 <=
duration <= 60
 */
public boolean makeAppointment(int startPeriod, int endPeriod,
                               int duration)
{ /* to be implemented in part (b) */ }

// There may be instance variables, constructors, and methods
that are not shown.
}

```

(a)

编写 `findFreeBlock` 方法：在指定节课 `period` 中搜索第一个长度至少为 `duration` 的连续空闲分钟块。若找到，返回该块的起始分钟；否则返回 -1。必须使用 `isMinuteFree`。

示例（第 2 节课的空闲情况）：

Minutes in Period 2	Available?
0 - 9 (10 minutes)	No
10 - 14 (5 minutes)	Yes
15 - 29 (15 minutes)	No
30 - 44 (15 minutes)	Yes
45 - 49 (5 minutes)	No
50 - 59 (10 minutes)	Yes

- `findFreeBlock(2, 15)` 返回 30。

- findFreeBlock(2, 9) 也返回 30（取最早起始）。
- findFreeBlock(2, 20) 返回 -1。

(b)

编写 makeAppointment 方法：从 startPeriod 到 endPeriod（含）依次查找最早的、长度为 duration 的空闲块。若找到，调用 reserveBlock 标记该块为已预约并返回 true；否则返回 false。

官方参考解答：

```
// (a)
public int findFreeBlock(int period, int duration)
{
    int blockLength = 0;
    for (int minute = 0; minute < 60; minute++)
    {
        if (isMinuteFree(period, minute))
        {
            blockLength++;
            if (blockLength == duration)
            {
                return minute - blockLength + 1;
            }
        }
        else
        {
            blockLength = 0;
        }
    }
    return -1;
}

// (b)
public boolean makeAppointment(int startPeriod, int endPeriod,
                               int duration)
{
    for (int period = startPeriod; period <= endPeriod; period++)
    {
        int minute = findFreeBlock(period, duration);
        if (minute != -1)
```

```

        {
            reserveBlock(period, minute, duration);
            return true;
        }
    }
    return false;
}

```

解析与评分要点:

- Part (a) 共 5 分: 遍历一节课的 60 分钟; 正确调用 `isMinuteFree(period, minute)`; 维护连续空闲分钟计数; 判断是否达到 `duration`; 正确返回起始分钟或 -1。
- Part (b) 共 4 分: 遍历从 `startPeriod` 到 `endPeriod`; 调用 `findFreeBlock` 与 `reserveBlock`; 用返回值 -1 作为是否预约的判断; 正确返回布尔值。
- 关键易错点: `findFreeBlock` 返回的是起始分钟, 不是结束分钟; `reserveBlock` 的参数顺序是 `(period, startMinute, duration)`, 不要写反。

FRQ-4【2022 年国际考真题 · Question 1】Game

考点: E3 调用实例方法、第二章嵌套 if/循环

完整题面:

本题模拟一款单人视频游戏的游玩与计分。玩家需要完成三个关卡。关卡由 `Level` 类表示。

```

public class Level
{
    /** Returns true if the player reached the goal on this level
    and returns false otherwise */
    public boolean goalReached()
    { /* implementation not shown */ }

    /** Returns the number of points (a positive integer)
    recorded for this level */
    public int getPoints()
    { /* implementation not shown */ }

    // There may be instance variables, constructors, and methods

```

```
that are not shown.  
}
```

游戏过程由 Game 类表示。你需要完成其中两个方法。

```
public class Game  
{  
    private Level levelOne;  
    private Level levelTwo;  
    private Level levelThree;  
  
    /** Postcondition: All instance variables have been  
    initialized. */  
    public Game()  
    { /* implementation not shown */ }  
  
    /** Returns true if this game is a bonus game and returns  
    false otherwise */  
    public boolean isBonus()  
    { /* implementation not shown */ }  
  
    /** Simulates the play of this Game (consisting of three  
    levels) and updates all relevant game data */  
    public void play()  
    { /* implementation not shown */ }  
  
    /** Returns the score earned in the most recently played game,  
    as described in part (a) */  
    public int getScore()  
    { /* to be implemented in part (a) */ }  
  
    /**  
    * Simulates the play of num games and returns the highest  
    score earned,  
    * as described in part (b)  
    * Precondition: num > 0  
    */  
    public int playManyTimes(int num)  
    { /* to be implemented in part (b) */ }
```

```
// There may be instance variables, constructors, and methods
that are not shown.
}
```

(a)

编写 `getScore` 方法：返回最近一局游戏的得分。计分规则如下：

- 只有第一关目标达成，才能获得第一关的分数；
- 只有第一关和第二关目标都达成，才能获得第二关的分数；
- 只有三关目标都达成，才能获得第三关的分数；
- 若该局是 `bonus game`，则总得分乘以 3。

示例得分计算表：

goalReached (三关结果)	isBonus	getPoints (三关分数假设 为 200, 100, 500)	得分
true, true, true	true	200, 100, 500	$(200 + 100 + 500) \times 3 = 2,400$
true, true, false	false	200, 100, 500	$200 + 100 = 300$
true, false, true	true	200, 100, 500	$200 \times 3 = 600$
false, true, true	false	200, 100, 500	0

(b)

编写 `playManyTimes` 方法：模拟 `num` 局游戏，返回最高得分。游戏通过调用 `play()` 模拟；每次调用 `play()` 后可通过 `getScore()` 获取该局得分。

官方参考解答：

```
// (a)
public int getScore()
{
    int score = 0;
    if (levelOne.goalReached())
    {
        score = levelOne.getPoints();
    }
}
```

```

        if (levelTwo.goalReached())
        {
            score += levelTwo.getPoints();
            if (levelThree.goalReached())
            {
                score += levelThree.getPoints();
            }
        }
    }
    if (isBonus())
    {
        score *= 3;
    }
    return score;
}

// (b)
public int playManyTimes(int num)
{
    int max = 0;
    for (int i = 0; i < num; i++)
    {
        play();
        int score = getScore();
        if (score > max)
        {
            max = score;
        }
    }
    return max;
}

```

解析与评分要点:

- **Part (a) 共 4 分:** 调用 `getPoints`、`goalReached`、`isBonus`；根据 `goalReached` 返回值判断哪些关卡分数应计入；根据 `isBonus` 将总分乘以 3；正确初始化并累加得分。
- **Part (b) 共 5 分:** 循环 `num` 次；每次调用 `play()` 和 `getScore()`；比较并记录最大值；返回最大值。

- 关键易错点：关卡的分数是"层层解锁"关系，必须用嵌套 if，不能简单把三个 goalReached() 并列；isBonus() 无参，不要写 isBonus（不加括号在 Java 中不合法，但在手写时常见漏写）。

FRQ-5【2021 年国际考真题 · Question 1】WordMatch

考点：E4 String 方法（length、substring、indexOf、compareTo）、第二章循环

完整题面：

本题涉及 WordMatch 类，它存储一个 secret 字符串，并提供方法比较其他字符串与 secret 字符串的匹配程度。

```
public class WordMatch
{
    /** The secret string. */
    private String secret;

    /** Constructs a WordMatch object with the given secret
    string of lowercase letters. */
    public WordMatch(String word)
    { /* implementation not shown */ }

    /**
     * Returns a score for guess, as described in part (a).
     * Precondition: 0 < guess.length() <= secret.length()
     */
    public int scoreGuess(String guess)
    { /* to be implemented in part (a) */ }

    /**
     * Returns the better of two guesses, as determined by
    scoreGuess and the rules for a
     * tie-breaker that are described in part (b).
     * Precondition: guess1 and guess2 contain all lowercase
    letters.
     *
     * guess1 is not the same as guess2.
     */
    public String findBetterGuess(String guess1, String guess2)
    { /* to be implemented in part (b) */ }
}
```

(a)

编写 `scoreGuess` 方法：返回 `guess` 的得分。先统计 `guess` 作为子串在 `secret` 中出现的次数（允许重叠），再将出现次数乘以 `guess.length()` 的平方。

示例 1: `secret = "mississippi"`

guess	出现次数	得分计算	返回值
"i"	4	$4 \times 1^2 = 4$	4
"iss"	2	$2 \times 3^2 = 18$	18
"issipp"	1	$1 \times 6^2 = 36$	36
"mississippi"	1	$1 \times 11^2 = 121$	121

示例 2: `secret = "aaaabb"`

guess	出现次数	得分计算	返回值
"a"	4	$4 \times 1^2 = 4$	4
"aa"	3	$3 \times 2^2 = 12$	12
"aaa"	2	$2 \times 3^2 = 18$	18
"aabb"	1	$1 \times 4^2 = 16$	16
"c"	0	$0 \times 1^2 = 0$	0

(b)

编写 `findBetterGuess` 方法：比较两个猜测字符串，返回"更好"的那个。若两者得分不同，返回得分高者；若得分相同，返回字典序更大者。

示例: `secret = "concatenation"`

- `scoreGuess("ten") = 9`, `scoreGuess("nation") = 36`, 返回 "nation"。
- `scoreGuess("con") = 9`, `scoreGuess("cat") = 9`, 返回 "con"（字典序更大）。

官方参考解答：

```
// (a)
public int scoreGuess(String guess)
```

```

{
    int count = 0;
    for (int i = 0; i <= secret.length() - guess.length(); i++)
    {
        if (secret.substring(i, i + guess.length()).equals(guess))
        {
            count++;
        }
    }
    return count * guess.length() * guess.length();
}

// (b)
public String findBetterGuess(String guess1, String guess2)
{
    int score1 = scoreGuess(guess1);
    int score2 = scoreGuess(guess2);
    if (score1 > score2)
    {
        return guess1;
    }
    if (score2 > score1)
    {
        return guess2;
    }
    if (guess1.compareTo(guess2) > 0)
    {
        return guess1;
    }
    return guess2;
}

```

解析与评分要点：

- **Part (a)** 共 5 分：将 `guess` 与 `secret` 的等长子串比较；使用正确的子串长度；遍历所有必要子串（不越界、不遗漏重叠 `occurrence`）；统计出现次数；返回次数 $\times$ 长度²。
- **Part (b)** 共 4 分：调用 `scoreGuess` 获取两个得分；比较得分；在得分相同时用 `compareTo` 判断字典序；返回对应的 `guess`。

- 关键易错点：比较 `String` 内容必须用 `.equals()`，不能用 `==`；`compareTo` 返回正数表示调用者字典序更大；循环上界是 `secret.length() - guess.length()`（含），以保证 `substring(i, i + guess.length())` 不越界。

五、备考策略与复习清单

本章虽是 Unit 1 基础章节，但分值权重 15 - 25%，且为 FRQ1 的直接考查范围。以下按"投入产出比"给出复习优先级、考前自查清单与教师使用建议，建议结合 AP Classroom Unit 1 Progress Check 进行章节终结性测评。

5.1 复习优先级（按投入产出比排序）

本节给出按投入产出比排序的五个复习优先级。优先攻克前两项即可覆盖考试中 60% 以上的本章相关题目。

1. **String 六方法**（FRQ1 Part B 必考 + MCQ 每年 3 - 5 题）——达到能默写签名、能手工追踪的程度。
2. **整数除法 / cast / 优先级组合题**（MCQ 每年 3 - 4 题）——专项刷"求输出值"类题目。
3. **Math.random 区间公式**（MCQ + FRQ 双线出现，2024 年 FRQ1 直接考）。
4. **构造器匹配与实例方法调用**（为第三章 Class Creation 打底）。
5. **概念题**（错误类型、API、前置/后置条件、对象 vs 类）——考前突击即可。

5.2 考前自查清单

以下 9 条是本章节最常被考生忽略、但却最容易快速通过反复练习拿分的知识点。每条都对应本手册的具体内容，建议考前逐条对照检查。

- ☐ 我能不看资料写出六个 `String` 方法的完整签名与返回值。
- ☐ 我能说明 `substring(2, 5)` 取哪些下标（答案：2、3、4）。
- ☐ 我知道 `(int)(Math.random() * n) + m` 中 `n`、`m` 各代表什么。
- ☐ 我能手工追踪含 `cast`、`%`、复合赋值的五行走代码段。
- ☐ 我知道 `int` 除 `int` 得 `int`，并知道三种修法（`cast` 一侧 / 提前声明 `double` / 乘 1.0）。
- ☐ 我知道 `NullPointerException` 和 `IndexOutOfBoundsException` 分别在什么情况下发生。
- ☐ 我知道 `static` 方法与实例方法分别用什么调用。

- □ 我知道 String 不可变，方法返回新串要接住。
- □ 我读过 Java Quick Reference（考试提供，平时就要熟悉排版）。

5.3 教师使用建议

本节给教师三条使用建议：随堂学案、课前小测与 AP Classroom 联动的终结性测评。所有建议均与本手册的具体章节相对应，可直接落地实施。

- 本章手册可作为学案随堂使用：第二章知识点梳理配合教材对应章节；第三章易错点适合做成课前小测（每条易错点就是一个现成的辨析题）。
- 第四节 MCQ 建议限时训练（15 题控制在 20 分钟内），FRQ 建议手写后对照评分要点互评，模拟真实阅卷视角。
- 与 AP Classroom 联动：Unit 1 Progress Check（Part A 约 12 题、Part B 约 15 题、Part C 约 18 题 + 1 道 FRQ）可作为章节终结性测评，错题对应本手册知识点编号回炉。

参考文献

1. College Board. *AP Computer Science A Course and Exam Description*[R]. Effective Fall 2025. New York: The College Board, 2025.
2. College Board. *AP Computer Science A 2021 International Exam Free-Response Questions*[R]. 2021.
3. College Board. *AP Computer Science A 2022 International Exam Free-Response Questions*[R]. 2022.
4. College Board. *AP Computer Science A 2023 International Exam Free-Response Questions*[R]. 2023.
5. College Board. *AP Computer Science A 2024 International Exam Free-Response Questions*[R]. 2024.
6. College Board. *AP Computer Science A 2025 International Exam Free-Response Questions*[R]. 2025.
7. College Board. *AP Computer Science A — Java Quick Reference*[R]. 考试现场提供的速查表，2025.

版权说明

本手册为教学自编资料，知识点框架依据 College Board《AP Computer Science A Course and Exam Description》（2025 版）整理；所引真题版权归 College Board 所有，仅限校内备考教学使用。

第二章 Selection and Iteration

项	内容
适用考试	AP Computer Science A 国际考 (2025 秋新纲 • 2026 及以后)
课程章节	第二章 Selection and Iteration (对应官方大纲 Unit 2)
考纲来源	College Board 《AP Computer Science A Course and Exam Description, Effective Fall 2025》
真题来源	2021 - 2025 年近五年 APCSA 国际考 FRQ 真题、2025 新版 CED 官方样题、AP Classroom 官方 Progress Check 试题 (含 Unit 2 Selection and Iteration 专题)

摘 要

本手册为 AP Computer Science A (APCSA) 国际考备考系列的第二章，主题为 Selection and Iteration，对应 College Board 2025 秋新版考纲 Unit 2 (适用于 2026 年及以后考试)。本章系统梳理 12 个官方 Topic (2.1 Algorithms with Selection and Repetition / 2.2 Boolean Expressions / 2.3 if Statements / 2.4 Nested if Statements / 2.5 Compound Boolean Expressions / 2.6 Comparing Boolean Expressions / 2.7 while Loops / 2.8 for Loops / 2.9 Implementing Selection and Iteration Algorithms / 2.10 Implementing String Algorithms / 2.11 Nested Iteration / 2.12 Informal Run-Time Analysis)，覆盖布尔与逻辑、if 选择结构、while / for 循环、嵌套循环、字符串算法与运行时间分析；按失分频率排序 Top 10 高频易错点并给出正误对照；精选 8 道 MCQ (来自 AP Classroom 官方 Unit 2 TopicQuestion MCQ 题库) 与 5 道 2025 新版 CED 配套 Progress Check FRQ 完整真题 (FRQ-1 Part A BeadSequence、FRQ-2~FRQ-5 Part B 字符串处理、控制流、模运算、indexOf/lastIndexOf) —— 覆盖考试结构 42 题 MCQ / 55% 与 4 题 FRQ / 45% 的最新分布。本章是一切写代码题的内功，掌握好坏直接决定 Q1 Methods & Control Structures 的方法编写能力。Q3 ArrayList 与 Q4 2D Array 属于后续 Unit 4 范围，本章不考。

关键词：AP Computer Science A；2025 新版考纲；选择结构；循环；嵌套循环；字符串算法；运行时间分析；FRQ 真题解析；国际考备考

目 录

- [一、本章定位与考情速览](#)
- [二、知识点与考点梳理（Topic 2.1 - 2.12）](#)
 - [S：算法与控制结构总览（Topic 2.1 / 2.9）](#)
 - [A：布尔与复合逻辑（Topic 2.2 / 2.5 / 2.6）](#)
 - [B：if / Nested if（Topic 2.3 / 2.4）](#)
 - [C：while 循环（Topic 2.7）](#)
 - [D：for 循环 / 字符串 / 嵌套（Topic 2.8 / 2.10 / 2.11）](#)
 - [R：运行时间分析（Topic 2.12）](#)
 - [F：易错点专题（综合）](#)
- [三、本章重难点与易错点清单](#)
- [四、MCQ 真题精选与详解（8 题）](#)
- [五、FRQ 真题精选与解析（5 题完整题面）](#)
- [六、应试技巧清单](#)
- [七、章节总结](#)

一、本章定位与考情速览

1.1 新大纲四大单元与考试权重

2025 年秋起，APCSA 课程框架由旧的 10 个单元重组为 4 个单元。Unit 2 Selection and Iteration 是 MCQ 权重最高的单元之一（25 - 35%），所有 FRQ 都会涉及 if / while / for 与循环计数，因此本章是整个课程的"分水岭"——掌握得好，后续 Unit 3、4（Array、ArrayList、2D Array）的方法编写将水到渠成；掌握不好，则全卷所有写代码题都会卡壳。

单元（本课程章节）	MCQ 权重	建议课时
Unit 1：Using Objects and Methods（第一章）	15 - 25%	约 32 - 34 课时
Unit 2: Selection	25 - 35%	约 29 - 31 课时

and Iteration（第二章）

Unit 3 : Class
Creation（第三章） 10 - 18% 约 20 - 22 课时

Unit 4 : Data
Collections（第四章） 30 - 40% 约 50 - 52 课时

1.2 考试结构

部分	题量与时长	占比
Section I : 选择题 (MCQ, 含独立题与 成组题)	42 题 / 90 分钟	55%
Section II : 自由作 答题 (FRQ)	4 题 / 90 分钟	45%

格式说明

2021 - 2025 年国际考采用旧版分制 FRQ（共 4 题，分值不固定）；2026 年起 FRQ 第 1 题固定为新结构（Part A 4 分 + Part B 3 分，明确要求调用 String 方法）。本章 FRQ 精选以 2025 新版 CED 配套 AP Classroom Unit 2 Progress Check 完整题面为主，每题含官方英文原文与中文参考解。

1.3 本章与 FRQ 的关联（备考重点提示）

本章覆盖的"判别-分支-循环-边界"四件套，是 Q1 与 Q2 的方法编写"地基"：

- **Q1 Methods and Control Structures:** Part A 必含 if / while 嵌套，Part B 必调用 String 方法（与第一章强联动）。
- **Q2 Class Design:** 构造方法 + 实例变量初始化中大量使用 if-else 校验合法性。
- **Q3 ArrayList:** 仅考察 ArrayList 遍历与算法（属于第三章 Unit 4 范围），不属本章。
- **Q4 2D Array:** 仅考察 2D Array 遍历（属于第四章 Unit 4 范围），不属本章。
- 本章明确不考 break / continue、StringBuilder、Array 等语法扩展——MCQ 与 FRQ 均严格落在 Topic 2.1 - 2.12 范围。

换句话说：本章是一切写代码题的内功。MCQ 也同样——Unit 2 的 Top 10 高频考点几乎全部是布尔表达式与循环计数的代码追踪。

1.4 本章学习目标（对应官方 12 个 Topic）

完成本章后应能：

- 用比较运算符（==、!=、<、<=、>、>=）和逻辑运算符（&&、||、!）写出布尔表达式。
- 理解短路求值（short-circuit evaluation）并能预测含副作用的复合条件的结果。
- 运用 De Morgan 定律化简复杂布尔表达式。
- 编写单分支、双分支、多分支（if/if-else/if-else if-else）以及嵌套 if。
- 用 while 写计数器循环与哨兵循环，识别和避免无限循环。
- 用 for 写定长迭代，掌握初始化、终止条件、步长三要素。
- 用 String.length() + substring(i, i+1) 或 charAt(i) 遍历字符串每个字符。
- 编写嵌套循环（外层 + 内层），明确总迭代次数 = 外层 × 内层。
- 区分 =（赋值）与 ==（相等比较），理解 String 比较必须用 .equals() 的原因。
- 在循环中正确使用复合赋值运算符 +=、-=、*=、/=、%= 以及自增/自减 ++/--。
- 对 int 与 double 的混合运算、% 模运算、/ 整数除法保持警觉。
- 能在 MCQ 中追踪 5 - 20 行代码段在控制流分支后的最终变量值；能识别 $O(1)$ / $O(n)$ / $O(n^2)$ 运行时间级别。

二、知识点与考点梳理（Topic 2.1 - 2.12）

本章共 12 个 Topic（依据 College Board《AP Computer Science A CED, Effective Fall 2025》），按知识结构分为六大板块。每个知识点均标注考纲 Learning Objective（LO）编号，便于对照官方大纲复习。

板块 S：算法与控制结构总览（Topic 2.1 / 2.9）

SI. Topic 2.1 — Algorithms with Selection and Repetition (LO 2.1.A - 2.1.B)

核心概念：

- 算法（Algorithm）：解决问题的有限步骤序列，APCSA 仅涉及选择（Selection）与重复（Repetition）两类控制结构。

- **选择结构 (Selection)**：通过 `if/if-else` 让程序在不同输入下走不同分支。
- **重复结构 (Repetition)**：通过 `while/for` 让同一段代码反复执行。
- 本章所有算法本质上都是这两类结构的组合，先识别"分支点"再识别"循环点"。

```
// 例子：求 1+2+...+100 中所有算法骨架

int sum = 0;
for (int i = 1; i <= 100; i++) {    // Repetition
    sum += i;
}
if (sum > 5000) {                  // Selection
    System.out.println("big");
}
```

S2. Topic 2.9 — Implementing Selection and Iteration Algorithms (LO 2.9.A)

核心概念：

- 把"伪代码/Pseudocode"翻译成可执行 Java 代码时要保留三件事：循环起止条件、每次迭代对变量的更新、终止边界。
- 选择结构在 Java 中必须用 `boolean` 表达式作为条件，没有 0/1 自动转 `boolean`。
- 实现策略："先写骨架 (`init` → `loop guard` → `body` → `update` → `return`)"再补语法细节。

A1. Topic 2.2 — Boolean Expressions: 基本比较运算符 (LO 2.2.A - 2.2.B)

核心概念：

- **基本类型 `boolean`**：只能取 `true` 或 `false` 两个值 (LO 1.2.B.1)。在 Java 中 1/0 不能直接当 `boolean` 用，这是与 C/Python 的关键差异。
- **比较运算符 (LO 2.1.A)**——共六种，结果一律为 `boolean`：

运算符	含义	适用类型
<code>==</code>	等于	基本类型、 <code>boolean</code> 、引用类型 (语义见下)

!=	不等于	同上
<	小于	int 、 double
<=	小于等于	int 、 double
>	大于	int 、 double
>=	大于等于	int 、 double

- **== 的引用陷阱（LO 2.1.B 高频考点！）：**
- 对**基本类型**，`a == b`比较的是"值是否相等"；
- 对**引用类型**（如 `String`、`ArrayList`），`a == b`比较的是"是否同一个对象"，**不是字符串内容相等！**
- 比较字符串内容必须用 `a.equals(b)`；比较时忽略大小写用 `a.equalsIgnoreCase(b)`。
- 比较 `ArrayList<String>` 全部元素相等，需结合循环逐个调用 `.equals()`（详见第四章）。

```
String s1 = "hi";
String s2 = new String("hi");
System.out.println(s1 == s2);           // false（不同对象）
System.out.println(s1.equals(s2));      // true（内容相等）

int x = 5, y = 5;
System.out.println(x == y);             // true（值相等）
```

选择题考法：给一段 `String` 比较代码，问输出什么；或问"以下哪种比较字符串内容的方式正确"。

A2. Topic 2.5 — Compound Boolean Expressions: &&/||/! (LO 2.5.A)

核心概念：

- **&&（与）：**左右都为 `true` 时整体为 `true`，否则为 `false`。
- **||（或）：**左右任一为 `true` 时整体为 `true`，都为 `false` 时才为 `false`。
- **！（非）：**取反，单目运算符。

A	B	A && B	A B	!A
false	false	false	false	true

false	true	false	true	true
true	false	false	true	false
true	true	true	true	false

运算符优先级（必须背下来）：

1. `!`（一元）— 最高
2. 算术（`*`、`/`、`%`、`+`、`-`）
3. 比较（`<`、`<=`、`>`、`>=`、`==`、`!=`）
4. `&&`
5. `||` — 最低

记忆口诀：非 → 算 → 比 → 与 → 或。即 `!` 最先算，`||` 最后算。

A3. Topic 2.6 — Comparing Boolean Expressions: De Morgan 与等价变换 (LO 2.6.A)

De Morgan 定律（双等价式，常考题目“以下哪一项与之等价”）：

- `!(A && B)` 等价于 `!A || !B`
- `!(A || B)` 等价于 `!A && !B`

直观理解：“整体否定 = 拆开否定后交换与或”。

```
// 原始：不是 (a > 0 且 b > 0)
boolean ok = !(a > 0 && b > 0);

// 等价：(用 De Morgan)
boolean ok2 = a <= 0 || b <= 0;
```

选择题考法：给一个布尔表达式，问“与之等价的是哪一项”。例如 `!(x > 5 || y <= 0)` 等价于 `x <= 5 && y > 0`。

A4. 短路求值 (Short-circuit evaluation, Topic 2.5 拓展, LO 2.5.A)

核心规则：

- `A && B`：若 `A` 为 `false`，则 `B` 不执行（结果必然 `false`）。
- `A || B`：若 `A` 为 `true`，则 `B` 不执行（结果必然 `true`）。

```
int x = 0;
if (x != 0 && 10 / x > 1) // x==0 时 && 后半段被跳过，不抛 ArithmeticException
{
```

```
System.out.println("safe");  
}
```

陷阱：考题经常设置"看起来会执行实际不会"的副作用语句，例如：

```
int a = 5, b = 10;  
boolean result = (a > 10) && (b++ > 5);  
System.out.println(b);    // 输出 10, b++ 没执行
```

选择题考法：追踪含 `&&` / `||` 与副作用的代码段最终变量的值。

板块 B: `if` / `Nested if` 选择结构 (Topic 2.3 / 2.4)

B1. Topic 2.3 — `if` Statements: `if` / `if-else` / `if-else if-else` (LO 2.3.A)

三种基本形态：

```
// 1. 单分支  
if (条件) {  
    // 只有条件为 true 才执行  
}  
  
// 2. 双分支  
if (条件) {  
    // 分支 A  
} else {  
    // 分支 B (条件为 false 时执行)  
}  
  
// 3. 多分支  
if (条件 1) {  
    // 1  
} else if (条件 2) {  
    // 2  
} else if (条件 3) {  
    // 3  
}
```

```

} else {
    // 兜底
}

```

执行顺序：自顶向下判断，第一个满足的分支执行后立即跳出整条链，剩余分支不再检查。

B2. Topic 2.4 —Nested if Statements (LO 2.4.A)

在 if 块里再写 if。写法可读性较差，但题目经常出现，更推荐重构为"组合条件"或 else if 链。

```

    if (age >= 13) {
        if (hasTicket) {
            System.out.println("可以进入");
        } else {
            System.out.println("需要票");
        }
    }
} else {
    System.out.println("年龄不够");
}

```

选择题考法：给一段嵌套 if，问条件为何时执行某分支；或问输出结果。

板块 C：while 循环 (Topic 2.7)

C1. Topic 2.7 —while Loops 语法与执行流程 (LO 2.7.A)

```

int i = 0;           // 1. 初始化
while (i < 5) {       // 2. 条件判断
    System.out.println(i);
    i++;             // 3. 更新（必须有！否则死循环）
}

```

执行流程：先判条件 → 真则进入循环体 → 执行体一次 → 再回到条件判断 → 直到条件为假跳出。

典型陷阱：

- 忘记更新 → 死循环（infinite loop），程序卡死。
- 更新方向错误（如希望递增却写成 `i--`）→ 循环不执行或立即终止。
- 条件永远为真（如 `while (true)` 且循环内不更新循环变量）→ 死循环。

C2. 哨兵循环 (Sentinel loop, Topic 2.7 拓展, LO 2.7.B)

核心思想：用一个特殊值（如 `-1`、`""`、`null`）作为结束标志。

```
Scanner sc = new Scanner(System.in);

int sum = 0;

System.out.print("输入数字 (0 结束) : ");

int n = sc.nextInt();

while (n != 0) {           // 0 是哨兵

    sum += n;

    System.out.print("输入数字 (0 结束) : ");

    n = sc.nextInt();

}

System.out.println("sum = " + sum);
```

选择题考法：给一段 `while` 循环 + 输入序列，问最终 `sum` 多少 / 输出多少；或问“哨兵是哪个值”。

板块 D：for 循环与字符串算法 (Topic 2.8 / 2.10 / 2.11)

D1. Topic 2.8 —for Loops 语法与三要素 (LO 2.8.A)

```
for (初始化; 条件; 更新) {

    // 循环体

}
```

例：从 1 加到 100

```
int sum = 0;

for (int i = 1; i <= 100; i++) {

    sum += i;

}
```

三要素：

6. 初始化：循环变量赋初值，仅执行 1 次。
7. 条件：每次循环体执行前判断；为 false 则终止。
8. 更新：每次循环体执行完后执行，再回到条件判断。

与 while 的关系：for 循环等价于 while，仅书写形式不同。for 适合"明确循环次数"的场景，while 适合"循环次数未知、依赖输入"的场景。

D2. Topic 2.10 — Implementing String Algorithms: 字符串迭代 (LO 2.10.A)

两种方式：

```
String s = "Hello";  
// 方式一：length + charAt (最常用)  
for (int i = 0; i < s.length(); i++) {  
    char c = s.charAt(i);  
    System.out.println(c);  
}  
// 方式二：substring(i, i+1) (效率低但合法)  
for (int i = 0; i < s.length(); i++) {  
    String sub = s.substring(i, i + 1);  
    System.out.println(sub);  
}
```

考纲提示：for-each (增强 for) 在 Unit 6 才正式登场，本章只用普通 for (int i = 0; ...)。

典型陷阱：s.length() 是方法调用，每次循环都重新计算。理论上可优化为 int n = s.length();，但选择题不考效率，所以直接 s.length() 即可。

D3. Topic 2.8 拓展 一步长、反向与多变量 (LO 2.8.B)

- 步长可任意：例如 for (int i = 0; i < 10; i += 2) 输出 0,2,4,6,8。
- 反向迭代：for (int i = s.length() - 1; i >= 0; i--)
- 多变量：for (int i = 0, j = 10; i < j; i++, j--) (少见但合法)。

D4. Topic 2.11 — Nested Iteration 嵌套循环 (LO 2.11.A)

结构：

```

    for (int i = 0; i < 3; i++) {           // 外层
    for (int j = 0; j < 4; j++) {           // 内层
        System.out.print("(" + i + ", " + j + ") ");
    }
    System.out.println();
}
// 输出:
// (0, 0) (0, 1) (0, 2) (0, 3)
// (1, 0) (1, 1) (1, 2) (1, 3)
// (2, 0) (2, 1) (2, 2) (2, 3)

```

总迭代次数 = 外层 × 内层。例如 $3 \times 4 = 12$ 次。MCQ 常考这种"求循环总次数"题。

板块 R: Informal Run-Time Analysis (Topic 2.12)

R1. Topic 2.12 — Informal Run-Time Analysis: 循环次数的 $O()$ 估算 (LO 2.12.A)

核心概念:

- 常数级 $O(1)$: 执行次数与输入规模无关, 例如 `if (x > 0)`。
- 线性 $O(n)$: 执行次数与 n 成正比, 例如 `for (int i = 0; i < n; i++)`。
- 平方级 $O(n^2)$: 嵌套两层循环, 例如 9×9 乘法表。
- 指数 $O(2^n)$: 递归求斐波那契 (本章不深考)。

```

// O(1)
if (num > 0) { ... }

// O(n)
for (int i = 0; i < n; i++) {
    System.out.println(i);
}

// O(n^2)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.print("*");
    }
}

```

```

    }
    System.out.println();
}

```

选择题考法：给定一段嵌套循环，问"输入 n 时总执行次数是多少"（答案通常是 n^2 级别，常数 n ， $n(n+1)/2$ ， $n^2 - n$ 等）。MCQ 不考 $O()$ 严格定义，只识别"线性"与"平方级"差异即可。

板块 F：易错点专题（综合）

F1. = vs == (LO 2.1.A 巩固)

```

int a = 5;

if (a = 10) { /* 编译错误 */ } // 编译期报错：int 不能转 boolean

boolean b = false;

if (b = true) { /* 合法！b 被赋值为 true，进入分支 */ }

```

在 `if (...)` 中写成 `b = true` 是合法代码，但语义是把 `true` 赋给 `b`，永远进入 `if` 体——是高频 bug 源。

F2. 自增自减与复合赋值 (LO 2.11.A)

- `i++ / i--`：先用再加 / 减。
- `++i / --i`：先加 / 减再用。
- `i += 2` 等价于 `i = i + 2`；`*=`、`/=`、`%=` 类似。

选择题考法：追踪 `int x = 5; int y = x++;` 与 `int y = ++x;` 后 `x`、`y` 的值。

F3. 整数除法与模运算 (LO 2.12.A)

- **整数除法 `/`：**`7 / 2 == 3`（截断小数部分，不四舍五入）。
- **模 `%`：**`7 % 2 == 1`（余数，符号与被除数一致）。
- **混合运算优先级：**`int / int → int`，**运算前**就会截断；想保留小数需先转 `double`：`7.0 / 2 == 3.5`。

典型陷阱：

```

int a = 7, b = 2;

System.out.println(a / b); // 3（整数除）

```

```
System.out.println((double) a / b); // 3.5
System.out.println(a % b);          // 1
```

F4. 隐式类型提升 (LO 2.13.A)

- `int` 与 `double` 运算时, `int` 自动提升为 `double`。
- 但 `int / int` 仍是 `int` (提升发生在赋值前)。

```
int x = 5;
double y = x / 2;    // y = 2.0 (先 int 除得 2, 再转 double)
double z = x / 2.0;  // z = 2.5 (int 转 double 后再除)
```

三、本章重难点与易错点清单 (按出错频率排序)

这份清单基于近五年 (2021 - 2025) AP Classroom Unit 2 选择题错题分布。

高频易错点 TOP 10

1. **String 用 == 比较内容** — 编译通过、运行"看起来对", 但内容不同时仍返回 `false`。必记口诀: 内容比较必 `.equals()`。
2. **= 误写成 ==** — 在 `if (x == 5)` 中漏写一个等号变成 `if (x = 5)`, `int` 变量会编译错误; `boolean` 变量则语义改变。
3. **while 死循环** — 忘记在循环体中更新循环变量。
4. **for 终止条件差一** — `i <= n` vs `i < n`, 多一个少一个就是 bug。
5. **嵌套循环总次数混淆** — 把内层循环次数当作总次数, 忘记乘以外层。
6. **模运算理解成“找倍数”** — `7 % 2 == 1` 经常被误答成 5 或 3。
7. **整数除截断** — `7 / 2` 期望得 3.5, 实际是 3。
8. **De Morgan 定律化简错** — `!(A || B)` 错写成 `!A || !B`, 正确应为 `!A && !B`。
9. **独立 if 链 vs else if 链混淆** — 误以为多个独立 `if` 等价于 `else if` 链, 实际结果会被后面的 `if` 覆盖。
10. **短路求值的副作用忽略** — `if (false && (i++ > 0))` 中 `i++` 不执行, 输出仍是原值。

学习建议

- 先做 5 道 `if-else` MCQ, 再学 `while`。选择结构是循环结构的基础。

- **手写 10 遍 for 循环三要素：**分别写"遍历字符串"、"求 1..n 和"、"嵌套循环输出 9×9 乘法表"。
- **把每一道错题“翻译”成一句话：**例如 "我把 `s == t` 当成内容比较了"→ 加深 `==` 引用陷阱印象。
- **写题先模拟：**拿到一段代码段，先用人手走一遍每行执行，再与答案对比。

四、MCQ 真题精选与详解

本章共精选 10 道 Unit 2 选择题，全部来自 AP Classroom 官方 Unit 2 TopicQuestion MCQ（College Board 2025 新版 CED 配套题库，2026 国际考适用）。每道题均包含：

- 英文原题（与考试完全一致）
- 选项 A - D
- 答案与详细解析
- 涉及的考点与易错提醒

体例约定：题号沿用 AP Classroom 原始顺序；题源为 AP Classroom 官方题库与近五年国际考 MCQ；所有题目经由王芳老师审核并加入中文解析。

MCQ-1（&&/|| 短路、De Morgan）

英文原题：

Consider the following code segment.

```
boolean b1 = true && ((5 % 2) == 0);
boolean b2 = /* missing expression */;
```

1. Which of the following can replace `/* missing expression */` so that the value stored in `b2`

is the same as the value stored in `b1` after the code segment has executed?

- (A) `false || ((5 % 2) == 1)`
- (B) `false && ((5 % 2) == 1)`
- (C) `true || ((5 % 2) == 1)`
- (D) `true && ((5 % 2) == 1)`

中文解读： $5 \% 2 == 0$ 是 `false`，所以 `b1 = true && false = false`。要找 `b2 = false` 的表达式。利用短路求值，`false || ...` 是 `false`（不论右边）；`false && ...` 是 `false`；`true || ...` 是 `true`；`true && ...` 是 $(5 \% 2 == 1)$ `= true`。只有 (A) 必为 `false`。

答案：A

解析： 考察 `&&/||` 短路求值。`false || X` 永远是 `false`，无需关心 `X` 的值；`true && X` 的值取决于 `X`。(B) 中 `false && true == false` 也是 `false`，但是若 `X` 不是 $(5 \% 2) == 1$ 时结果仍为 `false`，不唯一确定。题目要求“`b2` 与 `b1` 始终相同”，必须保证无论 `X` 怎么变，`b2` 都是 `false`——只有 (A) `false || X` 满足。

MCQ-2（混合条件运算符优先级）

英文原题：

Consider the following code segment.

```
if (false && true || false)
{
    if (false || true && false)
    {
        System.out.print("First");
    }
    else
    {
        System.out.print("Second");
    }
}
if (true || true && false)
{
    System.out.print("Third");
}
```

2. What is printed as a result of executing the code segment?

- (A) Second
- (B) Third
- (C) FirstThird

(D) SecondThird

中文解读：先算 `&&`，后算 `||`：

- 外层 `if (false && true || false)`：先算 `false && true == false`，再 `false || false == false`。整个外层 `if` 跳过，内部两个 `print` 都不执行。
- 第二个 `if (true || true && false)`：先 `true && false == false`，再 `true || false == true`。进入 `if`，打印 `Third`。

答案：B

解析：优先级 `! > 算 > 比 > && > ||`。题目的迷惑点在于把 `||` 写在 `&&` 前，但运算仍按优先级进行。

MCQ-3（for 循环 + 字符串反向遍历）

英文原题：

Consider the following method.

```
public static String changeStr(String str)
{
    String result = "";
    for (int i = str.length() - 1; i >= str.length() / 2; i -= 2)
    {
        result += str.substring(i, i + 1);
    }
    return result;
}
```

3. What value is returned as a result of the method call `changeStr("12345")`?

- (A) "4"
- (B) "53"
- (C) "531"
- (D) "543"

中文解读：`str.length() = 5`。

- 初始化 $i = 4$: 取 `str.substring(4, 5) = "5"`, `result = "5"`, $i = 4 - 2 = 2$ 。
 - $i = 2 \geq 5/2 = 2$: 取 `str.substring(2, 3) = "3"`, `result = "53"`, $i = 2 - 2 = 0$ 。
 - $i = 0 \geq 2$: **false**, 退出循环。
- 最终返回 "53"。

答案: B

解析: 关键在于 $\geq$ 边界与 $i -= 2$ 的步长。手算三步循环即可: i 依次为 4、2、0; 只有 4 和 2 满足 $i \geq \text{str.length()}/2$, 所以 `result` 拼接两个字符。

MCQ-4 (独立 if 链 vs if-else if 链行为差异)

英文原题:

Consider the following code segments, which are each intended to convert grades from a 100-point scale to a 4.0-point scale and print the result. A grade of 90 or above should yield a 4.0, a grade of 80 to 89 should yield a 3.0, a grade of 70 to 79 should yield a 2.0, and any grade lower than 70 should yield a 0.0.

Assume that `grade` is an `int` variable that has been properly declared and initialized.

Code Segment 1

```
double points = 0.0;
if (grade > 89)
{
    points = 4.0;
}
else if (grade > 79)
{
    points = 3.0;
}
else if (grade > 69)
{
```

```
        points = 2.0;
    }
    else
    {
        points = 0.0;
    }
    System.out.println(points);
```

Code Segment 2

```
double points = 0.0;
if (grade > 89)
{
    points = 4.0;
}
if (grade > 79)
{
    points = 3.0;
}
if (grade > 69)
{
    points = 2.0;
}
if (grade < 70)
{
    points = 0.0;
}
System.out.println(points);
```

7. Which of the following statements correctly compares the values printed by the two code segments?

- (A) The two code segments print the same value only when grade is below 80.
- (B) The two code segments print the same value only when grade is 90 or above.
- (C) Both code segments print the same value for all possible values of grade.
- (D) The two code segments print different values for all possible

values of grade.

中文解读：分段 1 是标准 else if 链；分段 2 是 4 个独立 if，全部判断、全部执行。

- grade = 95:
- 段 1: points = 4.0, 然后不再进入 else if。打印 4.0。
- 段 2: 第一个 if (95>89) 设 points = 4.0; 第二个 if (95>79) 设 points = 3.0; 第三个设 points = 2.0; 第四个 95<70 假。打印 **2.0!**
 - 不一致。
- grade = 50:
- 段 1: else 分支设 points = 0.0。打印 0.0。
- 段 2: 前 3 个 if 都不满足; 第四个 50<70 真, 设 points = 0.0。打印 0.0。
- 一致。

所以两种结构在 grade<70 时结果一致, 其他情况下不一致。

答案: A

解析：考察独立 if 与 if-else 链的行为差异。最容易出错的是"独立 if 一定会输出最大的匹配条件"——错的, 因为最后会被后面的 if 覆盖。

MCQ-5 (De Morgan / 表达式真值表)

英文原题:

In the following Boolean expressions, x and y are properly declared and initialized boolean variables.

Expression A

x && y

Expression B

!x && !y

8. Which of the following best describes the relationship between values produced by

Expression A and Expression B?

- (A) Expression A and Expression B evaluate to different values for all values of x and y.
- (B) Expression A and Expression B evaluate to the same value for all values of x and y.
- (C) Expression A and Expression B evaluate to the same value only when x and y have the same value.
- (D) Expression A and Expression B evaluate to the same value only when x and y have different values.

中文解读：列出真值表：

x	y	A: x && y	B: !x && !y
false	false	false	true
false	true	false	false
true	false	false	false
true	true	true	false

- 当 x、y 相同时：(F, F) 时 A=F, B=T 不一致；(T, T) 时 A=T, B=F 不一致。所以"x、y 相同"时反而不同。
- 当 x、y 不同时：两种组合都是 A=F, B=F，相等。

答案：D

解析：考察真值表与 De Morgan 定律 ($!x \ \&\& \ !y \equiv !(x \ || \ y)$)。本表达式与 $x \ \&\& \ y$ 互为否定但不总是反义——只在 (x, y) 一致时一致。

MCQ-6 (while 循环输出追踪)

英文原题：

Consider the following code segment.

```
int num = 1;
int count = 0;
while (num <= 5)
{
    if (num % 2 == 0)
    {
        count++;
    }
}
```

```
    }  
    num++;  
}  
System.out.println(count);
```

10. What is printed as a result of executing the code segment?

- (A) 2
- (B) 3
- (C) 5
- (D) 7

中文解读：while 循环 num 从 1 到 5，统计偶数出现次数：

- num=1: 奇数, count 不变; num++ → 2
- num=2: 偶数, count=1; num++ → 3
- num=3: 奇数; num++ → 4
- num=4: 偶数, count=2; num++ → 5
- num=5: 奇数; num++ → 6
- num=6>5: 退出

最终 count = 2。

答案：A

解析：典型 while + 计数 MCQ。务必手算每一轮 num 与 count 的值，不要跳步。

MCQ-7（整数除法 + 双重嵌套循环计数）

英文原题：

Consider the following code segment.

```
int count = 0;  
for (int i = 0; i < 4; i++)  
{  
    for (int j = i; j < 4; j++)  
    {  
        count++;  
    }  
}
```

```
}  
System.out.println(count);
```

13. What is printed as a result of executing the code segment?

- (A) 4
- (B) 8
- (C) 10
- (D) 16

中文解读：外层 $i = 0, 1, 2, 3$ ；内层 j 从 i 到 3（不含 4）：

- $i=0$: $j = 0, 1, 2, 3$, 共 4 次
- $i=1$: $j = 1, 2, 3$, 共 3 次
- $i=2$: $j = 2, 3$, 共 2 次
- $i=3$: $j = 3$, 共 1 次

总和 $= 4 + 3 + 2 + 1 = 10$ 。

答案：C

解析：内层起始条件随外层变化时，总次数不再是简单 $m \times n$ 。这类题常考"递减次数"——列清单是最稳的方式。

MCQ-8 (String 比较 + 判别陷阱)

英文原题：

Consider the following code segment.

```
String a = new String("cat");  
String b = new String("cat");  
String c = a;  
boolean p = (a == b);  
boolean q = (a.equals(b));  
boolean r = (a == c);
```

14. What are the values of p , q , and r after the code segment has executed?

- (A) p is true, q is true, r is true

- (B) p is false, q is true, r is true
- (C) p is false, q is true, r is false
- (D) p is true, q is false, r is false

中文解读：

- a 与 b 是用 `new String(...)` 创建的两个不同对象，内容相同但不是同一对象。
- `c = a` 是引用赋值，c 与 a 指向同一个对象。
- `a == b`：比较引用，false。
- `a.equals(b)`：比较内容，true。
- `a == c`：同一对象，true。

答案：B

解析：考察 `==` 与 `.equals()` 在 `String` 上的差异，以及引用赋值的语义。这是 Unit 1 + Unit 2 联合考点，全文最高频出错题之一。

MCQ 限时训练建议：以上 10 题计时 25 分钟完成，平均 2.5 分钟/题。新版考纲的 MCQ 1 - 4 选项很少出现“明显送分”，必须手算才能拿稳分。

五、FRQ 真题精选与解析

本章精选 5 道完整真题，全部来自 AP Classroom 官方 Unit 2 Selection and Iteration Progress Check 题目（College Board 2025 新版 CED 配套题库，与 2026 国际考同源）。FRQ-1 来自 Part A（4 段合一），FRQ-2 至 FRQ-5 来自 Part B（4 题各取一段）。每题均给出官方英文原题与中文思路、参考解法、评分要点，方便对照练习。

章节引言：Unit 2 Selection and Iteration 题目特别考察对布尔表达式的精确控制（避免 off-by-one）、对 `==` / `.equals()` 的正确选择、以及对整数除法与模运算的熟练度。AP Classroom 的 Progress Check 是备考这一章节的最佳练习材料。

FRQ-1 (AP Classroom Unit 2 FRQ Part A • BeadSequence)

Part A • Official English Wording (from College Board Unit 2 Progress Check FRQ Part A)

This question involves processing a String that represents a sequence of beads.

The String variable `beadSequence` represents a bead sequence. It contains a sequence of letters in which each letter represents the color of a bead.

A bead sequence is considered acceptable if it adheres to the following rules.

Otherwise, it is not considered acceptable.

- Rule 1: The number of beads in the sequence is between 5 and 10, inclusive.
- Rule 2: The sequence contains at least one blue bead (represented by "B") and at least one white bead (represented by "W").
- Rule 3: The sequence starts and ends with the same color.

You will create three boolean variables to help determine whether the sequence is acceptable, and then print a message indicating the result. The following table illustrates the intended behavior for different values of `beadSequence`.

Value of <code>beadSequence</code>	Meets Rule 1	Meets Rule 2	Meets Rule 3
Printed Output			
"OBWO"	false	true	true
"Not acceptable"			
"BWBGOGWBB"	true	true	true
"Acceptable"			
"ROYGBIVW"	true	true	false
"Not acceptable"			

"BOBOBOBO"	true	false	false
"Not acceptable"			
"WBWRWBWRWBWR"	false	true	false
"Not acceptable"			
"BWWWBBWWWB"	true	true	true
"Acceptable"			

1. Write a code segment that creates a boolean variable named `lengthGood` and
assigns it true if `beadSequence` meets rule 1 and assigns it false otherwise.
2. Write a code segment that creates a boolean variable named `correctColors` and
assigns it true if `beadSequence` meets rule 2 and assigns it false otherwise.
3. Write a code segment that creates a boolean variable named `startEndGood` and
assigns it true if `beadSequence` meets rule 3 and assigns it false otherwise.
4. Write a code segment that prints "Acceptable" if `beadSequence` adheres to all
three rules and prints "Not acceptable" otherwise. You must use the variables
`lengthGood`, `correctColors`, and `startEndGood` to receive full credit, regardless
of what you wrote for parts A, B, and C.

中文思路

Part A 一道题含 4 个子问题，考点全部是 Unit 2 标准选择结构：

- (1) 长度条件： $5 \leq \text{length} \leq 10$ 。
- (2) 至少一个 'B' 和至少一个 'W'：必须分别检查两个字符，或用循环扫描。
- (3) 首尾字符相同：`.charAt(0)` 与 `.charAt(length - 1)` 比较，注意非空字符串的前提。
- (4) 三个 boolean 与运算后输出。

参考解法

```
// (1) 长 5..10
boolean lengthGood = beadSequence.length() >= 5 && beadSequence.length() <= 10;

// (2) 至少一个 B 和一个 W
boolean hasB = false, hasW = false;
for (int i = 0; i < beadSequence.length(); i++) {
    char c = beadSequence.charAt(i);
    if (c == 'B') hasB = true;
    if (c == 'W') hasW = true;
}
boolean correctColors = hasB && hasW;

// (3) 首尾同色 (要求 beadSequence 非空)
boolean startEndGood = beadSequence.length() > 0
    && beadSequence.charAt(0) == beadSequence.charAt(beadSequence.length()
- 1);

// (4) 合并
if (lengthGood && correctColors && startEndGood) {
    System.out.println("Acceptable");
} else {
    System.out.println("Not acceptable");
}
```

评分要点

- (1) 1 分：正确写出 5..10 闭区间表达式。
- (2) 1 分：用 `.indexOf("B") >= 0` 或循环都能得分。
- (3) 1 分：未处理空字符串可能要扣 0.5。
- (4) 1 分：必须使用前三个变量——若新写一遍条件直接判三规则，此项不得分。

FRQ-2 (AP Classroom Unit 2 FRQ Part B • colWidth 字符串填充)

Part A • Official English Wording (from College Board Unit 2 Progress Check FRQ Part B, Question 1)

This question involves processing strings of text.

Consider the following variables.

- colWidth is an int variable with a positive value.
- text is a String variable with a non-null value.
- otherText is a String variable with a non-null value.

1. A code segment, which you will write, should print output according to the following rules.

- If the length of text is greater than colWidth, then the value of text will be printed.
- Otherwise, the printed output will consist of colWidth characters contained between two "|" characters. The text between the "|" characters will consist of a sequence of periods ("."), followed by the value of text, followed by another sequence of periods.
- The printed output will have an equal number of periods on each side of text, if possible. If this is not possible, then there will be one fewer period on the left side of text than on the right side of text.

Value of colWidth	Value of text	Printed Output
5	"ab"	.ab..
5	"abc"	.abc.
10	"ab"	...ab....
10	"abc"	...abc....
10	"abcdefghij"	abcdefghij

```
10          "abcdefghijk..." abcdefghijk...    (no bars,
length > colWidth)
```

Write a code segment to print the intended output. Your implementation must meet all specifications and conform to the examples shown in the table.

中文思路

分两种情形：`length > colWidth` 直接打印；否则用点填充到 `colWidth` 字符。

点的总数量 = `colWidth - text.length()`，左右分布：左侧 `left = total/2`，右侧 `right = total - left`。

参考解法

```
if (text.length() > colWidth) {
    System.out.println(text);
} else {
    int totalDots = colWidth - text.length();
    int leftDots  = totalDots / 2;           // 整数除，向下截断
    int rightDots = totalDots - leftDots;    // 多出来的点放右侧
    // 打印
    System.out.print("|");
    for (int i = 0; i < leftDots; i++) System.out.print(".");
    System.out.print(text);
    for (int i = 0; i < rightDots; i++) System.out.print(".");
    System.out.println("|");
}
```

评分要点

- 正确判断 "`length > colWidth`" 走分支，1 分。
- 用整数除法分点：`totalDots/2` 给左、剩余给右，1 分。
- 用循环或 `String.repeat` 都能得分（考纲允许 `String.repeat`）。
- 使用 `|` 包裹内容 1 分。

FRQ-3 (AP Classroom Unit 2 FRQ Part B • 最长公共前缀)

Part A • Official English Wording (from College Board Unit 2 Progress Check FRQ Part B, Question 2)

2. A code segment, which you will write, should print the length of the longest prefix of the text that is a substring of otherText. A prefix is a substring that occurs at the beginning of another string.

Value of text	Value of otherText	Printed Output
Explanation		
"apple"	"pear"	1
The prefix "a" is a substring of "pear", but "ap" is not.		
"apple"	"pineapple"	5
The prefixes "a", "ap", "app", "appl", and "apple" are substrings of "pineapple". The length of the longest substring is 5.		
"apple"	"happy"	3
The prefixes "a", "ap", and "app" are substrings of "happy". The length of the longest substring is 3.		
"apple"	"leek"	0
No prefix of "apple" is a substring of "leek".		

Write a code segment to print the intended output. Your implementation must meet all specifications and conform to the examples shown in the table.

中文思路

从长到短依次检查 `text.substring(0, k)` 是否在 `otherText` 中，一旦命中立即输出 `k`。关键技巧：用 `otherText.indexOf(prefix) >= 0` 检查子串。

参考解法

```
int answer = 0;
boolean found = false;
for (int k = text.length(); k >= 1 && !found; k--) {
    String prefix = text.substring(0, k);
    if (otherText.indexOf(prefix) >= 0) {
        answer = k;
        found = true;                // 第一个命中即最长
    }
}
System.out.println(answer);
```

评分要点

- 用 `indexOf` 检查子串 1 分。
- 从长到短遍历前缀，找到即设置 `found`（或等价布尔变量）提前结束 1 分。
- 若用从短到长、需要每命中更新 `answer`，可以等价得分，但代码略长。

FRQ-4 (AP Classroom Unit 2 FRQ Part B • 个位数恰好出现 1 次)

Part A • Official English Wording (from College Board Unit 2 Progress Check FRQ Part B, Question 3)

This question involves writing two code segments that use information about downloaded songs.

A record company uses the following variables to maintain information about each song in the company's catalog.

- `downloads` is an `int` variable that represents the number of times a song was

downloaded. Assume the value of this variable is positive.

- `songInfo` is a `String` variable that contains information about the song. The

string consists of the artist name, followed by a hyphen ("-"),

followed by

the album title, followed by another hyphen, followed by the song title.

Assume that no other hyphens appear in the string.

3. A code segment, which you will write, should determine whether the ones

digit of the number stored in `downloads` appears exactly one other time in

the number. If so, the code segment should print "exactly once";

otherwise, it should print "not exactly once".

For example, if `downloads` has a value of 1734567, then the segment should

print "exactly once" because the ones digit, which is 7, appears one other time.

As another example, if `downloads` has a value of 325322, then the segment

should print "not exactly once" because the ones digit, which is 2, appears two other times.

Write a code segment to print the intended output. Your implementation must conform to the examples.

中文思路

先取个位数 `int ones = downloads % 10;`, 再统计去掉个位后其余位中等于 `ones` 的个数。

- 计数 = 1 → "exactly once"
- 否则 → "not exactly once"

参考解法

```
int ones = downloads % 10;
int otherCount = 0;
int n = downloads / 10;           // 去掉个位
while (n > 0) {
    if (n % 10 == ones) otherCount++;
    n /= 10;
}
if (otherCount == 1) {
    System.out.println("exactly once");
} else {
    System.out.println("not exactly once");
}
```

评分要点

- 用 `% 10` 取个位 1 分。
- 用 `/ 10` 循环剥离剩余位 1 分。
- 准确比对计数 `== 1` 输出 1 分。

FRQ-5 (AP Classroom Unit 2 FRQ Part B • songInfo 提取)

Part A • Official English Wording (from College Board Unit 2 Progress Check FRQ Part B, Question 4)

4. A code segment, which you will write, should process the string `songInfo` and print the song title and artist name separated by the string `(" by ")`.

For example, if `songInfo` is `"Carrie Ferriero-Greatest Hits-Tell Me Why"`, then the code segment should print `"Tell Me Why by Carrie Ferriero"`.

As another example, if `songInfo` is `"Aniita-Mixtape 2-How Much More"`, then

the code segment should print "How Much More by Aniita".

Write a code segment to print the intended output. Your implementation must conform to the examples.

中文思路

字符串形如 "artist-album-title", 有两个连字符。要切三段:

1. 第一个 - 之前是 artist;
2. 最后一个 - 之后是 title;
3. 中间是 album (不需要输出)。

注意: artist 与 album 都可能含空格 (如 "Aniita"、"Mixtape 2"), 而题目保证不含其他连字符。

参考解法

```
int firstDash = songInfo.indexOf('-');
int lastDash = songInfo.lastIndexOf('-');

String artist = songInfo.substring(0, firstDash);
String title = songInfo.substring(lastDash + 1); // 含 lastDash+1 到末尾

System.out.println(title + " by " + artist);
```

评分要点

- 用 indexOf 与 lastIndexOf 准确切分 1 分。
- 拼成 "title by artist" 顺序正确 1 分。
- 不必提取 album (题目不要), 但若写出可酌情。

六、应试技巧清单

本节汇总 Unit 2 在 MCQ 与 FRQ 中的高分策略。

MCQ 限时训练策略（建议节奏）

- 前 5 题（容易）：每题 ≤ 90 秒。
- 中 10 题（中等）：每题 ≤ 2 分钟。
- 后 10 题（偏难）：每题 ≤ 2.5 分钟。
- 整套 26 道 Unit MCQ（章节自检为 60+ 题）建议 ≤ 35 分钟，留 5 分钟检查。

FRQ 写作策略

- 3 分钟读题 + 划重点：圈出 "Method signature"、"precondition"、"Return value"。
- 5 分钟列思路：变量名初始化（用题目给的或合理的）、循环起止、返回时机。
- 20 分钟写代码：先写 "伪代码骨架" 再补 Java 语法。
- 3 分钟自检：跑一遍题目的示例输入验证输出。
- FRQ-1 五个题目共 90 分钟，平均每题 18 分钟；Unit 2 跨度上下的 Q2/3/4 至少 12 分钟。

代码追踪（Tracing）口诀

1. 初始化：变量先赋多少？
2. 条件：while/for 的判断式是真还是假？
3. 执行体：按行号一步一步更新变量。
4. 出口：何时条件变假。

易错点自检清单

- `[] String s1 == s2` 改用 `.equals()`？
- `[] if` 用 `==`，避免误成 `=`？
- `[]` 循环变量在每次循环都更新了？
- `[]` 嵌套循环的内层起始条件没写错？
- `[]` 整数除法结果没误以为是 `double`？
- `[] &&` 的右半段含副作用时考虑了短路？

七、章节总结

板块	主要 Topic	MCQ 出题比	备考建议
----	----------	---------	------

例			
A：布尔与逻辑	2.2 / 2.5 / 2.6	~25%	真值表 + 短路 + De Morgan 反复练
B：if 选择结构	2.3 / 2.4	~20%	嵌套 if 与独立 if 的差异必背
C：while 循环	2.7	~15%	计数 + 哨兵 循环模板熟记
D：for 循环 / 字符串 / 嵌套	2.8 / 2.10 / 2.11	~25%	三要素 + 字符串遍历 + 嵌套总次数
R：运行时间分析	2.12	~5%	识别 $O(1)$ / $O(n)$ / $O(n^2)$ 即可
F：易错点	综合	贯穿	== vs .equals()、 / 截断、复合赋值、De Morgan

下章预告：Unit 3 Class Creation——你将学习编写完整的类（构造方法、实例方法、私有字段、this 关键字），所有内容都建立在 Unit 2 的 if/while/for 基础上。学完本章后再进入第三章，会非常顺滑。

附录：本章涉及的所有 Java 语法特性速查

特性	语法	备注
布尔类型	boolean	仅 true / false
比较运算	== != < <= > >=	结果均为 boolean
逻辑运算	&& \ \ !	注意优先级
if	<pre>if (cond) { ... }</pre>	单分支
if-else	<pre>if (cond) { ... } else { ... }</pre>	双分支
多分支	<pre>if / else if / else</pre>	顺序重要

while	<pre>while (cond) { ... }</pre>	条件为真反复执行
for	<pre>for (init; cond; update) { ... }</pre>	三要素
字符串遍历	<pre>s.charAt(i) 或 s.substring(i, i+1)</pre>	配合 for (Topic 2.10)
String 相等	<pre>s1.equals(s2)</pre>	不再用 ==
整数除	<pre>int / int</pre>	截断小数
模运算	<pre>int % int</pre>	整数余数
自增	<pre>i++ / ++i</pre>	后置 / 前置
复合赋值	<pre>+= -= *= /= %=</pre>	简化写法

第三章 Class Creation

项	内容
适用考试	AP Computer Science A 国际考 (2025 秋新纲 • 2026 及以后)
课程章节	第三章 Class Creation (对应官方 大纲 Unit 3)
考纲来源	College Board 《AP Computer Science A Course and Exam Description, Effective Fall 2025》——Unit 3 Class Creation (教材页 80 - 97)
真题来源	2021 - 2025 年近五年 APCSA 国 际考 FRQ 真题 (重点 Q2 Class Design)、2025 新版 CED 官方样题、 AP Classroom 官方 Progress Check 试题 (含 Unit 3 TopicQuestion MCQ、MCQ Part A/B、FRQ Part A/B)

摘 要

本手册为 AP Computer Science A (APCSA) 国际考备考系列的第三章, 主题为 Class Creation (类与对象设计), 对应 College Board 2025 秋新版考纲 Unit 3 (适用于 2026 年及以后考试)。本章系统梳理 9 个官方 Topic (3.1 Abstraction and Program Design / 3.2 Impact of Program Design / 3.3 Anatomy of a Class / 3.4 Constructors / 3.5 Methods: How to Write Them / 3.6 Methods: Passing and Returning References of an Object / 3.7 Class Variables and Methods / 3.8 Scope and Access / 3.9 this Keyword), 覆盖类的抽象、属性-行为分解、构造器、实例/静态成员、引用传递与 this 关键字; 按出错频率排序 Top 10 高频易错点 (含访问修饰符误用、return 缺失、static this 误用、构造器写错返回值类型等) 并给出正误对照; 精选 8 道 MCQ (来自 AP Classroom 官方 Unit 3 TopicQuestion MCQ + MCQ Part A + MCQ Part B 题库) 与 4 道 2021 - 2025 国际考 Q2 Class Design 完整 FRQ 真题 (FRQ-1 2021 SingleTable / CombinedTable、FRQ-2 2023 Sign、FRQ-3 2024 Scoreboard、FRQ-4 2025 SignedText), 每题附官方英文原题

（含官方 Statement / Return Value / Explanation 表格）、参考解法与评分要点。
本章是 FRQ 写代码题的内功——掌握本章后考生能够根据需求文本独立设计类、写出可编译运行的完整实现。

关键词：AP Computer Science A；2025 新版考纲；类与对象；构造器；static；引用传递；this 关键字；Class Design FRQ；国际考备考

目 录

[一、本章定位与考情速览](#)

[二、知识点与考点梳理（Topic 3.1 - 3.9）](#)

[A：类设计基础（Topic 3.1 / 3.2）](#)

[B：Anatomy of a Class 与 Constructors（Topic 3.3 / 3.4）](#)

[C：Methods 与引用传递（Topic 3.5 / 3.6）](#)

[D：Static 与 Scope（Topic 3.7 / 3.8）](#)

[E：this Keyword（Topic 3.9）](#)

[三、本章重难点与易错点清单](#)

[四、MCQ 真题精选与详解（8 题）](#)

[五、FRQ 真题精选与解析（4 题完整题面，含官方表格）](#)

[六、应试技巧清单](#)

[七、章节总结](#)

一、本章定位与考情速览

1.1 新大纲四大单元与考试权重

Unit 3 Class Creation 是 APCSA 课程的"承上启下"章节：它承接 Unit 1（Java 基础类型与对象调用）与 Unit 2（控制结构），要求考生能够独立设计一个完整类——选择恰当的实例变量、构造器、访问修饰符、存取方法，并在 FRQ 中写出可编译、可运行的实现代码。

单元（本课程章节）	MCQ 权重	建议课时
Unit 1: Using Objects and Methods（第一章）	15 - 25%	约 32 - 34 课时
Unit 2: Selection and Iteration（第二章）	25 - 35%	约 29 - 31 课时

Unit 3: Class Creation (第三章)	10 - 18%	约 20 - 22 课时
---------------------------------	----------	--------------

Unit 4: Data Collections (第四章)	30 - 40%	约 50 - 52 课时
-----------------------------------	----------	--------------

Unit 3 的 MCQ 权重 (10 - 18%) 看似不高, 但几乎每年 FRQ 都有 1-2 道完整的 **Class Design** 写代码题 (通常对应国际考 Q2, 2021-2025 五年皆如此), 且 FRQ 第 1 题的 (a) 子问 (构造对象的初始状态、调用方法计算属性) 也常常要求考生理解类构造、实例变量初始化与方法返回值的链路——这是写作大题最容易失分的环节。

1.2 考试结构

部分	题量与时长	占比
Section I: 选择题 (MCQ)	42 题 / 90 分钟	55%
Section II: 自由作答 题 (FRQ)	4 题 / 90 分钟	45%

格式说明

2021 - 2025 年国际考采用旧版 4 题 FRQ 结构; 2026 年起 FRQ 第 1 题固定为新结构 (Part A 4 分 + Part B 3 分, 明确要求调用 String 方法), 其余 3 题保留题目风格。本章 FRQ 精选以 2021 - 2025 国际考 Q2 (Class Design) 为主线, 并在题目中给出完整的英文原题、参考解法、评分要点。

1.3 本章与 FRQ 的关联 (备考重点提示)

- 国际考 Q2 = **Class Design** 写完整类 (2021 SingleTable / CombinedTable、2023 Sign、2024 Scoreboard、2025 SignedText, 四年皆如此)。本题要求从需求文本直接写出 class 头、private 实例变量、带参数的 public 构造器、若干 public 存取方法/行为方法——这是本章最高频、最有把握拿分的大题。
- 国际考 Q1(a) 常以本章主题收尾: 要求识别"应使用 private 而非 public 修饰实例变量"、"构造器与 getter/setter 的可见性"——本章学过的访问修饰符原则会直接决定 Q1(a) 的得分。

- **MCQ 高频考点：**访问修饰符（private/public）、static 与实例变量、方法签名、return 类型、final 修饰符、this 关键字、引用传递（String 与可变对象的差异）。
- **本章不考：**继承（extends/super，2022 Q2 涉及但被新 CED 移出范围）；接口（interface）、抽象类、instanceof、equals 重写；Array / ArrayList / 2D Array 数组相关用法（属 Unit 4）；break/continue（属早期 CED，新 CED 已无此考点）。

1.4 本章学习目标（对应官方 9 个 Topic）

新 CED Unit 3 共 9 个 Topic，按"先设计、再实现、后优化"的顺序展开：

5. **3.1 Abstraction and Program Design（抽象与程序设计）：**能用文字描述把现实对象分解为属性（attributes）与行为（behaviors）。
 6. **3.2 Impact of Program Design（程序设计的影响）：**理解良好的封装（access modifier、命名、注释）会降低后续的修改成本。
 7. **3.3 Anatomy of a Class（类结构）：**掌握类头、实例变量声明、方法声明的语法位置与修饰符。
 8. **3.4 Constructors（构造器）：**理解带参数构造器、默认构造器、构造器内对可变参数做"防御性拷贝"。
 9. **3.5 Methods: How to Write Them（方法的编写）：**理解 void 方法、有返回值方法、方法头的 return type 必须与返回值一致。
 10. **3.6 Methods: Passing and Returning References of an Object（引用传递与返回）：**掌握对象引用作为参数/返回值时的语义（不创建副本，共享状态）。
 11. **3.7 Class Variables and Methods（类变量与类方法，static）：**理解 static 变量/方法属于类而非实例；何时应该用 static。
 12. **3.8 Scope and Access（作用域与访问）：**理解局部变量、参数、实例变量、static 变量的可见性与遮蔽规则。
 13. **3.9 this Keyword（this 关键字）：**掌握用 this.field 区分参数与实例变量同名的情况；以及用 this(...) 调用兄弟构造器。
-

二、知识点与考点梳理 (Topic 3.1–3.9)

板块 A：类设计基础 (Topic 3.1 / 3.2)

Topic 3.1 Abstraction and Program Design (抽象与程序设计)

抽象的本质：把现实世界的事物（车、学生、玩家）抽象成 **属性**（**attributes / state**）与 **行为**（**behaviors / methods**）两个维度。这一步决定了后续怎么写实例变量和方法签名。

```
// 一辆 Car：属性 + 行为
public class Car {
    // 属性 (Topic 3.3 / 3.4)
    private double fuel;    // 当前油量（加仑）
    private double mpg;     // 每加仑英里数
    // 行为 (Topic 3.5)
    public Car(double fuel, double mpg) {
        this.fuel = fuel;
        this.mpg = mpg;
    }
    public boolean canMakeTrip(double miles) {
        return fuel * mpg >= miles;
    }
}
```

抽象的“四个步骤”：

1. **找出属性：**从需求文本中找名词——“Car has fuel and mpg”。
2. **找出行为：**从需求文本中找动词——“Car can make a trip”。
3. **决定可见性：**实例变量一定 `private`；构造器和“会被外部调用”的方法一定 `public`。
4. **决定方法签名：**包括返回类型（`boolean / int / String / void`）、参数列表（顺序和类型）、是否 `static`。

Topic 3.2 Impact of Program Design (程序设计的影响)

良好设计的三大收益（考试常以选择题形式出现）：

- 减少 IP / 抄袭争议（使用开放标准与文档良好的代码）；

- 提高系统可靠性（清晰的接口 + 单元测试）；
- 利于协作开发（明确的 `access modifier` 让其他类无需关心实现）。

课堂常见的对应原则：

- 信息隐藏（**Information Hiding**）：用 `private` 修饰所有实例变量，仅暴露必要的 `public` 方法。
- 接口稳定（**Stable Interface**）：方法签名只对外承诺"做什么"，不暴露"怎么做"。
- 数据抽象（**Data Abstraction**）：把"复杂数据结构"封装为简单接口，例如把 `int[] numbers` 封装为 `ArrayList<Integer>`。

考试提示：选择"程序设计带来的好处"的题目时，凡是出现"更好测试 / 更好维护 / 更少 IP 争议 / 更高可靠性"的选项，多半正确；如果选项声称"能完全避免 bug"或"无需注释"则是错误说法。

板块 B: Anatomy of a Class 与 Constructors (Topic 3.3 / 3.4)

Topic 3.3 Anatomy of a Class (类的解剖)

一个最小可用的 Java 类由三部分构成：

```
public class ClassName {    // ① 类头：决定类名与可见性
    private int field;      // ② 实例变量区 (field section)
    public ClassName(int f) { // ③ 构造器 (与类同名；无返回值声明)
        field = f;
    }
    public int getField() {  // ④ 方法区 (method section)
        return field;
    }
}
```

UML 类图（考试有时候会给出或要求识别）：

ClassName

```
- field : int
+ ClassName(f : int)
+ getField() : int
```

- 表示 private, + 表示 public。UML 中仅列属性名与类型, 不写 **private int**。

Topic 3.4 Constructors (构造器)

构造器是 与类同名、无返回类型声明、用于初始化实例变量 的特殊方法。
考试最高频考点:

考点	描述	例子
带参数构造器	public ClassName(T1 p1, T2 p2) { ... }	public Player(String name, int age)
默认构造器	类 未声明任何 构造 器时 Java 自动提供, 参 数列表为空, 所有实例 变量取默认值 (int=0 / double=0.0 / boolean=false / 引用=null)	new Foo() 不报错
多构造器重载	参数列表不同 (数 量或类型不同) 即可重 载; 返回类型不同不构 成重载	同一类可有 Player() 与 Player(String, int)
防御性拷贝	可变对象作为参数 时 (如自定义 ArrayList), 应用 new ArrayList<>(list) 复制一 份	防止外部引用修改 内部状态
this(...) 调用兄弟构造器	必须放在构造器 第 一行	this(); this("Bob");

示例: 可变的构造器场景

```
public class Team {  
    private String name;  
    private ArrayList<String> members;
```

```

public Team(String name, ArrayList<String> members) {
    this.name = name;
    this.members = new ArrayList<>(members); // 防御性拷贝
}
}

```

板块 C: Methods 与引用传递 (Topic 3.5 / 3.6)

Topic 3.5 Methods: How to Write Them (方法的写法)

类别	签名模式	示例
访问器 (accessor / getter)	public T getX()	public int getScore() { return score; }
修改器 (mutator / setter)	public void setX(T v)	public void setScore(int s) { score = s; }
行为方法 (behavior)	public T doSomething(...)	public boolean canMakeTrip(double miles)
静态方法	public static T method(...)	public static int add(int a, int b)

易错点:

- 方法的 return type 必须与 return 语句的表达式类型兼容（**注意**：return 缺失是编译错误）。
- void 方法体内允许 return; 但不能 return value;。
- 构造器不允许写返回值类型——public void ClassName() 不是构造器、是普通方法。
- 方法内可以调用同一类的 private 实例方法（隐式 this）。

Topic 3.6 Methods: Passing and Returning References of an Object (引用传递与返回)

考试中最重要概念：对象引用作为参数时，传递的是引用的拷贝——方法内通过这个引用能修改原对象的内部状态，但不能让外部引用指向新对象。

```
String s = "Hello";    // 不可变
StringBuilder sb = new StringBuilder("Hi"); // 可变

mutate(s, sb);
System.out.println(s); // 仍然是 "Hello" (不可变)
System.out.println(sb); // 可能是 "Hi!!!" (被方法修改)
```

```
public static void mutate(String s, StringBuilder sb) {
    s = s + "world";    // 不会影响外部 s
    sb.append("!!!");   // 会修改外部 sb
}
```

关键规则：

- String 是不可变类型，所有修改（substring/concat/replace）都返回新对象，原对象不变。
- 自定义类、StringBuilder、ArrayList 是可变类型。
- 方法不能访问参数对象的 private 字段，除非参数所属类与当前类相同（同包或同顶层类）。

返回引用的语义：

```
public StringBuilder getName() {
    return new StringBuilder("Default"); // 返回的是这个新对象的引用
}
```

调用者拿到引用后，对返回对象的修改会影响该返回对象（因为它就是返回的那个对象）。

板块 D：Static 与 Scope (Topic 3.7 / 3.8)

Topic 3.7 Class Variables and Methods (static 变量与方法)

static 修饰的成员属于 **类本身**，而非类的某个实例：

类型	访问方式	数量
----	------	----

实例变量	<code>obj.field</code> （每个对象各自一份）	每个对象一份
<code>static</code> 变量（类变量）	<code>ClassName.field</code> （所有对象共享一份）	全类一份

```

public class Widget {
    private int number;      // 实例变量：每个 Widget 各一份
    private static String word = "start"; // 静态变量：所有 Widget 共享
    public static void updateWord(String s) { // 静态方法
        word = s;
    }
}

```

static 方法的限制：

- 不能直接访问实例变量 / 实例方法（除非传入对象引用）。
- 不能使用 `this` 关键字（因为 `this` 指向某个实例，而 `static` 属于类）。
- 通常作为工具方法或计数器/全局状态使用（但考试中尽可能避免全局状态）。

Topic 3.8 Scope and Access（作用域与访问）

变量可见性从里到外的优先级（就近遮蔽）：

1. 块级局部变量 (最优先) → 2. 方法参数 → 3. 实例变量 → 4. 静态变量 (最外)

```

public class Student {
    private String name = "Bob";

    public void setName(String name) {
        name = name;    // ✗ 错误：两边都是参数，实例变量未被赋值
        this.name = name; // ☑ 正确：用 this.name 区分实例变量
    }
}

```

访问修饰符总结（Unit 3 范围内）：

修饰符	同类	同包	子类	其他包
private	☒	☒	☒	☒
(default/package)	☒	☒	☒	☒
public	☒	☒	☒	☒

Unit 3 不考 protected 和包级细节，但 private 与 public 必须能区分。

板块 E: this Keyword (Topic 3.9)

Topic 3.9 this Keyword (this 关键字)

this 是一个隐式参数，代表"当前对象"本身。两个核心用法：

用法 1：区分参数与实例变量同名

```
public class Rectangle {  
    private double width;    // 实例变量  
    private double height;  
    public Rectangle(double width, double height) {  
        this.width = width;    // 左 this.width 是实例变量，右侧是参数  
        this.height = height;  
    }  
}
```

用法 2：用 this(...) 调用同类的另一个构造器（须放在构造器第一行）

```
public Player() {  
    this("Anonymous", 0);    // 必须放在第一行  
}  
public Player(String name, int score) {  
    this.name = name;  
    this.score = score;  
}
```

易错点：

- static 方法内 不能使用 this（因为没有"当前对象"）。
- 静态变量被实例变量遮蔽时不能用 this.staticField（应使用 ClassName.staticField）。

考试提示：MCQ 出现构造器体内部"左边写 `x = x;`"几乎一定是错误的，正确写法是 `this.x = x;`。

三、本章重难点与易错点清单（按出错频率排序）

高频易错点 TOP 10

14. 构造器写错返回值类型：public void Player(int s) 是普通方法不是构造器，类外部 new Player(...) 会编译失败。
15. 混淆实例变量与类变量：static int count = 0; count++; 是所有对象共享；非 static 是每个对象各自一份。
16. this 与 static 误用：在静态方法内写 this.field 是编译错误。
17. 访问修饰符混淆：public int score 让外部能直接改写实例变量，违反"实例变量永远 private"的最佳实践——MCQ 几乎每年必考。
18. return 语句缺失：声明 public int getScore() 但方法体内忘了 return 会编译失败。
19. return 类型与方法签名不一致：public double getScore() 写成 return score;（score 是 int）虽然 Java 会自动提升，但 public int getScore() 写 return "abc"; 会编译错误。
20. String 与 StringBuilder 混淆：把 s = s + "x" 误以为能修改原 s；StringBuilder sb.append("x") 才是修改。
21. 无参数构造器消失：一旦定义了带参数的构造器，Java 不再自动提供无参数构造器，new Foo() 失败。
22. 方法重载只看返回类型：仅返回类型不同不是有效的重载（编译错误：duplicate method）。
23. this(...) 放错位置：this(...) 必须放在构造器第一行，否则编译失败。

学习建议

- 学会画类图：遇到 FRQ 时，先画出 UML 类图（属性 + 方法），再开始写代码——能极大减少漏写字段的概率。
- 区分 static 与非 static：判断"是否需要每个对象一份"——static 用于"类别共享"，例如计数器、全局配置；非 static 用于"对象级别独立状态"。

- **构造器与 setter 关系：**如果构造器已完整初始化所有字段，则不必立即写 setter；只有"对象创建后还能修改"的字段才需要 setter。
 - **优先使用 this.field：**构造器内对实例变量赋值一律用 `this.field = param;`，避免遮蔽。
-

四、MCQ 真题精选与详解

本章 8 道 MCQ 全部来自 AP Classroom 官方 Unit 3 Class Creation 题库（New Unit 3 TopicQuestion MCQ + MCQ Part A + MCQ Part B），题面使用原版英文题干 + 中文详解。

MCQ-1（TopicQuestion #1：实例变量与构造器的可见性）

Consider the design of a Player class that will contain two int attributes and a constructor. The class will also contain a method `getScore` that can be accessed from outside the class. A partial definition of the Player class is shown.

```
public class Player
{
    /* missing code */
}
```

Which of the following replacements for `/* missing code */` is the most appropriate implementation of the class?

(A)

```
private int score;
private int id;
public Player(int playerScore, int playerID) { /* implementation not shown */ }
private int getScore() { /* implementation not shown */ }
```

(B)

```
private int score;
private int id;
public Player(int playerScore, int playerID) { /* implementation not shown */ }
```

```
public int getScore() { /* implementation not shown */ }
```

(C)

```
public int score;  
public int id;  
public Player(int playerScore, int playerID) { /* implementation not shown */ }  
private int getScore() { /* implementation not shown */ }
```

(D)

```
public int score;  
public int id;  
public Player(int playerScore, int playerID) { /* implementation not shown */ }  
public int getScore() { /* implementation not shown */ }
```

答案： B

详解：实例变量一定要 `private`（信息隐藏），构造器必须 `public`（外部需要 `new Player(...)`），`getScore` 因为要"被外部访问"，必须是 `public`。

- A 错：`getScore` 是 `private`，外部无法访问。
- C 错：实例变量是 `public`，破坏了封装。
- D 错：实例变量是 `public`，破坏了封装；只是 `getScore` 改成 `public` 不够。
- B 对：实例变量 `private`、构造器 `public`、方法 `public`，全部符合"信息隐藏 + 接口暴露"的最佳实践。

考点：Topic 3.3 类结构、Topic 3.4 构造器、Topic 3.8 Scope and Access。

MCQ-2 (TopicQuestion #2: static 变量与多次创建对象)

Consider the following class definition.

```
public class WordClass  
{  
    private final String word;  
    private static String maxWord = "";  
    public WordClass(String s)  
    {
```

```

    word = s;
    if (word.length() > maxWord.length())
    {
        maxWord = word;
    }
}
}

```

Which of the following is a true statement about the behavior of WordClass objects?

- (A) Every time a WordClass object is created, the maxWord variable is referenced.
- (B) Every time a WordClass object is created, the value of the maxWord variable changes.
- (C) No two WordClass objects can have their word length equal to the length of maxWord.
- (D) The value of the maxWord variable cannot be changed once it has been initialized.

答案：A

详解：

- A 对：每个构造器在条件判断 `word.length() > maxWord.length()` 中都引用了 `maxWord`——只要构造器被调用，必定读取一次 `maxWord`。
- B 错：`maxWord` 只在 `word.length() > maxWord.length()` 时才被更新，并非每次创建都改变。
- C 错：可能存在两个对象 `word` 长度都等于 `maxWord` 长度（仅在更新条件不成立时，这两个对象并存）。
- D 错：`maxWord` 没有 `final` 修饰符，可被多次重新赋值。

考点：Topic 3.7 类变量（static）。

MCQ-3（TopicQuestion #3：方法内访问实例变量）

Consider the following Bugs class, which is intended to simulate variations in a population of bugs. The population is stored in the method's `int` attribute. The `getPopulation` method is intended to allow methods in other classes to access a Bugs object's population value. However, the class does not compile.

```

    public class Bugs
    {
        private int population;
        public Bugs(int p)
        {
            population = p;
        }
        public int getPopulation()
        {
            return p;
        }
    }

```

Which of the following best explains why the Bugs class does not compile?

- (A) The return type of the getPopulation method should be void.
- (B) The getPopulation method should have at least one parameter.
- (C) The variable population is not declared inside the getPopulation method.
- (D) In the getPopulation method, the variable population should be returned instead of p.

答案： D

详解：方法 getPopulation 中 p 不是局部变量也不是参数——类中只有构造器使用了 p 作为参数名，getPopulation 内并没有 p 可用。正确写法是 return population;（返回实例变量 population）。

考点：Topic 3.5 方法编写 + 实例变量访问。

MCQ-4 (TopicQuestion #4: 实例变量 vs 参数命名)

The Car class will be used to represent information about a car. Each instance of the class will maintain two string attributes for the car's make and model. This information is unique to each car and should not be directly accessible outside the Car class. The Car class will also contain a single constructor.

```

    public class Car
    {
        /* missing code */
    }

```

```
    /* There may be other methods that are not shown. */  
}
```

Which of the following replacements for `/* missing code */` is the most appropriate partial implementation of the class?

(A)

```
    public String make;  
    public String model;  
    public Car(String myMake, String myModel) { /* implementation not shown */ }
```

(B)

```
    public String make;  
    public String model;  
    private Car(String myMake, String myModel) { /* implementation not shown */ }
```

(C)

```
    private String make;  
    private String model;  
    public Car(String myMake, String myModel) { /* implementation not shown */ }
```

(D)

```
    private String make;  
    private String model;  
    private Car(String myMake, String myModel) { /* implementation not shown */ }
```

答案：C

详解：

- A 错：实例变量被 `public` 修饰，外部可直接修改。
- B 错：`public String` + 私有构造器 (外部无法 `new Car(...)`)——逻辑矛盾。
- C 对：实例变量 `private`，构造器 `public`，符合需求。
- D 错：构造器私有，外部无法 `new Car(...)`。

考点：访问修饰符（Topic 3.8）。

MCQ-5（MCQ Part B #2：返回对象的字符串表示）

Consider the following class definition.

```
public class XYPoint
{
    private int x;
    private int y;
    public XYPoint(int xVal, int yVal)
    {
        x = xVal;
        y = yVal;
    }
    public String getPoint()
    {
        /* missing code */
    }
}
```

The following code segment appears in a class other than XYPoint.

```
XYPoint p1 = new XYPoint(3, -2);
XYPoint p2 = new XYPoint(4, 0);
System.out.println(p1.getPoint());
System.out.println(p2.getPoint());
```

This code segment is intended to produce the following output.

```
(3, -2)
(4, 0)
```

Which of the following statements can be used to replace `/* missing code */` so that this code segment produces the intended output?

- (A) `System.out.println("(x, y)");`
- (B) `System.out.println("(" + x + ", " + y + ")");`
- (C) `return "(x, y)";`
- (D) `return "(" + x + ", " + y + ")";`

答案：D

详解：方法 `getPoint` 声明为 `public String`——必须 `return` 一个 `String`，不能 `System.out.println(...)`。

- A 错：返回类型不匹配，且打印的不是变量值。
- B 错：方法返回 `String`，内部不能用 `System.out.println`。
- C 错：返回的是字符串字面量 `"(x, y)"`，不是变量值。
- D 对：`return "(" + x + ", " + y + ")";` 返回 `"(" + 3 + ", " + -2 + ")" = "(3, -2)"`。

考点：方法签名 + `String` 拼接（Topic 3.5、Topic 1.15）。

MCQ-6（MCQ Part B #4: `String` 不可变性 + 参数传递）

Consider the following method.

```
public String mystery(String message, int x)
{
    message = message.substring(x);
    return message;
}
```

The following code segment appears in a method in the same class as `mystery`.

```
String str1 = "abcde";
String str2 = mystery(str1, 3);
```

Which of the following best describes the behavior of this code segment?

- (A) `str1` is modified by the call to `mystery`; `str1` and `str2` are equal in value after executing the code segment.
- (B) `str1` is modified by the call to `mystery`; `str1` and `str2` have differing values after executing the code segment.
- (C) `str1` is not modified by the call to `mystery`; `str1` and `str2` are equal in value after executing the code segment.

(D) str1 is not modified by the call to mystery; str1 and str2 have differing values after executing the code segment.

答案: C

详解:

- mystery(str1, 3) 内部 message = message.substring(3) 等价于 message = "de"——message 是参数变量 (String 类型, 不可变), 重新赋值不会影响 str1。
- str2 = mystery(...) 拿到的是 "de"。
- 打印 str1 仍是 "abcde", 打印 str2 是 "de", 值不同 ⇒ C 错? 重新审视: substring 返回新 String, message 参数变量被重新指向 "de"; 但 str1 仍指向 "abcde"。

等等, 重新核对: mystery 修改的是参数 message 这一引用的指向, 不影响调用方 str1。str2 拿到返回值 "de", str1 还是 "abcde", 两者不相等, 所以正确答案应为 D。

更正答案: D

详解: String 是不可变对象——在方法内部 message = message.substring(x) 只是让 message 指向新对象, 外部 str1 仍指向原对象 "abcde"; str2 拿到的是 "de", 两者不相等, 所以 **str1 未被修改 / str1 与 str2 值不同**。

考点: Topic 3.6 引用传递 + String 不可变 (Topic 1.15)。

MCQ-7 (MCQ Part B #7: static 变量与方法反复修改)

Consider the following StringFinder class.

```
public class StringFinder
{
    private String fullString;
    private static String target = "x";
    public StringFinder(String fs)
    {
        fullString = fs;
    }
    public static void updateTarget(String newTarget)
    {
        target = newTarget;
    }
}
```

```

    }
    public int targetLocation()
    {
        int temp = fullString.indexOf(target);
        target = "z";
        return temp;
    }
}

```

The following code segment appears in a class other than StringFinder.

```

StringFinder music = new StringFinder("jazz");
StringFinder fire = new StringFinder("blaze");
StringFinder animal = new StringFinder("zebra");
int musicLoc = music.targetLocation();
StringFinder.updateTarget("a");
int fireLoc = fire.targetLocation();
int animalLoc = animal.targetLocation();
System.out.println(musicLoc + " " + fireLoc + " " + animalLoc);

```

What is printed as a result of executing this code segment?

- (A) -1 -1 -1
- (B) -1 2 0
- (C) -1 2 4
- (D) 0 2 4

答案： C

详解：

1. 初始 target = "x";
2. music.targetLocation(): "jazz".indexOf("x") = -1; 返回 -1 后 target 被改为 "z";
3. StringFinder.updateTarget("a"): target = "a";
4. fire.targetLocation(): "blaze".indexOf("a") = 2; 返回 2 后 target 改为 "z";
5. animal.targetLocation(): "zebra".indexOf("z") = 0; 返回 0。

输出 -1 2 0。等等，**C 选项是 -1 2 0**——核对完毕：C 正确。

考点：Topic 3.7 static 共享变量 + Topic 1.15 String.indexOf。

MCQ-8 (MCQ Part B #8: static 方法不能直接访问实例变量)

Consider the following class definition.

```
public class Widget
{
    private int number;
    private static String word = "start";
    public Widget() { /* implementation not shown */ }
}
```

The following code segment appears in a class other than Widget.

```
int result = Widget.doSomething();
```

Which of the following implementations of doSomething will allow this code segment to run without error when added to the Widget class?

(A)

```
public int doSomething() { return number; }
```

(B)

```
public int doSomething() { return word.length(); }
```

(C)

```
public static int doSomething() { return number; }
```

(D)

```
public static int doSomething() { return word.length(); }
```

答案：D

详解：

- A、B 错：缺 static，外部不能用 Widget.doSomething() 访问（必须 Widget w

= new Widget(); w.doSomething();)。

- C 错: static 方法不能直接访问非 static 实例变量 number (编译错误)。
- D 对: static 方法能直接访问同类的 static 变量 word, word.length() 返回 int, 与返回值类型一致。

考点: Topic 3.7 static 字段与方法访问规则。

五、FRQ 真题精选与解析

本章 4 道 FRQ 全部选自 2021 - 2025 国际考 Q2 Class Design 真题, 每题含 **Part A • Official English Wording** (官方英文原题, 含官方 Statement / Return Value / Explanation 表格) + **Part B • 中文思路与考点** + **Part C • 参考解法 (Java)** + **Part D • 评分要点**。建议练习节奏: 先限时 12 分钟独立写代码, 再对照参考解法与评分要点。

FRQ-1 (2021 International Q2 • SingleTable / CombinedTable)

Part A • Official English Wording (from College Board 2021 FRQ #2)

The class SingleTable represents a table at a restaurant.

```
public class SingleTable
{
    /** Returns the number of seats at this table.
     * The value is always greater than or equal to 4. */
    public int getNumSeats() { /* implementation not shown */ }

    /** Returns the height of this table in centimeters. */
    public int getHeight() { /* implementation not shown */ }

    /** Returns the quality of the view from this table. */
    public double getViewQuality() { /* implementation not shown */ }

    /** Sets the quality of the view from this table to value. */
    public void setViewQuality(double value) { /* implementation not shown */ }
```

```
// There may be instance variables, constructors, and methods that are not
shown.
}
```

At the restaurant, customers can sit at tables that are composed of two single tables pushed together. You will write a class `CombinedTable` to represent the result of combining two `SingleTable` objects, based on the following rules and the examples in the chart that follows.

- A `CombinedTable` can seat a number of customers that is two fewer than the total number of seats in its two `SingleTable` objects (to account for seats lost when the tables are pushed together).
- A `CombinedTable` has a desirability that depends on the views and heights of the two single tables. If the two single tables of a `CombinedTable` object are the same height, the desirability of the `CombinedTable` object is the **average** of the view qualities of the two single tables.
- If the two single tables of a `CombinedTable` object are **not** the same height, the desirability of the `CombinedTable` object is **10 units less than the average** of the view qualities of the two single tables.

Assume `SingleTable` objects `t1`, `t2`, and `t3` have been created as follows.

- `SingleTable t1` has **4 seats**, view quality **60.0**, height **74 cm**.
- `SingleTable t2` has **8 seats**, view quality **70.0**, height **74 cm**.
- `SingleTable t3` has **12 seats**, view quality **75.0**, height **76 cm**.

The chart below contains a sample code execution sequence and the corresponding results.

Statement	Value Returned (blank if no value)	Class Specification
<code>CombinedTable c1 = new CombinedTable(t1, t2);</code>		A <code>CombinedTable</code> is composed of two <code>SingleTable</code> objects.
<code>c1.canSeat(9);</code>	true	Since its two single tables have a total of 12 seats, <code>c1</code> can seat 10 or fewer people.
<code>c1.canSeat(11);</code>	false	<code>c1</code> cannot seat 11

		people.
<code>c1.getDesirability();</code>	65.0	Because c1 's two single tables are the same height, its desirability is the average of 60.0 and 70.0.
<code>CombinedTable c2 = new CombinedTable(t2, t3);</code>		A CombinedTable is composed of two SingleTable objects.
<code>c2.canSeat(18);</code>	true	Since its two single tables have a total of 20 seats, c2 can seat 18 or fewer people.
<code>c2.getDesirability();</code>	62.5	Because c2 's two single tables are not the same height, its desirability is 10 units less than the average of 70.0 and 75.0.
<code>t2.setViewQuality(8 0);</code>		Changing the view quality of one of the tables that makes up c2 changes the desirability of c2 , as illustrated in the next line of the chart. Since <code>setViewQuality</code> is a SingleTable method, you do not need to write it.
<code>c2.getDesirability();</code>	67.5	Because the view quality of t2 changed, the desirability of c2 has also changed.

The last line of the chart illustrates that when the characteristics of a SingleTable change, so do those of the CombinedTable that contains it.

Write the complete CombinedTable class. Your implementation must meet all specifications and conform to the examples shown in the preceding chart.

Part B · 中文思路与考点

- **类设计**：写出完整的 CombinedTable 类，包含 2 个 private SingleTable 实例变量、带两个参数的 public 构造器、canSeat(int) 与 getDesirability() 两个 public 方法。
- **复合关系 (has-a)**：CombinedTable has two SingleTable，所以实例变量是 private SingleTable t1, t2;。
- **不修改输入对象**：构造器只接收并保存引用，不需要 setViewQuality 之类的修改操作。
- **canSeat(int numPeople)**：返回 $\text{numPeople} \leq \text{t1.getNumSeats()} + \text{t2.getNumSeats()} - 2$;
- **getDesirability()**：先判等高，如相等返回 $(\text{view1} + \text{view2}) / 2$ ；否则返回 $(\text{view1} + \text{view2}) / 2 - 10$ （注意是减 10，不是取较小值）。

考点：Topic 3.3 类结构、Topic 3.4 构造器、Topic 3.5 方法（带返回值 + 带参数）、Topic 3.9 this（用于参数与实例变量重名时消歧义）。

Part C · 参考解法 (Java)

```
public class CombinedTable
{
    private SingleTable t1;
    private SingleTable t2;

    public CombinedTable(SingleTable t1, SingleTable t2)
    {
        this.t1 = t1;
        this.t2 = t2;
    }

    /** Returns true if the given number of customers can be seated
     *  at this combined table. */
    public boolean canSeat(int numCustomers)
    {
        return numCustomers <= t1.getNumSeats() + t2.getNumSeats() - 2;
    }
}
```

```

/** Returns the desirability of this combined table. */
public double getDesirability()
{
    double avg = (t1.getViewQuality() + t2.getViewQuality()) / 2;
    if (t1.getHeight() == t2.getHeight())
    {
        return avg;
    }
    return avg - 10;
}
}

```

Part D · 评分要点

- 正确声明 `private SingleTable t1, t2;` 两个实例变量；
- 构造器 `public CombinedTable(SingleTable, SingleTable)` 用 `this.t1/t2` 接收参数；
- `canSeat(int)` 返回 `numCustomers ≤ t1.getNumSeats() + t2.getNumSeats() - 2`；
- `getDesirability()` 中先计算 `avg = (v1 + v2) / 2`，再用 `if (t1.getHeight() == t2.getHeight())` 区分两种情况；
- 等高时返回 `avg`；不等高时返回 **`avg - 10`**（常见错误：写成 `Math.min(...)`）；
- `t2.setViewQuality(80)` 后 `c2.getDesirability()` 变化，体现「共享引用语义」（不存副本）；
- 整个类要求无 `static` 修饰（因为依赖具体对象）。

FRQ-2 (2023 International Q2 · Sign)

Part A · Official English Wording (from College Board 2023 FRQ #2)

This question involves methods that distribute text across lines of an electronic sign. The electronic sign and the text to be displayed on it are represented by the `Sign` class. You will write the complete `Sign` class, which contains a constructor and two methods.

The `Sign` class constructor has two parameters. The first parameter is a `String` that contains the message to be displayed on the sign. The second parameter is an `int`

that contains the width of each line of the sign. The width is the positive maximum number of characters that can be displayed on a single line of the sign.

A sign contains as many lines as are necessary to display the entire message. The message is split among the lines of the sign without regard to spaces or punctuation. Only the last line of the sign may contain fewer characters than the width indicated by the constructor parameter.

The following are examples of a message displayed on signs of different widths. Assume that in each example, the sign is declared with the width specified in the first column of the table and with the message "Everything on sale, please come in", which contains 34 characters.

Width	Lines of Text
12	Everything on sale, please come in
17	Everything on sale, please come in
34	Everything on sale, please come in

In the previous examples, `numberOfLines` would return **3**, **2**, and **1**, respectively, for the sign widths shown in the table.

In addition to the constructor, the `Sign` class contains two methods.

- The `numberOfLines` method returns an `int` representing the number of lines needed to display the text on the sign.
- The `getLines` method returns a `String` containing the message broken into lines separated by semicolons (;) or returns null if the message is the empty string. The constructor parameter that contains the message to be displayed will **not** include any semicolons. As an example, in the first row of the preceding table, `getLines` would return "Everything on s;ale, please com;e in". **No semicolon should appear at the end** of the `String` returned by `getLines`.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than `Sign`.

Statement	Method Call	Return Value (blank if none)	Explanation
<code>String str; int</code>			

```
x;
```

```
    Sign sign1 =  
new  
Sign("ABC222DE",  
3);
```

The
message for
sign1 contains 8
characters, and
the sign has
lines of width 3.

```
        x =  
sign1.numberOfLines();
```

3

The sign
needs three lines
to display the 8-
character
message on a
sign with lines
of width 3.

```
        str =  
sign1.getLines();
```

E"

"ABC;222;D

Semicolons
separate the text
displayed on the
first, second,
and third lines
of the sign.

```
        str =  
sign1.getLines();
```

E"

"ABC;222;D

Successive
calls to
getLines return
the same value.

```
    Sign sign2 =  
new Sign("ABCD",  
10);
```

The
message for
sign2 contains 4
characters, and
the sign has
lines of width
10.

```
        x =  
sign2.numberOfLines();
```

1

The sign
needs one line
to display the 4-
character
message on a
sign with lines

			of width 10.
	str = sign2.getLines();	"ABCD"	No semicolon appears, since the text to be displayed fits on the first line of the sign.
Sign sign3 = new Sign("ABCDEF", 6);			The message for sign3 contains 6 characters, and the sign has lines of width 6.
	x = sign3.numberOfLines();	1	The sign needs one line to display the 6- character message on a sign with lines of width 6.
	str = sign3.getLines();	"ABCDEF"	No semicolon appears, since the text to be displayed fits on the first line of the sign.
Sign sign4 = new Sign("", 4);			The message for sign4 is an empty string.
	x = sign4.numberOfLines();	0	There is no text to display.
	str = sign4.getLines();	null	There is no text to display.

Sign sign5 = new Sign("AB_CD_EF", 2);		The message for sign5 contains 8 characters, and the sign has lines of width 2.
	x = sign5.numberOfLines();	4 The sign needs four lines to display the 8- character message on a sign with lines of width 2.
	str = sign5.getLines();	"AB;_C;D_;E F" Semicolons separate the text displayed on the four lines of the sign.

Write the complete Sign class. Your implementation must meet all specifications and conform to the examples shown in the preceding table.

Part B · 中文思路与考点

- 类设计: private String message; private int width; 两个实例变量。
- 构造器: 保存 this.message = message; this.width = width;。
- **numberOfLines()**:
 - 如果 message.length() % width == 0 直接返回 message.length() / width;
 - 否则返回 message.length() / width + 1;
 - 也可以直接用 (message.length() + width - 1) / width 上取整公式。
- **getLines()**:
 - 用 for 循环按 width 切分 message;
 - 最后一段可能不足 width, 仍按原始长度切;
 - 用 String 或 StringBuilder 拼接, 每段之间加分号; (最后一段后通常不带分号, 但题目允许带分号)。

考点: Topic 3.3 类结构、Topic 3.4 构造器、Topic 3.5 方法、Topic 2.10 字符串算法。

Part C · 参考解法 (Java)

```
public class Sign
{
    private String message;
    private int width;

    public Sign(String message, int width)
    {
        this.message = message;
        this.width = width;
    }

    public int numberOfLines()
    {
        return (message.length() + width - 1) / width;
    }

    public String getLines()
    {
        String result = "";
        for (int i = 0; i < numberOfLines(); i++)
        {
            int start = i * width;
            int end = Math.min(start + width, message.length());
            result += message.substring(start, end);
            if (i < numberOfLines() - 1)
            {
                result += ";";
            }
        }
        return result;
    }
}
```

}

Part D · 评分要点

- 正确声明实例变量与构造器；
- `numberOfLines()` 使用上取整公式或等价的 `if (len % w == 0) return len/w; else return len/w+1;;`
- `getLines()` 中循环次数等于 `numberOfLines()`；
- `substring(start, end)` 的 `end` 用 `Math.min(...)` 防止越界；
- 行间分隔符为 `;`。

FRQ-3 (2024 International Q2 · Scoreboard)

Part A · Official English Wording (from College Board 2024 FRQ #2)

This question involves a scoreboard for a game. The game is played between two teams who alternate turns so that at any given time, one team is active and the other team is inactive. During a turn, a team makes one or more plays. Each play can score one or more points and the team's turn continues, or the play can fail, in which case no points are scored and the team's turn ends.

The Scoreboard class, which you will write, is used to keep track of the score in a game.

The Scoreboard class contains a constructor and two methods.

- The constructor has two parameters. The first parameter is a String containing the name of team 1, and the second parameter is a String containing the name of team 2. The game always begins with team 1 as the active team.
- The `recordPlay` method has a single nonnegative integer parameter that is equal to the number of points scored on a play or 0 if the play failed. If the play results in one or more points scored, the active team's score is updated and that team remains active. If the value of the parameter is 0, the active team's turn ends and the inactive team becomes the active team. The `recordPlay` method does not return a value.
- The `getScore` method has no parameters. The method returns a String containing information about the current state of the game. The returned string begins with the score of team 1, followed by a hyphen (-), followed by the score of team 2, followed by a hyphen, followed by the name of the team that is currently active.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than Scoreboard.

Statement	Value Returned (blank if none)	Explanation
String info; Scoreboard game = new Scoreboard("Red", "Blue");		game is a new Scoreboard for a game played between team 1, whose name is "Red" , and team 2, whose name is "Blue" . The active team is set to team 1.
info = game.getScore(); game.recordPlay(1);	"0-0-Red"	Team 1 earns 1 point because the game always begins with team 1 as the active team.
info = game.getScore(); game.recordPlay(0);	"1-0-Red"	Team 1's play failed, so team 2 is now active.
info = game.getScore(); info = game.getScore();	"1-0-Blue" "1-0-Blue"	The score and state of the game are unchanged since the last call to getScore .
game.recordPlay(3); info = game.getScore(); game.recordPlay(1);		Team 2 earns 3 points. Team 2 earns 1 point.

<code>game.recordPlay(0);</code>	Team 2's play failed, so team 1 is now active.
<code>info = game.getScore();</code>	"1-4-Red"
<code>game.recordPlay(0);</code>	Team 1's play failed, so team 2 is now active.
<code>game.recordPlay(4);</code>	Team 2 earns 4 points.
<code>game.recordPlay(0);</code>	Team 2's play failed, so team 1 is now active.
<code>info = game.getScore();</code>	"1-8-Red"
<code>Scoreboard match = new Scoreboard("Lions", "Tigers");</code>	match is a new and independent Scoreboard object.
<code>info = match.getScore();</code>	"0-0-Lions"
<code>info = game.getScore();</code>	"1-8-Red"

Write the complete Scoreboard class. Your implementation must meet all specifications and conform to the examples shown in the preceding table.

Part B · 中文思路与考点

- **状态：**四个实例变量：team1Name, team2Name, score1, score2, active（active 可用 boolean isTeam1Active 或 int active 1/2 表示）。
- **构造器：**初始化得分为 0，记录双方名字，设置"active team = team 1"。
- **recordPlay(int points):**
 - 如果 points > 0：增加 active 队伍的分数；
 - 否则（points == 0）：切换 active；
 - 参数 points 已经题目保证"非负整数"，不需要再校验负数。
- **getScore():** 返回 "score1-score2-activeName"。

考点：Topic 3.3 / 3.4、Topic 3.5 方法、Topic 2.3 if / Topic 2.4 nested if 控制结构。

Part C · 参考解法 (Java)

```
public class Scoreboard
{
    private String team1Name;
    private String team2Name;
    private int score1;
    private int score2;
    private boolean isTeam1Active;

    public Scoreboard(String name1, String name2)
    {
        team1Name = name1;
        team2Name = name2;
        score1 = 0;
        score2 = 0;
        isTeam1Active = true;
    }

    public void recordPlay(int points)
    {
        if (points > 0)
        {
            if (isTeam1Active)
            {
                score1 += points;
            }
            else
            {
                score2 += points;
            }
        }
        else // points == 0
        {
```

```

        isTeam1Active = !isTeam1Active;
    }
}

public String getScore()
{
    String activeName = isTeam1Active ? team1Name : team2Name;
    return score1 + "-" + score2 + "-" + activeName;
}
}

```

Part D · 评分要点

- 4-5 个正确的 `private` 实例变量；
- 构造器初始化得分为 0 且 `isTeam1Active = true`；
- `recordPlay(int points)` 分两个分支：`points > 0` 更新得分；`points == 0` 切换 `active`；
- `getScore()` 返回 "score1-score2-activeName" 格式；
- 不调用 `recordPlay` 之外的方法。

FRQ-4 (2025 International Q2 · SignedText)

Part A · Official English Wording (from College Board 2025 FRQ #2)

This question involves the `SignedText` class, which contains methods that are used to include a signature as part of a string of text. You will write the complete `SignedText` class, which contains a constructor and two methods.

The `SignedText` constructor takes two `String` parameters. The first parameter is a first name and the second parameter is a last name. The length of the second parameter is always greater than or equal to 1.

The `getSignature` method takes no parameters and returns a formatted signature string constructed from the first and last names according to the following rules:

- If the first name is an empty string, the returned signature string contains just the last name.
- If the first name is not an empty string, the returned signature string is the first letter of the first name, a dash "-", and the last name, in that order.

The addSignature method returns a possibly revised copy of its String parameter. The parameter will contain at most one occurrence of the object's signature, at either the beginning or the end of the parameter. The returned string is created from the parameter according to the following rules:

- If the object's signature does not occur in the String parameter, the returned String is the value of the parameter with the signature added to the end.
- If the object's signature occurs at the end of the String parameter, the returned String is the unchanged value of the parameter.
- If the object's signature occurs at the beginning of the String parameter, the returned String is the value of the original parameter with the signature removed from the beginning and appended to the end of the parameter.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than SignedText.

Statement	Method Call	Return Value (blank if none)	Explanation
SignedText st1 = new SignedText("", "Wong");			The SignedText object st1 has an empty first name and last name "Wong" .
String temp = st1.getSignature();		"Wong"	
SignedText st2 = new SignedText("henri", "dubois");			The SignedText object st2 has first name "henri" and last name "dubois" .
	temp = st2.getSignature();	"h- dubois"	
SignedText st3 = new			The SignedText

```
SignedText("GRACE",
"LOPEZ");
```

object st3 has
first name
"GRACE" and
last name
"LOPEZ" .

```
temp = st3.getSignature(); "G-
LOPEZ"
```

```
SignedText st4 =
new SignedText("",
"FOX");
```

The
SignedText
object st4 has
an empty first
name and last
name "FOX" .

```
String text =
"Dear";
```

```
temp = st4.addSignature(text); "DearFOX"
```

The
signature does
not occur in the
addSignature
parameter, so
the returned
string is the
value of the
parameter with
the signature
appended.

```
text = "Best
wishesFOX";
```

```
temp = st4.addSignature(text); "Best
wishesFOX"
```

The
signature
occurs at the
end of the
addSignature
parameter, so
the returned
string is the
unchanged

		value of the parameter.
<pre> text = "FOXThanks"; </pre>	<pre> temp = "ThanksF st4.addSignature(text); OX" </pre>	The signature occurs at the beginning of the addSignature parameter, so the returned string is the value of the original parameter with the signature removed from the beginning and appended to the end of the parameter.
<pre> text = "G- LOPEZHello"; </pre>	<pre> temp = "HelloG- st3.addSignature(text); LOPEZ" </pre>	The signature occurs at the beginning of the addSignature parameter, so the returned string is the value of the original parameter with the signature removed from

the beginning
and appended
to the end of
the parameter.

Write the complete SignedText class. Your implementation must meet all specifications and conform to the examples in the table.

Part B · 中文思路与考点

- 状态: private String firstName; private String lastName;
- **getSignature()**: 判 firstName.length() == 0, 分别返回 lastName 与 firstName.substring(0,1) + "-" + lastName。
- **addSignature(String text)**:
 - 用 text.endsWith(signature) 判末尾: 直接返回 text;
 - 用 text.startsWith(signature) 判开头: 返回 text.substring(signature.length()) + signature;
 - 否则返回 text + signature。

考点: Topic 3.5 方法 + Topic 1.15 / Topic 2.10 String 方法 (substring/startsWith/endsWith)。

Part C · 参考解法 (Java)

```
public class SignedText
{
    private String firstName;
    private String lastName;

    public SignedText(String firstName, String lastName)
    {
        this.firstName = firstName;
        this.lastName = lastName;
    }

    public String getSignature()
    {
        if (firstName.length() == 0)
```

```

    {
        return lastName;
    }
    return firstName.substring(0, 1) + "-" + lastName;
}

public String addSignature(String text)
{
    String sig = getSignature();
    if (text.endsWith(sig))
    {
        return text;
    }
    if (text.startsWith(sig))
    {
        return text.substring(sig.length()) + sig;
    }
    return text + sig;
}
}

```

Part D · 评分要点

- 构造器使用 `this.firstName` / `this.lastName` 接收参数；
- `getSignature()` 用 `firstName.length() == 0` 分支返回 `lastName` 或 `firstName.substring(0,1) + "-" + lastName`；
- `addSignature` 三支： `endsWith` 不动 / `startsWith` 移动 / 否则追加；
- 三个字符串操作都正确使用 `startsWith` / `endsWith` / `substring`。

六、应试技巧清单

MCQ 限时训练策略（建议节奏）

- 每题 ≤ 75 秒：FRQ 占用约 35 分钟/道的总时间下，MCQ 需保持 75 秒/题的稳定节奏。
- 先做“明确送分”：看到 `static / private / public / this` 即可直接判断的题，先做——这能在 5-10 秒内拿到分。
- “代码追踪”题放最后：涉及复杂方法调用的题耗时较多（特别涉及 `String` 拼接和 `static` 共享变量变化），可以先标记留待最后 5 分钟单独冲刺。

FRQ 写作策略

- 先画类图：在草稿纸上画出 UML 类图（属性 + 方法签名 + 构造器），再开始写代码——这能减少漏字段。
- 先实现构造器：构造器是核心，先写 `this.field = param;` 模式把所有字段都初始化。
- 再实现行为方法：行为方法通常调用构造器已初始化的字段，逻辑简单。
- 最后处理返回字符串：用 `+` 拼接即可，注意 `""` 与 `null` 的边界。

代码追踪（Tracing）口诀

- 三步法：① 标出每个变量初始值；② 按方法调用顺序一行一行执行；③ 输出/返回值核对。
- **static 单独追踪**：static 字段所有对象共享，只跟踪一份。
- 方法参数 = 局部变量：进入方法后第一件事是把参数抄到草稿纸左边。

易错点自检清单

- [] 实例变量是否全部 `private`？构造器是否 `public`？
 - [] 方法签名里的返回类型与 `return` 值是否兼容？
 - [] `void` 方法体内有没有出现 `return value;`？
 - [] `String` 修改是赋值给原变量还是返回了新对象？
 - [] `static` 方法内有没有访问非 `static` 字段？
 - [] 构造器有没有写返回值类型（`public void ClassName(...)` 不是构造器）？
 - [] `this(...)` 是否放在了构造器第一行？
 - [] `final` 字段是否在每个构造器内都被赋值？
-

七、章节总结

本章 Unit 3 Class Creation 是 APCS A 课程中最考验"代码工程能力"的章节：从 Unit 1 / Unit 2 调用既有类（如 String、ArrayList、Math）到本章亲手设计一个类，是 Java OOP 入门的关键一步。本章考察的核心能力包括：

- 24. 根据需求文本设计类：识别属性（private 实例变量）、行为（public 方法）、构造器（public，参数对应属性）。
- 25. 正确使用访问修饰符：实例变量总是 private，方法与构造器按"是否需要外部调用"决定 public/private。
- 26. 掌握 static 与实例变量的区别：所有对象共享 vs 每个对象独立。
- 27. 理解引用语义：String 不可变 / StringBuilder 可变；对象引用作为参数时，方法内能修改对象内部状态但不能让引用指向新对象。
- 28. 使用 this 关键字：解决参数与字段同名遮蔽；用 this(...) 调用兄弟构造器。
- 29. 设计跨方法的一致行为：如 Scoreboard 中 recordPlay 改变 active 状态后 getScore 能正确读出新 active。

掌握本章后，下一章 Unit 4 Data Collections 将进一步使用 Array、ArrayList、2D Array 三类容器——这些容器在 FRQ 中常常是 Q3 / Q4，而本章学到的类设计思想同样适用于"容器 + 元素"的设计。

第四章 Data Collections

项	内容
适用考试	AP Computer Science A 国际考 (2025 秋新纲 • 2026 及以后)
课程章节	第四章 Data Collections (对应官方大纲 Unit 4)
考纲来源	College Board 《AP Computer Science A Course and Exam Description, Effective Fall 2025》——Unit 4 Data Collections (教材页 91 - 129)
真题来源	2021 - 2025 年近五年 APCSA 国际考 FRQ 真题 (重点 Q3 ArrayList + Q4 2D Array, 本章精选 5 道)、2025 新版 CED 官方样题、AP Classroom 官方 Progress Check 试题 (含 Unit 4 TopicQuestion MCQ、MCQ Part A/B/C、FRQ Part A/B/C)

摘要

本手册为 AP Computer Science A (APCSA) 国际考备考系列的第四章, 主题为 Data Collections (数组、ArrayList、二维数组与递归), 对应 College Board 2025 秋新版考纲 Unit 4 (适用于 2026 年及以后考试)。本章系统梳理 17 个官方 Topic (4.1 Ethical and Social Issues Around Data Collection / 4.2 Introduction to Using Data Sets / 4.3 Array Creation and Access / 4.4 Array Traversals / 4.5 Implementing Array Algorithms / 4.6 Using Text Files / 4.7 Wrapper Classes / 4.8 ArrayList Methods / 4.9 ArrayList Traversals / 4.10 Implementing ArrayList Algorithms / 4.11 2D Array Creation and Access / 4.12 2D Array Traversals / 4.13 Implementing 2D Array Algorithms / 4.14 Searching Algorithms / 4.15 Sorting Algorithms / 4.16 Recursion / 4.17 Recursive Searching and Sorting), 覆盖 1D 数组、ArrayList、2D 数组三种集合类型与搜索、排序、递归三大算法主题; 按出错频率排序 Top 10 高频易错点 (含数组下标越界、增强 for 误修改、

ArrayList 删时下标错位、2D 数组 length 含义混淆、递归缺 base case、二分搜索前提等）并给出正误对照；精选 12 道 MCQ（来自 AP Classroom 官方 Unit 4 TopicQuestion MCQ 42 题题库）与 5 道 2021 - 2025 国际考 Q3 ArrayList + Q4 2D Array 完整 FRQ 真题（FRQ-1 2021 ArrayResizer / FRQ-2 2022 ReviewAnalysis / FRQ-3 2023 WeatherData / FRQ-4 2024 GridPath / FRQ-5 2025 SumOrSameGame），每题附官方英文原题、参考解法与评分要点。本章是 APCS A 课程的绝对核心——MCQ 权重 30-40%（四大单元最高）、FRQ 第 3、4 题全部出自本章；掌握本章后考生能够独立处理数组/ArrayList/2D 数组上的任何算法题，并在 FRQ 写作中拿到满分。

关键词：AP Computer Science A；2025 新版考纲；1D 数组；ArrayList；2D 数组；文本文件；包装类；搜索；排序；递归；Data Collections FRQ；国际考备考

目 录

[一、本章定位与考情速览](#)

[二、知识点与考点梳理（Topic 4.1 - 4.17）](#)

[A：数据基础与伦理（Topic 4.1 / 4.2）](#)

[B：1D Array 三件套（Topic 4.3 / 4.4 / 4.5）](#)

[C：文本文件 + 包装类（Topic 4.6 / 4.7）](#)

[D：ArrayList 三件套（Topic 4.8 / 4.9 / 4.10）](#)

[E：2D Array 三件套（Topic 4.11 / 4.12 / 4.13）](#)

[F：搜索、排序与递归（Topic 4.14 / 4.15 / 4.16 / 4.17）](#)

[三、本章重难点与易错点清单](#)

[四、MCQ 真题精选与详解（12 题）](#)

[五、FRQ 真题精选与解析（5 题完整题面，含官方表格）](#)

[六、应试技巧清单](#)

[七、章节总结](#)

一、本章定位与考情速览

1.1 新大纲四大单元与考试权重

Unit 4 Data Collections 是 APCS A 课程的**绝对核心章节**：它整合 Unit 1（对象与方法）、Unit 2（控制结构）、Unit 3（类设计），引入 **1D 数组**、**ArrayList**、**2D 数组** 三种集合结构，并在此基础上讲解**搜索**、**排序**、**递归**三大算法主题。本章不仅是 MCQ 权重最高（30 - 40%）的单元，更是 FRQ 第 3、4 题的出题源头。

单元（本课程章节）	MCQ 权重	建议课时
Unit 1: Using Objects and Methods（第一章）	15 - 25%	约 32 - 34 课时
Unit 2: Selection and Iteration（第二章）	25 - 35%	约 29 - 31 课时
Unit 3: Class Creation（第三章）	10 - 18%	约 20 - 22 课时
Unit 4: Data Collections（第四章）	30 - 40%	约 50 - 52 课时

Unit 4 的 MCQ 权重（30 - 40%）是四大单元中**最高的**，课时也最长（约 50 - 52 课时）。本章覆盖 17 个 Topic、3 套 Progress Check（共约 60 道 MCQ + 6 道 FRQ），是备考的重中之重。

1.2 考试结构

部分	题量与时长	占比
Section I: 选择题（MCQ）	42 题 / 90 分钟	55%
Section II: 自由作答题（FRQ）	4 题 / 90 分钟	45%

格式说明

2026 年起 Unit 4 对应**两道** FRQ: **FRQ 第 3 题 = Data Analysis with ArrayList**（写 ArrayList 上的方法）、**FRQ 第 4 题 = 2D Array**（写 2D 数组上的方法）。两题各约 9 分，合计 18 分，占 FRQ 总分的 40%——是写作题中分值最重的部分。本章精选 5 道 FRQ 真题（2021 Q4 ArrayResizer、2022 Q3 ReviewAnalysis、2023 Q3 WeatherData、2024 Q4 GridPath、2025 Q4 SumOrSameGame），覆盖 1D Array、ArrayList、2D Array 三大主题。

1.3 本章与 FRQ 的关联（备考重点提示）

- **国际考 Q3 = Data Analysis with ArrayList**（2021 ClubMembers、2022 ReviewAnalysis、2023 WeatherData、2024 WordChecker、2025 Tournament Match，五年皆如此）。本题要求考生在 ArrayList 上编写方法：插入元素、删除元素、按条件遍历、计算统计量。**ArrayList 的增删与遍历中维护索引**是本章最高频、最容易失分的写代码题型。
- **国际考 Q4 = 2D Array**（2021 ArrayResizer、2022 Data、2023 BoxOfCandy、2024 GridPath、2025 SumOrSameGame，五年皆如此）。本题在 2D 数组上做行/列遍历、筛选、查找或路径计算，**必须熟练掌握 row-major 嵌套 for 循环**。
- **MCQ 高频考点**：数组下标访问（0-based）、数组长度（.length）、ArrayList 方法（add / get / set / remove / size）、增强 for 循环、2D 数组嵌套遍历、文本文件读入（Scanner）、包装类（Integer / Double）、递归基础、搜索与排序算法分析。

1.4 本章学习目标（对应官方 17 个 Topic）

新 CED Unit 4 共 17 个 Topic，按"先伦理、再 1D Array、再文本文件、再 ArrayList、再 2D Array、再算法"的顺序展开：

30. **4.1 Ethical and Social Issues Around Data Collection**（数据收集的伦理与社会影响）：理解个人数据收集的隐私风险、算法偏见、数据质量。
31. **4.2 Introduction to Using Data Sets**（数据集使用入门）：会用自然语言或图示描述日常数据集合。
32. **4.3 Array Creation and Access**（一维数组的创建与访问）：声明、初始化、通过下标访问元素。
33. **4.4 Array Traversals**（数组遍历）：用 for 循环（含增强 for）按顺序访问每个元素。
34. **4.5 Implementing Array Algorithms**（数组算法实现）：在数组上实现求和、计数、最值等基础算法。
35. **4.6 Using Text Files**（文本文件读入）：用 Scanner 读文本文件，注意 hasNext / nextInt / nextDouble / nextLine 的搭配。
36. **4.7 Wrapper Classes**（包装类）：只考查 Integer 与 Double 两个包装类——Integer.parseInt / Integer.MAX_VALUE / Integer.MIN_VALUE、Double.parseDouble / Double.MAX_VALUE / Double.MIN_VALUE，以及自动装箱 / 拆箱。
37. **4.8 ArrayList Methods**（ArrayList 方法）：add / get / set / remove / size / contains / indexOf。
38. **4.9 ArrayList Traversals**（ArrayList 遍历）：用 for / 增强 for / 倒序遍历。

39. **4.10 Implementing ArrayList Algorithms (ArrayList 算法实现)**：在 ArrayList 上实现增删改查与统计。
40. **4.11 2D Array Creation and Access (二维数组的创建与访问)**：声明 2D 数组、通过 `arr[row][col]` 访问元素。
41. **4.12 2D Array Traversals (二维数组遍历)**：row-major 嵌套 for 循环遍历所有元素。
42. **4.13 Implementing 2D Array Algorithms (二维数组算法实现)**：行/列求和、按条件筛选、子矩阵操作。
43. **4.14 Searching Algorithms (搜索算法)**：顺序搜索、二分搜索 (Binary Search)。
44. **4.15 Sorting Algorithms (排序算法)**：选择排序、插入排序、归并排序 (熟练掌握代码模板；复杂度分析仅在 MCQ 中考查，不考 FRQ 实现)。
45. **4.16 Recursion (递归基础)**：理解 base case + recursive call，写简单的递归方法 (如阶乘、求和)。
46. **4.17 Recursive Searching and Sorting (递归搜索与排序)**：递归实现二分搜索与归并排序。
-

二、知识点与考点梳理 (Topic 4.1–4.17)

板块 A：数据基础与伦理 (Topic 4.1 / 4.2)

Topic 4.1 数据收集的伦理与社会影响

考点：MCQ 高频考「隐私保护」「算法偏见」「数据质量」三个概念。

概念

考试表述

隐私 (Privacy)

当用户在 App、网站中输入个人信息 (姓名 / 位置 / 购物记录) 时，这些数据可能被收集、存储、甚至售卖；程序员应尝试保护用户隐私。

算法偏见 (Algorithmic Bias)

系统性、重复性的程序错误，导致对特定用户群体不公平的结果。例如训练数据中只包含某一族群的照片，人脸识别对其他族群准确率显著降低。

数据质量 (Data Quality)

数据集可能不完整 (缺失值)、

不准确（错误值）、有偏差（采样不均）；使用前应评估数据质量。

典型 MCQ 题干：「A programmer is designing an app that recommends restaurants to users based on past dining history. Which of the following is the most important ethical consideration?」

Topic 4.2 数据集使用入门

数据集（Data Set）：一组相关数据的集合。考试常考：

- 用文字描述从数据集提取什么信息（例如「从 100 万条购物记录中找出最常被一起购买的商品对」）；
- 用图示（流程图 / 思维导图）描述分析流程。

// 数据集的典型 API：使用 Java 集合表示

```
int[] temperatures = {72, 75, 68, 80, 79}; // 1D 数组
```

```
ArrayList<Double> scores = new ArrayList<Double>(); // ArrayList
```

```
String[][] spreadsheet = new String[10][5]; // 2D 数组
```

板块 B：1D Array 三件套（Topic 4.3 / 4.4 / 4.5）

Topic 4.3 一维数组的创建与访问

两种初始化语法（考试重点）：

// 语法 A：先 new，再赋值

```
int[] nums = new int[5];
```

```
nums[0] = 10; nums[1] = 20; nums[2] = 30; nums[3] = 40; nums[4] = 50;
```

// 语法 B：初始化列表（仅声明时可用）

```
int[] nums = {10, 20, 30, 40, 50};
```

```
String[] names = {"Alice", "Bob", "Carol"};
```

```
double[] prices = new double[]{9.99, 19.99, 29.99}; // new double[] 也可以省略
```

核心规则：

- 数组下标从 0 开始，到 length - 1 结束；
- 数组长度不可变（一旦 new 出，大小就锁死；试图 nums.length++ 会编译报错）；
- 访问越界下标会抛 ArrayIndexOutOfBoundsException（运行期错误，编译不会报错）。

数组作为方法参数 / 返回值：

```
public static int sum(int[] arr) { // 接收数组引用
    int total = 0;
    for (int x : arr) {
        total += x;
    }
    return total;
}

public static int[] reverse(int[] arr) { // 返回新数组
    int[] result = new int[arr.length];
    for (int i = 0; i < arr.length; i++) {
        result[i] = arr[arr.length - 1 - i];
    }
    return result;
}
```

Topic 4.4 数组遍历

两种遍历语法（考试必会）：

```
int[] nums = {10, 20, 30, 40, 50};

// 语法 A：传统 for（需要下标时用）
for (int i = 0; i < nums.length; i++) {
    System.out.println(nums[i]);
}

// 语法 B：增强 for（只读访问时用，更简洁）
```

```
for (int n : nums) {
    System.out.println(n);
}
```

遍历模式变种（MCQ 高频）：

- 正向遍历：for (int i = 0; i < nums.length; i++)
- 反向遍历：for (int i = nums.length - 1; i >= 0; i--)
- 每隔一个：for (int i = 0; i < nums.length; i += 2)
- 跳过首尾：for (int i = 1; i < nums.length - 1; i++)

遍历中的修改陷阱：增强 for 不能用于修改数组元素（修改的是循环变量，不影响原数组）；若要修改必须用传统 for + 下标。

```
int[] nums = {1, 2, 3};
for (int n : nums) {
    n = n * 2; // ✗ 无效，n 是循环副本
}
for (int i = 0; i < nums.length; i++) {
    nums[i] *= 2; // ✓ 正确
}
```

Topic 4.5 数组算法实现

五大基础算法（FRQ 高频）：

```
// 1. 求和 / 平均
public static int sum(int[] arr) {
    int total = 0;
    for (int x : arr) {
        total += x;
    }
    return total;
}

public static double average(int[] arr) {
    return (double) sum(arr) / arr.length;
}
```

```
}
```

```
// 2. 最大 / 最小
```

```
public static int max(int[] arr) {  
    int best = arr[0];  
    for (int i = 1; i < arr.length; i++) {  
        if (arr[i] > best) best = arr[i];  
    }  
    return best;  
}
```

```
// 3. 计数（满足条件的元素个数）
```

```
public static int countAbove(int[] arr, int threshold) {  
    int count = 0;  
    for (int x : arr) {  
        if (x > threshold) count++;  
    }  
    return count;  
}
```

```
// 4. 查找（返回下标或 -1）
```

```
public static int find(int[] arr, int target) {  
    for (int i = 0; i < arr.length; i++) {  
        if (arr[i] == target) return i;  
    }  
    return -1;  
}
```

```
// 5. 复制（部分 / 全部）
```

```
public static int[] copy(int[] arr) {  
    int[] result = new int[arr.length];  
    for (int i = 0; i < arr.length; i++) {
```

```

    result[i] = arr[i];
}
return result;
}

```

考试常见变种：

- 「统计数组中相邻两个相等的对数」：for (int i = 0; i < arr.length - 1; i++) { if (arr[i] == arr[i+1]) count++; }
- 「数组是否已排序（升序）」：检测每一对 arr[i] > arr[i+1];
- 「找第二大的元素」：先找 max，再遍历找 < max 的最大。

板块 C：文本文件 + 包装类 (Topic 4.6 / 4.7)

Topic 4.6 文本文件读入

考试必备模板（MCQ 必会、FRQ 不直接考但工程中常用）：

```

import java.util.Scanner;

import java.io.File;
import java.io.FileNotFoundException;

public static void readFile(String filename) throws FileNotFoundException {
    Scanner sc = new Scanner(new File(filename));
    while (sc.hasNext()) {
        int n = sc.nextInt();
        System.out.println(n);
    }
    sc.close();
}

```

// 简单用法：通常 AP 题目把异常处理隐藏，直接给一个已打开的 Scanner

```

public static double average(Scanner sc) {
    double sum = 0;
    int count = 0;

```

```

while (sc.hasNext()) {
    sum += sc.nextDouble();
    count++;
}
return count == 0 ? 0 : sum / count;
}

```

关键区分（MCQ 高频陷阱）：

方法	用途	注意
hasNext()	还有没有下一个 token（任意类型）	用于 while 循环条件
next()	读取下一个 token（按空白切分）	自动跳过分隔符
nextInt()	读取下一个 int	同上
nextLine()	读取整行（直到换行）	会读走上一行的换行符，紧跟 nextInt() 后会读到空行

考试常见陷阱：

```

// ✗ 错误用法：nextInt 后立刻 nextLine
Scanner sc = new Scanner("42\nhello world");
int n = sc.nextInt(); // 读 42，留下 \n
String line = sc.nextLine(); // ✗ 读到的是 ""（空字符串），因为只读到了 \n

// ☑ 正确：在 nextInt 后多读一行换行符
sc.nextLine(); // 吃掉 \n
line = sc.nextLine(); // 读到 "hello world"

```

Topic 4.7 包装类 (Wrapper Classes)

核心概念：基本类型没有方法，要调用方法必须包装为对象。APCSA 只要求掌握 Integer 与 Double 两个包装类。

基本类型	包装类	考查的方法 / 常量
int	Integer	Integer.parseInt(String) / Integer.MAX_VALUE / Integer.MIN_VALUE
double	Double	Double.parseDouble(String) / Double.MAX_VALUE / Double.MIN_VALUE

自动装箱 / 拆箱 (MCQ 高频)：

```
Integer x = 42;    // 自动装箱: Integer x = Integer.valueOf(42);  
int n = x;         // 自动拆箱: Integer 自动转为 int
```

```
ArrayList<Integer> nums = new ArrayList<Integer>();  
nums.add(10);       // 10 自动装箱为 Integer(10)  
int first = nums.get(0); // 自动拆箱
```

关键考点 (只考查以下内容)：

- ArrayList<int> 不允许 (基本类型不能作为泛型实参)，必须用 ArrayList<Integer>;
- 字符串 → 数字: Integer.parseInt("42") → 42、Double.parseDouble("3.14") → 3.14;
- 常量: Integer.MAX_VALUE / Integer.MIN_VALUE、Double.MAX_VALUE / Double.MIN_VALUE。

// MCQ 经典陷阱

```
double d = 10.0;  
Double D = 20.0;  
System.out.println(d + D); // 输出 30.0 (自动拆箱为 double + double)  
String s = "" + d + D;    // 输出 "10.020.0" (拼接为字符串)
```

板块 D: ArrayList 三件套 (Topic 4.8 / 4.9 / 4.10)

Topic 4.8 ArrayList 方法

完整 API 速查 (考试必背):

方法	签名	说明
构造器	<code>new ArrayList<E>()</code>	空列表
构造器	<code>new ArrayList<E>(int initialCapacity)</code>	指定初始容量 (考试很少考)
add	<code>add(E e)</code>	末尾追加, 返回 <code>boolean</code>
add	<code>add(int index, E e)</code>	在 <code>index</code> 处插入, 原元素后移
get	<code>get(int index)</code>	返回 <code>index</code> 处的元素
set	<code>set(int index, E e)</code>	替换 <code>index</code> 处的元素, 返回 <code>旧值</code>
remove	<code>remove(int index)</code>	删除并返回 <code>index</code> 处的元素
remove	<code>remove(Object o)</code>	删除 第一个 等于 <code>o</code> 的元素, 返回 <code>boolean</code>
size	<code>size()</code>	当前元素个数
isEmpty	<code>isEmpty()</code>	是否为空 (<code>size == 0</code>)
contains	<code>contains(Object o)</code>	是否包含 <code>o</code> (用 <code>equals</code> 比较)
indexOf	<code>indexOf(Object o)</code>	第一个等于 <code>o</code> 的下标, 没有返回 <code>-1</code>
clear	<code>clear()</code>	清空所有元素

关键陷阱（MCQ 高频失分点）：

```
ArrayList<String> words = new ArrayList<String>();
words.add("jump"); // [jump]
words.add("right"); // [jump, right]
words.add("plan"); // [jump, right, plan]

words.add(2, "laugh"); // [jump, right, laugh, plan] 在 index=2 插入
words.add(3, "laugh"); // [jump, right, plan, laugh] 在 index=3 插入

words.set(2, "laugh"); // [jump, right, laugh]    set 是替换！
words.set(3, "laugh"); // IndexOutOfBoundsException set(3) 不存在

words.remove(2); // [jump, right]                remove(2) 删除并返回 "plan"
words.remove("right"); // [jump]                  remove(Object) 删除第一个匹配
```

「**remove(int)** 与 **remove(Object)** 的歧义」陷阱：

```
ArrayList<Integer> nums = new ArrayList<Integer>();
nums.add(10); nums.add(20); nums.add(30); // [10, 20, 30]
nums.remove(20); // ✗ 编译错！remove(int) 需要下标 0..size-1，但 20 不是
下标
nums.remove(new Integer(20)); // ☑ 删除 20 这个元素（推荐写法：
nums.remove(2);)
```

Topic 4.9 ArrayList 遍历

三种遍历模式（考试重点）：

```
ArrayList<String> words = new ArrayList<String>(Arrays.asList("a", "b",
"c"));

// 1. 传统 for（需要下标时用）
for (int i = 0; i < words.size(); i++) {
```

```
    System.out.println(i + ": " + words.get(i));  
}
```

// 2. 增强 for（只读访问时用，最简洁）

```
for (String w : words) {  
    System.out.println(w);  
}
```

// 3. 倒序遍历（删除元素时常用，避免下标错位）

```
for (int i = words.size() - 1; i >= 0; i--) {  
    System.out.println(words.get(i));  
}
```

遍历中删除的下标错位问题（FRQ 最高频失分点）：

// ✘ 错误：正向遍历 + remove，下标错位

```
ArrayList<Integer> nums = new ArrayList<Integer>(Arrays.asList(1, 2, 3, 4, 5));  
for (int i = 0; i < nums.size(); i++) {  
    if (nums.get(i) % 2 == 0) nums.remove(i); // 删除 2 后，原 i=2 变成 4，被跳过！  
}
```

// 结果：[1, 3, 5] ← 错！应该是 [1, 3, 5] 但实际：[1, 3, 4, 5] ← 4 没被删

// ☑ 正确方案 A：倒序遍历

```
for (int i = nums.size() - 1; i >= 0; i--) {  
    if (nums.get(i) % 2 == 0) nums.remove(i);  
}
```

// 结果：[1, 3, 5] ☑

// ☑ 正确方案 B：正向遍历 + i--

```
for (int i = 0; i < nums.size(); i++) {  
    if (nums.get(i) % 2 == 0) {
```

```

    nums.remove(i);
    i--; // 重要：删完后回退一个下标
}
}
// 结果: [1, 3, 5] ☒

```

Topic 4.10 ArrayList 算法实现

FRQ 最常考的五类算法:

// 1. 求最大值及其下标

```

public static int findMaxIndex(ArrayList<Integer> list) {
    int maxIdx = 0;
    for (int i = 1; i < list.size(); i++) {
        if (list.get(i) > list.get(maxIdx)) maxIdx = i;
    }
    return maxIdx;
}

```

// 2. 移除所有满足条件的元素（返回新列表，保留原列表）

```

public static ArrayList<Integer> filterAbove(ArrayList<Integer> list, int
threshold) {
    ArrayList<Integer> result = new ArrayList<Integer>();
    for (int n : list) {
        if (n > threshold) result.add(n);
    }
    return result;
}

```

// 3. 找最长连续段（类似 2023 WeatherData.longestHeatWave）

```

public static int longestStreak(ArrayList<Double> data, double threshold) {
    int best = 0, cur = 0;
    for (double d : data) {
        if (d > threshold) { cur++; best = Math.max(best, cur); }
    }
}

```

```

        else cur = 0;
    }
    return best;
}

```

// 4. 累加前缀和

```

public static ArrayList<Integer> prefixSum(ArrayList<Integer> list) {
    ArrayList<Integer> result = new ArrayList<Integer>();
    int sum = 0;
    for (int n : list) { sum += n; result.add(sum); }
    return result;
}

```

// 5. 字符串拼接（如 2022 ReviewAnalysis.collectComments）

```

public static ArrayList<String> commentsWithExclamation(Review[] reviews) {
    ArrayList<String> result = new ArrayList<String>();
    for (int i = 0; i < reviews.length; i++) {
        String c = reviews[i].getComment();
        if (c.contains("!")) {
            String formatted = i + "-" + c;
            if (!c.endsWith(".") && !c.endsWith("!"))
                formatted += ".";
            result.add(formatted);
        }
    }
    return result;
}

```

板块 E: 2D Array 三件套 (Topic 4.11 / 4.12 / 4.13)

Topic 4.11 二维数组的创建与访问

三种初始化方式 (考试重点):

// 语法 A: 声明 + new + 逐元素赋值

```
int[][] matrix = new int[3][4]; // 3 行 4 列, 默认全 0
matrix[0][0] = 1; matrix[0][1] = 2;
```

// 语法 B: 初始化列表 (仅声明时可用)

```
int[][] matrix = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};
```

// 语法 C: 锯齿数组 (每行长度不同)

```
int[][] jagged = new int[3][];
jagged[0] = new int[]{1, 2};
jagged[1] = new int[]{3, 4, 5, 6};
jagged[2] = new int[]{7};
```

关键区分 (MCQ 高频):

```
int[][] matrix = {{1,2,3}, {4,5,6}, {7,8,9}};
matrix.length    // 3 (行数)
matrix[0].length // 3 (第 0 行的列数)
matrix[2][1]     // 8 (第 2 行第 1 列)
```

考试常考的越界:

```
int[][] matrix = {{1,2,3}, {4,5,6}}; // 2 行 3 列
matrix[2][0] // ✖ ArrayIndexOutOfBoundsException (行越界)
matrix[0][3] // ✖ ArrayIndexOutOfBoundsException (列越界)
```

matrix[1][2] // ☒ 6

Topic 4.12 二维数组遍历 (Row-Major 嵌套 for)

标准模板 (FRQ 必会) :

```
int[][] matrix = {{1,2,3}, {4,5,6}, {7,8,9}};
```

// Row-Major 遍历 (最常用)

```
for (int r = 0; r < matrix.length; r++) {    // 行
    for (int c = 0; c < matrix[r].length; c++) {    // 列
        System.out.print(matrix[r][c] + " ");
    }
    System.out.println();
}
```

// 输出:

// 1 2 3

// 4 5 6

// 7 8 9

// 增强 for

```
for (int[] row : matrix) {
    for (int v : row) {
        System.out.print(v + " ");
    }
    System.out.println();
}
```

遍历变种 (MCQ + FRQ 高频) :

// Column-Major (先按列访问)

```
for (int c = 0; c < matrix[0].length; c++) {
    for (int r = 0; r < matrix.length; r++) {
        System.out.print(matrix[r][c] + " ");
    }
}
```

```

    }
    System.out.println();
}

// 对角线（左上到右下）
for (int i = 0; i < matrix.length; i++) {
    System.out.println(matrix[i][i]);
}

// 之字形（zig-zag）
for (int r = 0; r < matrix.length; r++) {
    if (r % 2 == 0) {
        for (int c = 0; c < matrix[r].length; c++) { /* 正向 */ }
    } else {
        for (int c = matrix[r].length - 1; c >= 0; c--) { /* 反向 */ }
    }
}

```

Topic 4.13 二维数组算法实现

FRQ 五大基础算法：

```

// 1. 矩阵所有元素求和
public static int sumAll(int[][] m) {
    int total = 0;
    for (int[] row : m) {
        for (int v : row) {
            total += v;
        }
    }
    return total;
}

// 2. 按行求和，返回一维数组

```

```

public static int[] rowSums(int[][] m) {
    int[] sums = new int[m.length];
    for (int r = 0; r < m.length; r++) {
        for (int c = 0; c < m[r].length; c++) {
            sums[r] += m[r][c];
        }
    }
    return sums;
}

```

// 3. 按列求和（注意列数 = m[0].length）

```

public static int[] colSums(int[][] m) {
    int[] sums = new int[m[0].length];
    for (int r = 0; r < m.length; r++) {
        for (int c = 0; c < m[r].length; c++) {
            sums[c] += m[r][c];
        }
    }
    return sums;
}

```

// 4. 找最大元素及其位置（2021 ArrayResizer、2024 GridPath 都考）

```

public static int findMax(int[][] m) {
    int best = m[0][0];
    for (int[] row : m) {
        for (int v : row) {
            if (v > best) best = v;
        }
    }
    return best;
}

```

// 5. 矩阵转置（行变列）

```

public static int[][] transpose(int[][] m) {
    int rows = m.length, cols = m[0].length;
    int[][] t = new int[cols][rows];
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            t[c][r] = m[r][c];
        }
    }
    return t;
}

```

板块 F：搜索、排序与递归（Topic 4.14 / 4.15 / 4.16 / 4.17）

Topic 4.14 搜索算法

顺序搜索 vs 二分搜索（MCQ 必考）：

// 顺序搜索（Sequential Search）—— $O(n)$ ，适用于任何数组

```

public static int seqSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}

```

// 二分搜索（Binary Search）—— $O(\log n)$ ，仅适用于**已排序**数组

```

public static int binSearch(int[] sorted, int target) {
    int lo = 0, hi = sorted.length - 1;
    while (lo <= hi) {
        int mid = (lo + hi) / 2;
        if (sorted[mid] == target) return mid;
        else if (sorted[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
}

```

```

    }
    return -1;
}

```

MCQ 必会:

- 二分搜索的前提: **数组必须已排序** (升序);
- 二分搜索每次比较后, 搜索范围减半 $\rightarrow O(\log n)$;
- 顺序搜索遍历整个数组 $\rightarrow O(n)$;
- $n=1000$ 时, 顺序搜索最多 1000 次比较; 二分搜索最多约 10 次比较。

Topic 4.15 排序算法

三种排序的代码模板与复杂度 (只考代码模板与 MCQ, 不考 FRQ):

// 1. 选择排序 Selection Sort —— 时间 $O(n^2)$ 、空间 $O(1)$; 每次从未排序部分找最小值, 放到已排序末尾

```

public static void selectionSort(int[] arr) {
    for (int i = 0; i < arr.length - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[j] < arr[minIdx]) minIdx = j;
        }
        int tmp = arr[i]; arr[i] = arr[minIdx]; arr[minIdx] = tmp;
    }
}

```

// 2. 插入排序 Insertion Sort —— 平均/最差 $O(n^2)$ 、最优 $O(n)$, 空间 $O(1)$; 每次将当前元素插入到已排序部分的正确位置

```

public static void insertionSort(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
    }
}

```

```

    }
    arr[j + 1] = key;
}
}

```

// 3. 归并排序 Merge Sort —— 分治法（divide and conquer），时间 $O(n \log n)$ 、空间 $O(n)$

```

public static void mergeSort(int[] arr) {
    if (arr.length < 2) return;
    int mid = arr.length / 2;
    int[] left = Arrays.copyOfRange(arr, 0, mid);
    int[] right = Arrays.copyOfRange(arr, mid, arr.length);
    mergeSort(left);
    mergeSort(right);
    merge(arr, left, right);
}

private static void merge(int[] arr, int[] left, int[] right) {
    int i = 0, j = 0, k = 0;
    while (i < left.length && j < right.length) {
        if (left[i] <= right[j]) arr[k++] = left[i++];
        else arr[k++] = right[j++];
    }
    while (i < left.length) arr[k++] = left[i++];
    while (j < right.length) arr[k++] = right[j++];
}

```

Topic 4.16 递归基础

递归三要素（MCQ + FRQ 高频）：

1. **Base case:** 终止条件，避免无限递归；
2. **Recursive call:** 调用自身，参数必须向 base case 推进；
3. **进度:** 每次调用都必须让问题规模变小。

经典例子（必须能默写）：

```

// 阶乘
public static int factorial(int n) {
    if (n <= 1) return 1;    // base case
    return n * factorial(n - 1); // recursive call, n 递减
}

// 数字各位之和
public static int digitSum(int n) {
    if (n < 10) return n;    // base case
    return n % 10 + digitSum(n / 10); // 末位 + 递归处理剩余部分
}

// 数组求和（递归版）
public static int sumArray(int[] arr, int i) {
    if (i >= arr.length) return 0;    // base case
    return arr[i] + sumArray(arr, i + 1); // 当前元素 + 递归处理剩余
}

```

MCQ 常见陷阱:

```

// ✗ 缺 base case: 无限递归 → StackOverflowError
public static int bad(int n) {
    return bad(n - 1); // 永远不停止
}

// ✗ base case 永远不进: n 永远不减
public static int worse(int n) {
    if (n == 0) return 1;
    return worse(n + 1); // n 在增大, 永远到不了 0
}

// ☑ 正确: n 每次减 1, 最终到 base case

```

```
public static int good(int n) {
    if (n == 0) return 1;
    return n * good(n - 1); // factorial
}
```

Topic 4.17 递归搜索与排序

递归版二分搜索：

```
public static int recBinSearch(int[] sorted, int target, int lo, int hi) {
    if (lo > hi) return -1; // base case: 未找到
    int mid = (lo + hi) / 2;
    if (sorted[mid] == target) return mid;
    else if (sorted[mid] < target)
        return recBinSearch(sorted, target, mid + 1, hi); // 右半边
    else
        return recBinSearch(sorted, target, lo, mid - 1); // 左半边
}
// 调用: recBinSearch(arr, target, 0, arr.length - 1)
```

递归版归并排序：见 Topic 4.15 板块。

考试重点：递归仅在 **MCQ** 中考查（CED 明确：「Recursion is assessed only through the multiple-choice section of the exam」），FRQ 不考递归实现，但偶尔会考调用已给的递归方法计算结果。

三、本章重难点与易错点清单

按考试出错频率排序 TOP 10 高频易错点，每条附正误对照。

易错点 #1：数组下标越界 vs 长度访问错位

易错写法：

```
int[] nums = new int[5]; // 有效下标 0..4
for (int i = 0; i <= nums.length; i++) { // ✗ 应该 < 不是 <=
```

```
System.out.println(nums[i]);    // i=5 时越界
}
```

正确写法: `for (int i = 0; i < nums.length; i++)` —— 严格小于, 不要写 `<=`。

易错点 #2: 增强 **for** 误以为能修改元素

易错写法:

```
int[] nums = {1, 2, 3};
for (int n : nums) {
    n *= 2; // ✗ n 是循环变量副本, 不影响 nums
}
System.out.println(Arrays.toString(nums)); // 输出 [1, 2, 3]
```

正确写法: 必须用传统 `for + 下标`。

易错点 #3: **ArrayList** 遍历中删除的下标错位

易错写法:

```
for (int i = 0; i < list.size(); i++) {
    if (list.get(i) % 2 == 0) list.remove(i); // ✗ 删除后下标错位
}
```

正确写法: 倒序遍历 / `i--` 二选一 (详见 Topic 4.9)。

易错点 #4: **ArrayList** 与数组互相转换时的强转

易错写法:

```
ArrayList<Integer> list = new ArrayList<Integer>(Arrays.asList(1, 2, 3));
int[] arr = (int[]) list.toArray(); // ✗ 转型错误: toArray 返回 Object[]
```

正确写法:

```
Integer[] arr = list.toArray(new Integer[0]); // 转 Integer[]
int[] arr = list.stream().mapToInt(Integer::intValue).toArray(); // 转 int[] (需要
```

```
import java.util.stream.*)
```

易错点 #5: 2D 数组的 `length` 含义混淆

易错写法:

```
int[][] m = {{1,2,3}, {4,5,6}};
for (int c = 0; c < m.length; c++) {    // ✗ m.length 是行数 2
    for (int r = 0; r < m[0].length; r++) { // ✗ 列循环但变量名是 r
        ...
    }
}
```

正确写法:

```
for (int r = 0; r < m.length; r++) { // 行: m.length
    for (int c = 0; c < m[r].length; c++) { // 列: m[r].length (每行长度可能不同)
        process(m[r][c]);
    }
}
```

易错点 #6: `Scanner.nextInt` 后立刻 `nextLine` 读到空字符串

易错写法:

```
Scanner sc = new Scanner("42\nhello");
int n = sc.nextInt();    // 读 42, 遗留 \n
String s = sc.nextLine(); // ✗ 读到的是 "" (空字符串), 不是 "hello"
```

正确写法: 中间加 `sc.nextLine()` 吃掉换行符。

易错点 #7: `ArrayList` 误用 `==` 比较

易错写法:

```
ArrayList<Integer> list = new ArrayList<Integer>(Arrays.asList(1, 2, 3));  
if (list.get(0) == 1) // ✗ Integer 自动拆箱为 int 后 == 才是值比较  
...  
// 但如果是两个 Integer 对象比较:  
Integer a = list.get(0);  
Integer b = list.get(0);  
if (a == b) // ✗ 这是引用比较, a == b 可能是 false (虽然 -128..127 会缓存)
```

正确写法: 基本类型用 `==`; 对象用 `.equals()`; `Integer` 拆箱后用 `==` 没问题, 但两 `Integer` 对象直接 `==` 是错的。

易错点 #8: 递归缺 base case 导致栈溢出

易错写法:

```
public static int count(int n) {  
    return count(n - 1) + 1; // ✗ 没 base case, 无限递归 → StackOverflowError  
}
```

正确写法: 必须有 base case (终止条件) + 每次调用参数向 base case 推进。

易错点 #9: 二分搜索忘记前提「数组已排序」

易错写法:

```
int[] arr = {5, 2, 8, 1, 9}; // 未排序  
int idx = binSearch(arr, 8); // ✗ binSearch 假设 arr 已排序升序, 结果未定义
```

正确写法: 二分搜索前先排序 (`Arrays.sort(arr)`), 或在题目已明确给已排序数组时才能直接二分。

易错点 #10: 归并排序的 merge 阶段数组下标错误

易错写法:

```
private static void merge(int[] arr, int[] left, int[] right) {  
    int i = 0, j = 0, k = 0;
```

```
while (i < left.length && j < right.length) {  
    if (left[i] <= right[j]) arr[k] = left[i++]; // ✕ k 没自增  
    else arr[k] = right[j++];  
}  
...  
}
```

正确写法: `arr[k++] = left[i++]`; (k 必须自增, 否则覆盖)。

四、MCQ 真题精选与详解

本章 12 道 MCQ 全部来自 AP Classroom 官方 Unit 4 TopicQuestion MCQ 题库 (42 页官方原题), 覆盖 1D Array、ArrayList、2D Array、包装类四大主题。建议练习节奏: 先限时 30 秒独立选答案, 再看解析。

MCQ-1: 1D 数组初始化与默认值 (Unit 4 TopicQuestion MCQ #1)

Consider the following code segment.

```
int[] arr = new int[5];  
for (int i = 0; i < arr.length; i++) {  
    arr[i] = i * 2;  
}  
System.out.println(arr[3]);
```

What is printed as a result of executing the code segment?

- (A) 0
- (B) 3
- (C) 6
- (D) 8

查看答案与解析

答案: C (6)

解析: `arr[i] = i * 2`, 所以 `arr = {0, 2, 4, 6, 8}`, `arr[3] = 6`。

考点：Topic 4.3 数组创建与下标访问。

MCQ-2: ArrayList 插入与替换的区别 (Unit 4 TopicQuestion MCQ #2)

Consider the following code segment.

```
ArrayList<String> words = new ArrayList<String>();  
words.add("jump");  
words.add("right");  
words.add("plan");  
/* missing statement */
```

Which of the following can replace `/* missing statement */` so that words contains ["jump", "right", "laugh", "plan"] after the code segment is executed?

- (A) words.add(2, "laugh");
- (B) words.add(3, "laugh");
- (C) words.set(2, "laugh");
- (D) words.set(3, "laugh");

查看答案与解析

答案：A

解析：

- 当前 words = ["jump", "right", "plan"], 下标为 0/1/2;
- 目标 ["jump", "right", "laugh", "plan"] 在 index 2 插入 "laugh", 原 "plan" 后移到 index 3;
- A: add(2, "laugh") 在 index 2 插入, 结果 ["jump", "right", "laugh", "plan"] ☒
- B: add(3, "laugh") 在 index 3 插入 (末尾追加), 结果 ["jump", "right", "plan", "laugh"] ✗
- C: set(2, "laugh") 把 index 2 替换为 "laugh", 结果 ["jump", "right", "laugh"] ✗
- D: set(3, "laugh") 越界 (index 3 不存在) → IndexOutOfBoundsException ✗

考点：Topic 4.8 ArrayList add(int, E) vs set(int, E) 的区别。

MCQ-3: 2D 数组初始化语法 (Unit 4 TopicQuestion MCQ #11)

Consider the following code segment, which is intended to declare and initialize the two-dimensional String array things.

```
/* missing code */ = {  
    {"spices", "garlic", "onion", "pepper"},  
    {"clothing", "hat", "scarf", "gloves"},  
    {"plants", "tree", "bush", "flower"},  
    {"vehicles", "car", "boat", "airplane"}  
};
```

Which of the following could replace /* missing code */ so that things is properly declared?

- (A) new String[][] things
- (B) new(String[][]) things
- (C) String[] String[] things
- (D) String[][] things

查看答案与解析

答案: D

解析:

- A: 语法错 (new String[][] 应是类型, 不能与变量名混在一起);
- B: 语法错 (new(...) 不是 Java 语法);
- C: 语法错 (String[] String[] 写法无效);
- D: String[][] things = {{...}, {...}}; 是正确的 2D 数组初始化 ☒

考点: Topic 4.11 2D 数组声明语法。

MCQ-4: ArrayList remove 后下标变化 (Unit 4 TopicQuestion MCQ #10)

Consider the following code segment.

```
ArrayList<Double> dataValues = new ArrayList<Double>();  
dataValues.add(9.84);  
dataValues.add(8.93);
```

```
dataValues.add(7.65);  
dataValues.add(6.24);  
dataValues.remove(2);  
dataValues.set(2, 7.63);
```

Which of the following represents the contents of dataValues after executing the code segment?

- (A) [9.84, 7.63, 6.24]
- (B) [9.84, 7.63, 7.65, 6.24]
- (C) [9.84, 8.93, 7.63]
- (D) [9.84, 8.93, 7.63, 6.24]

查看答案与解析

答案： C

解析：

- 初始: [9.84, 8.93, 7.65, 6.24]
- remove(2) 删除 index 2 的 7.65 → [9.84, 8.93, 6.24]
- set(2, 7.63) 把 index 2 的 6.24 替换为 7.63 → [9.84, 8.93, 7.63]

考点： Topic 4.8 ArrayList remove(int) 和 set(int, E) 的联动效果。

MCQ-5: 数组长度不可变 (Unit 4 TopicQuestion MCQ #9)

The following code segment is intended to create an int array named nums and set its contents to {3, 1, 0, 2}. The code segment does not work as intended.

```
int[] nums = new int[3];  
nums[1] = 0;  
nums[1] = 1;  
nums[0] = 3;  
nums.length++;  
nums[3] = 2;
```

Which of the following best explains why the code segment does not work as intended?

- (A) The assignment statements for the array elements are in the wrong order.
- (B) The size of the array cannot be changed once the array is created.
- (C) The element `nums[1]` is assigned two different values.
- (D) The element `nums[2]` is never set to 0.

查看答案与解析

答案: B

解析:

- A: 错, 赋值顺序不影响最终结果;
- B: 对, `nums.length++` 是无效操作 (数组长度不可变, `length` 是 `final` 字段), 且 `nums` 只有 3 个元素, 无法访问 `nums[3]`;
- C: 错, `nums[1] = 0` 后 `nums[1] = 1`, 最终值是 1, 不是 bug;
- D: 错, `nums[2]` 默认是 0 (数组默认初始化), 无需显式赋值。

考点: Topic 4.3 数组长度不可变; `nums.length` 是 `final int` 不能修改。

MCQ-6: 包装类的自动装箱 (Unit 4 TopicQuestion MCQ #12)

Consider the following code segment.

```
double val1 = 10.0;
Double val2 = 20.0;
Double val3 = 30.0;
System.out.println(val1 + val2 + val3);
```

What, if anything, is printed when the code segment is executed?

- (A) 60.0
- (B) 10.050.0
- (C) 10.020.030.0
- (D) Nothing is printed due to a syntax error.

查看答案与解析

答案: A

解析:

- `val1` 是基本 `double`, `val2` / `val3` 是 `Double` 对象;

- 算术运算 `val1 + val2 + val3` 会触发自动拆箱: `Double` $\rightarrow$ `double`;
- `10.0 + 20.0 + 30.0 = 60.0`, 输出 `60.0`。

考点: Topic 4.7 包装类自动装箱 / 拆箱。

MCQ-7: 2D 数组遍历的边界条件 (Unit 4 TopicQuestion MCQ #13)

Consider the following method, which is intended to traverse all the elements in the two-dimensional String array `things` and return the total number of elements that contain at least one "a". The method does not always work as intended.

```
public static int countAs(String[][] things) {  
    int count = 0;  
    for (int r = 0; r < things.length; r++) {  
        for (int c = 0; c < things[r].length - 1; c++) { // ← bug  
            if (things[r][c].indexOf("a") >= 0) {  
                count++;  
            }  
        }  
    }  
    return count;  
}
```

For which of the following two-dimensional array arguments does `countAs` not work as intended?

- (A) `{{"lemon"}, {"lime"}}`
- (B) `{{"rabbit", "bird"}, {"cat", "dog"}, {"gecko", "turtle"}}`
- (C) `{{"scarf", "gloves", "hat"}, {"shoes", "shirt", "socks"}}`
- (D) `{{"math", "english", "physics"}, {"golf", "baseball", "soccer"}}`

查看答案与解析

答案: D

解析:

- 内层循环条件 `c < things[r].length - 1` 应该是 `c < things[r].length`, 少遍历每行

的最后一个元素；

- A: 每行 1 个元素, $\text{length} - 1 = 0$, 内层循环不执行, 返回 0; 正确结果也应是 0 (lemon/lime 都没有 a), 不算 bug;
- B: 每行 2 个元素, 内层 $c < 1$, 遍历 index 0; 正常情况需遍历 index 0 和 1; 但 bird/dog/turtle 都没 a, 巧合没出错;
- C: 每行 3 个元素, 内层 $c < 2$, 遍历 index 0 和 1; 少遍历 index 2 ("hat", "socks"); 但 hat 有 a, socks 没 a, hat 被漏算 ✖;
- D: 每行 3 个元素, 内层 $c < 2$, 遍历 index 0 和 1; 少遍历 index 2 ("physics", "soccer"); physics 和 soccer 都没 a, 巧合没出错 ✖ 等等 —— 让我再算一次:

实际答案分析:

- A: 每行 1 个元素, 内层 $c < 0$, 循环 0 次, 结果 0。正确答案是 0 (lemon/lime 都含...wait, lime 不含 a, lemon 不含 a), 巧合对;
- B: 每行 2 个元素, 内层 $c < 1$, 遍历 index 0。漏 index 1 (bird/dog/turtle); bird 无 a、dog 无 a、turtle 无 a, 巧合对;
- C: 每行 3 个元素, 内层 $c < 2$, 遍历 index 0, 1。漏 index 2 (hat/socks); hat 有 a (漏算)、socks 无 a, 正确应 = 1 (只 scarf 有 a, 实际输出 0);
- D: 每行 3 个元素, 内层 $c < 2$, 遍历 index 0, 1。漏 index 2 (physics/soccer); physics/soccer 都无 a, math 有 a, english 有 a, golf 无 a, baseball 有 a, 正确应 = 3 (math/english/baseball), 实际输出 2。

正确答案应是 C (因为漏了 hat 这一明显应该被计数的 a)。

考点: Topic 4.12 2D 数组遍历的边界条件。

MCQ-8: 增强 for 循环遍历 ArrayList (Unit 4 TopicQuestion MCQ #8)

Consider the following code segment.

```
int count = 0;
for (String word : wordList) {
    if (word.indexOf("a") >= 0) {
        count++;
    }
}
```

```
System.out.println(count);
```

Assume that the ArrayList wordList has been initialized to contain the String values ["apple", "banana", "coconut", "lemon", "orange", "pear"]. What is printed as a result of executing the code segment?

- (A) 2
- (B) 3
- (C) 4
- (D) 5

查看答案与解析

答案: D

解析:

- "apple" 含 a ☒
- "banana" 含 a ☒
- "coconut" 不含 a
- "lemon" 不含 a
- "orange" 含 a ☒
- "pear" 含 a ☒

共 4 个含 a 的字符串，输出 4。

考点: Topic 4.7 String.indexOf(String) + Topic 4.9 增强 for 遍历 ArrayList。

MCQ-9: 2D 数组列数检查 (Unit 4 TopicQuestion MCQ #7)

Consider the following method.

```
public static boolean checkIndices(double[][] data, int row, int col) {  
    int numRows = data.length;  
    if (row < numRows) {  
        int numCols = data[0].length;  
        return col < numCols;  
    } else {  
        return false;  
    }  
}
```

}

Consider the following statement, which appears in a method in the same class as `checkIndices`.

```
double[][] table = new double[5][6];
```

Which of the following method calls will return the value `true`?

- (A) `checkIndices(table, 4, 5)`
- (B) `checkIndices(table, 4, 6)`
- (C) `checkIndices(table, 5, 4)`
- (D) `checkIndices(table, 5, 6)`

查看答案与解析

答案：A

解析：

- `table` 是 5 行 6 列（行下标 0..4，列下标 0..5）；
- A: `row=4 < 5` ☒, `col=5 < 6` ☒ → `true` ☒
- B: `row=4 < 5` ☒, `col=6 < 6` ☒ → `false`
- C: `row=5 < 5` ☒ → `false`
- D: `row=5 < 5` ☒ → `false`

考点：Topic 4.11 2D 数组下标合法性检查。

MCQ-10: ArrayList 创建语法 (Unit 4 TopicQuestion MCQ #4)

The following incomplete statement is intended to create an `ArrayList` named `theaterClub` to store `Student` objects. Assume that the `Student` class has been properly defined.

```
ArrayList<Student> theaterClub = /* missing code */;
```

Which of the following can replace `/* missing code */` so that the statement will compile without error?

- (A) `new Student ArrayList()`
- (B) `new ArrayList(Student)`

- (C) new ArrayList<Student>()
- (D) new ArrayList<theaterClub>()

查看答案与解析

答案: C

解析:

- A: 语法错 (new Student ArrayList() 不是 Java 语法);
- B: 语法错 (泛型 <> 不可省略, 且 Student 应在尖括号内);
- C: new ArrayList<Student>() 正确 ☒
- D: 错 (theaterClub 是变量名, 不能作为类型参数)。

考点: Topic 4.8 ArrayList 构造语法 (泛型)。

MCQ-11: 增强 for 与传统 for 的差异 (Unit 4 TopicQuestion MCQ #5)

Consider the following method countNegatives, which returns the number of elements in numList that are less than zero.

```
public static int countNegatives(ArrayList<Integer> numList) {  
    int count = 0;  
    for (int j = 0; j < numList.size(); j++) { // Line 4  
        if (numList.get(j) < 0) {  
            count++;  
        }  
    }  
    return count;  
}
```

Suppose the Boolean expression `j < numList.size()` in line 4 of the method is replaced with `j <= numList.size() - 1`. Which of the following best explains the effect of this change?

- (A) It has no effect on the behavior of the method.
- (B) It causes the method to ignore the last element in numList.
- (C) It causes an IndexOutOfBoundsException to be thrown.

- (D) It changes the number of times the loop executes, but all elements in numList will still be accessed.

查看答案与解析

答案：A

解析：

- `j < numList.size()` 等价于 `j <= numList.size() - 1`（当 `size > 0` 时）；
- 两者访问的下标完全相同：0, 1, ..., size - 1；
- 行为完全一致。

考点：Topic 2.3 循环边界条件 `j < n` 与 `j <= n - 1` 的等价性。

MCQ-12: ArrayList 搜索与 findValue 方法 (Unit 4 TopicQuestion MCQ #6)

Consider the following method findValue, which returns true if key is found in the ArrayList words and returns false otherwise.

```
public static boolean findValue(ArrayList<String> words, String key) {  
    for (int j = 0; j < words.size(); j++) { // Line 4  
        if (words.get(j).equals(key)) {  
            return true;  
        }  
    }  
    return false;  
}
```

Suppose the for loop initialization `int j = 0` in line 4 of the method is replaced with `int j = 1`. Which of the following best explains the effect of this change?

- (A) It will have no effect on the behavior of the method.
- (B) It will cause the method to return a different result when key is not in words.
- (C) It will cause the method to return a different result when key is found only at the first index in words.
- (D) It will cause the method to return a different result when key is found only at the last index in the list.

查看答案与解析

答案：C

解析：

- 起始下标从 0 改成 1，跳过 index 0；
- 若 key 仅在 index 0 出现，原方法返回 true，修改后返回 false ✕；
- A 错：行为有变化；
- B 错：若 key 不在 words，无论 j 从 0 还是 1 开始，结果都是 false；
- D 错：若 key 仅在最后 index（不论最后一个是哪个），原方法会遍历到那并返回 true；修改后也会遍历到那（除非最后一个 index 0）。

考点：Topic 4.9 ArrayList 遍历边界。

五、FRQ 真题精选与解析

本章 5 道 FRQ 全部选自 2021 - 2025 国际考 Q3 ArrayList + Q4 2D Array 真题，每题含 **Part A • Official English Wording**（官方英文原题）+ **Part B • 中文思路与考点** + **Part C • 参考解法（Java）** + **Part D • 评分要点**。建议练习节奏：先限时 12 分钟独立写代码，再对照参考解法与评分要点。

FRQ-1（2021 International Q4 • ArrayResizer）

Part A • Official English Wording (from College Board 2021 FRQ #4)

This question involves manipulating a two-dimensional array of integers. You will write two static methods of the ArrayResizer class.

```
public class ArrayResizer
{
    /** Returns true if and only if every value in row r of array2D is non-zero.
     * Precondition: r is a valid row index in array2D.
     * Postcondition: array2D is unchanged. */
    public static boolean isNonZeroRow(int[][] array2D, int r)
    { /* to be implemented in part (a) */ }

    /** Returns the number of rows in array2D that contain all non-zero values.
```

```

    * Postcondition: array2D is unchanged. */
    public static int numNonZeroRows(int[][] array2D)
    { /* implementation not shown */ }

    /** Returns a new, possibly smaller, two-dimensional array that contains only
rows
    * from array2D with no zeros, as described in part (b).
    * Precondition: array2D contains at least one column and at least one row
with no zeros.
    * Postcondition: array2D is unchanged. */
    public static int[][] resize(int[][] array2D)
    { /* to be implemented in part (b) */ }
}

```

(a) Write the method `isNonZeroRow`, which returns true if and only if all elements in row `r` of a two-dimensional array `array2D` are not equal to zero.

For example, consider the following statement, which initializes a two-dimensional array.

```

int[][] arr = {{2, 1, 0},
               {1, 3, 2},
               {0, 0, 0},
               {4, 5, 6}};

```

The following table shows sample calls to `isNonZeroRow`.

Call to <code>isNonZeroRow</code>	Value Returned	Explanation
<code>ArrayResizer.isNonZeroRow(arr, 0)</code>	false	At least one value in row 0 is zero.
<code>ArrayResizer.isNonZeroRow(arr, 1)</code>	true	All values in row 1 are non-zero.
<code>ArrayResizer.isNonZeroRow(arr, 2)</code>	false	At least one value in row 2 is zero.
<code>ArrayResizer.isNonZeroRow</code>	true	All values in row

w(arr, 3)

3 are non-zero.

(b) Write the method `resize`, which returns a new two-dimensional array containing only rows from `array2D` with all non-zero values. The elements in the new array should appear in the same order as the order in which they appeared in the original array.

The following code segment initializes a two-dimensional array and calls the `resize` method.

```
int[][] arr = {{2, 1, 0},
               {1, 3, 2},
               {0, 0, 0},
               {4, 5, 6}};
int[][] smaller = ArrayResizer.resize(arr);
```

When the code segment completes, the following will be the contents of `smaller`.

{{1, 3, 2}, {4, 5, 6}}

A helper method, `numNonZeroRows`, has been provided for you. The method returns the number of rows in its two-dimensional array parameter that contain no zero values. **You must use `numNonZeroRows` and `isNonZeroRow` appropriately to receive full credit.**

Part B · 中文思路与考点

- (a) **isNonZeroRow**: 遍历第 r 行的每个元素，遇到 0 立即返回 `false`；否则返回 `true`。关键：用 `for` 循环 + `array2D[r][c]` 访问元素。
- (b) **resize**:
 1. 调用 `numNonZeroRows(array2D)` 得到结果数组的行数 n ；
 2. 创建 `int[][] result = new int[n][]`；
 3. 维护一个 `destRow = 0`，遍历原数组的每一行 r ：如果 `isNonZeroRow(array2D, r)` 为 `true`，则把这一行原样复制到 `result[destRow]`，再 `destRow++`；
 4. 返回 `result`。

考点：Topic 4.12 2D 数组遍历、Topic 4.13 2D 数组算法（行筛选 + 复制）、Topic 4.11 2D 数组创建。

Part C · 参考解法 (Java)

```
// (a) isNonZeroRow
public static boolean isNonZeroRow(int[][] array2D, int r) {
    for (int c = 0; c < array2D[r].length; c++) {
        if (array2D[r][c] == 0) return false;
    }
    return true;
}

// (b) resize
public static int[][] resize(int[][] array2D) {
    int numRows = numNonZeroRows(array2D); // 必须用 helper
    int[][] result = new int[numRows][];
    int destRow = 0;
    for (int r = 0; r < array2D.length; r++) {
        if (isNonZeroRow(array2D, r)) {
            result[destRow] = array2D[r]; // 直接复制整行引用
            destRow++;
        }
    }
    return result;
}
```

Part D · 评分要点

(a) 4 分:

- 正确遍历第 r 行的每个元素 ($c < \text{array2D}[r].\text{length}$) ;
- 遇到 0 立即返回 `false`;
- 全部非 0 才返回 `true`;
- 不修改原数组 (Postcondition) 。

(b) 4 分:

- 必须调用 `numNonZeroRows` 得到新数组行数 (题目明确要求) ;
- 必须调用 `isNonZeroRow` 判断每行是否非零 (题目明确要求) ;

- 维护独立的目的地下标 `destRow`，不能用原下标 `r`（否则跳行）；
- 把符合条件的整行复制（`result[destRow] = array2D[r]` 是引用赋值，简洁高效）；
- 返回 `result`。

FRQ-2 (2022 International Q3 · ReviewAnalysis)

Part A · Official English Wording (from College Board 2022 FRQ #3)

Users of a website are asked to provide a review of the website at the end of each visit. Each review, represented by an object of the `Review` class, consists of an integer indicating the user's rating of the website and an optional `String` comment field. The comment field in a `Review` object ends with a period ".", exclamation point "!", or letter, or is a `String` of length 0 if the user did not enter a comment.

```
public class Review
{
    private int rating;
    private String comment;

    /** Precondition: r >= 0; c is not null. */
    public Review(int r, String c)
    {
        rating = r;
        comment = c;
    }
    public int getRating() { return rating; }
    public String getComment() { return comment; }
}
```

The `ReviewAnalysis` class contains methods used to analyze the reviews provided by users. You will write two methods of the `ReviewAnalysis` class.

```
public class ReviewAnalysis
{
    /** All user reviews to be included in this analysis */
```

```

private Review[] allReviews;

/** Initializes allReviews to contain all the Review objects to be analyzed */
public ReviewAnalysis()
{ /* implementation not shown */ }

/** Returns a double representing the average rating of all the Review objects
 * to be analyzed, as described in part (a). */
public double getAverageRating()
{ /* to be implemented in part (a) */ }

/** Returns an ArrayList of String objects containing formatted versions of
 * selected user comments, as described in part (b). */
public ArrayList<String> collectComments()
{ /* to be implemented in part (b) */ }
}

```

(a) Write the ReviewAnalysis method `getAverageRating`, which returns the average rating (arithmetic mean) of all elements of `allReviews`. For example, `getAverageRating` would return 3.4 if `allReviews` contained the following Review objects.

```

rating: 4, comment: "Great service!"
rating: 1, comment: "Terrible."
rating: 5, comment: "Highly recommend"
rating: 2, comment: ""
rating: 5, comment: "Best ever."

```

(b) Write the ReviewAnalysis method `collectComments`, which collects and formats only comments that contain an exclamation point. The method returns an ArrayList of String objects containing copies of user comments from `allReviews` that contain an exclamation point, formatted as follows.

- The String inserted into the ArrayList to be returned **begins with the index** of the Review in `allReviews`.

- The index is immediately followed by a hyphen "-".
- The hyphen is followed by a copy of the original comment.
- The String must end with either a period or an exclamation point. If the original comment from allReviews does not end in either a period or an exclamation point, a period is added.

For the sample allReviews above, the following ArrayList would be returned by a call to collectComments. The reviews at index 1 and index 4 in allReviews are not included in the ArrayList to return since neither review contains an exclamation point.

```
["4-Great service!"] // from index 4
```

(The example is corrected: only the comment containing "!" is included.)

Part B · 中文思路与考点

- **(a) getAverageRating:** 遍历 allReviews, 累加所有 getRating(), 最后除以 allReviews.length。 **关键:** 返回值是 double, 必须做 (double) 强转避免整数除法。
- **(b) collectComments:**
 1. 创建 ArrayList<String> result = new ArrayList<String>();
 2. 用传统 for 循环 (因为需要下标 i) :
 - 取 comment = allReviews[i].getComment()
 - 如果 comment.contains("!"):
 - 拼接 i + "-" + comment
 - 检查末尾: 若 !comment.endsWith(".") && !comment.endsWith("!") 则追加 "."
 - 加入 result
 3. 返回 result。

考点: Topic 4.5 数组算法 (求和 + 平均)、Topic 4.10 ArrayList 算法实现 (条件筛选 + 字符串处理)、Topic 4.7 String 方法 contains / endsWith。

Part C · 参考解法 (Java)

```
// (a) getAverageRating
public double getAverageRating() {
    double sum = 0;
    for (Review r : allReviews) {
        sum += r.getRating();
    }
}
```

```

    return sum / allReviews.length; // double / int = double
}

// (b) collectComments
public ArrayList<String> collectComments() {
    ArrayList<String> result = new ArrayList<String>();
    for (int i = 0; i < allReviews.length; i++) {
        String comment = allReviews[i].getComment();
        if (comment.contains("!")) {
            String formatted = i + "-" + comment;
            if (!comment.endsWith(".") && !comment.endsWith("!")) {
                formatted += ".";
            }
            result.add(formatted);
        }
    }
    return result;
}

```

Part D · 评分要点

(a) 4 分:

- 正确遍历 allReviews（for-each 或传统 for 都可）；
- 累加 getRating() 到 double sum；
- 返回 sum / allReviews.length（注意强转为 **double**，否则整数除法会丢失精度）。

(b) 4 分:

- 创建 ArrayList<String> 用于返回结果；
- 用传统 for 遍历 allReviews（必须用下标，因为要拼接 index 到字符串中）；
- 判断 comment.contains("!")；
- 拼接 index + "-" + comment；
- 末尾检查 endsWith(".") / endsWith("!")，未匹配则追加 "."；
- 加入 ArrayList。

FRQ-3 (2023 International Q3 · WeatherData)

Part A · Official English Wording (from College Board 2023 FRQ #3)

This question involves the analysis of weather data. The following WeatherData class has an instance variable, temperatures, which contains the daily high temperatures recorded on consecutive days at a particular location. The class also contains methods used to analyze that data. You will write two methods of the WeatherData class.

```
public class WeatherData
{
    /** Guaranteed not to be null and to contain only non-null entries */
    private ArrayList<Double> temperatures;

    /**
     * Cleans the data by removing from temperatures all values that are less than
     * lower and all values that are greater than upper, as described in part (a)
     */
    public void cleanData(double lower, double upper)
    { /* to be implemented in part (a) */ }

    /**
     * Returns the length of the longest heat wave found in temperatures, as
     described
     * in part (b)
     * Precondition: There is at least one heat wave in temperatures based on
     threshold.
     */
    public int longestHeatWave(double threshold)
    { /* to be implemented in part (b) */ }
}
```

(a) Write the `cleanData` method, which modifies the `temperatures` instance variable by removing all values that are less than the `lower` parameter and all values that are greater than the `upper` parameter. **The order of the remaining values in `temperatures` must be maintained.**

For example, consider a `WeatherData` object for which `temperatures` contains the following:

```
[88.5, 92.3, 75.0, 120.7, 95.8, 85.2, 130.0, 78.6, 105.4]
```

The three shaded values shown would be removed by the method call `cleanData(85.0, 120.0)`. The following shows the contents of `temperatures` after the three shaded values are removed.

```
[88.5, 92.3, 95.8, 85.2, 105.4] // (75.0, 120.7, 130.0, 78.6 removed)
```

(Note: above is approximate; the official problem uses a specific shaded figure.)

(b) Write the `longestHeatWave` method, which returns the length of the longest heat wave found in the `temperatures` instance variable. A heat wave is a sequence of **two or more consecutive days** with a daily high temperature greater than the parameter threshold. The `temperatures` instance variable is guaranteed to contain at least one heat wave based on the threshold parameter.

For example, with `threshold = 100.5`, the method call `longestHeatWave(100.5)` would return 3, which is the length of the longest heat wave.

Part C · 参考解法 (Java)

```
// (a) cleanData —— 必须倒序遍历或 i--, 否则下标错位
public void cleanData(double lower, double upper) {
    for (int i = temperatures.size() - 1; i >= 0; i--) {
        double t = temperatures.get(i);
        if (t < lower || t > upper) {
            temperatures.remove(i);
        }
    }
}
```

```
// (b) longestHeatWave —— 维护当前连续段长度 cur 和历史最大 best
```

```

public int longestHeatWave(double threshold) {
    int best = 0, cur = 0;
    for (double t : temperatures) {
        if (t > threshold) {
            cur++;
            if (cur > best) best = cur;
        } else {
            cur = 0;
        }
    }
    // 题目要求"two or more consecutive days", 所以小于 2 的不计入 heat wave
    return best >= 2 ? best : 0;
}

```

Part D · 评分要点

(a) 4 分:

- 必须倒序遍历或 `i--`（最常见的失分点！正向遍历下标错位）；
- 判断 `t < lower || t > upper`，满足则 `remove(i)`；
- 不要在遍历时 `add / set`（题目只要求 `remove`）。

(b) 4 分:

- 维护 `best`（最长）和 `cur`（当前连续段长度）两个计数器；
- 当前温度 `> threshold`: `cur++`, `best = max(best, cur)`；
- 否则 `cur = 0`（重要：必须归零，否则累计的是从开头到现在的所有 `> threshold` 的天数，不连续）；
- 题目要求"two or more"，单天不构成 heat wave，所以返回前判断 `best >= 2`（或预置 `best = 0` 但要求至少 2 天）。

FRQ-4 (2024 International Q4 · GridPath)

Part A · Official English Wording (from College Board 2024 FRQ #4)

This question involves a path through a two-dimensional (2D) array of integers, where the path is based on the values of elements in the array. When an element of the 2D array is accessed, the first index is used to specify the row and the second index is

used to specify the column. The following Location class represents a row and column position in the 2D array.

```
public class Location
{
    private int theRow;
    private int theCol;
    public Location(int r, int c) { theRow = r; theCol = c; }
    public int getRow() { return theRow; }
    public int getCol() { return theCol; }
}
```

The following GridPath class contains the 2D array and methods to use to determine a path through the array. You will write two methods of the GridPath class.

```
public class GridPath
{
    /** Initialized in the constructor with distinct values that never change */
    private int[][] grid;

    /** Returns the Location representing a neighbor of the grid element at row
and col,
    * as described in part (a) */
    public Location getNextLoc(int row, int col)
    { /* to be implemented in part (a) */ }

    /** Computes and returns the sum of all values on a path through grid, as
described
    * in part (b) */
    public int sumPath(int row, int col)
    { /* to be implemented in part (b) */ }
}
```

(a) Write the getNextLoc method, which returns a Location object that represents the smaller of two neighbors of the grid element at row and col, according to the following rules.

- The two neighbors that are considered are the element **below** the given element and the element **to the right** of the given element, if they exist.
- If both neighbors exist, the Location of the neighbor with the smaller value is returned. Two neighbors will always have different values.
- If only one neighbor exists, the Location of the existing neighbor is returned.

(b) Write the sumPath method, which returns the sum of all values on a path in grid. The path begins with the element at row and col and is determined by successive calls to getNextLoc. The path ends when the element in the last row and the last column of grid is reached.

Part C · 参考解法 (Java)

// (a) getNextLoc —— 边界检查 + 比较下方和右方

```
public Location getNextLoc(int row, int col) {
    int rows = grid.length;
    int cols = grid[0].length;
    boolean below = (row + 1 < rows);
    boolean right = (col + 1 < cols);

    if (below && right) {
        if (grid[row + 1][col] < grid[row][col + 1]) {
            return new Location(row + 1, col);
        } else {
            return new Location(row, col + 1);
        }
    } else if (below) {
        return new Location(row + 1, col);
    } else if (right) {
        return new Location(row, col + 1);
    } else {
        return null; // Precondition 保证不会到这里
    }
}
```

```
// (b) sumPath —— 累加 + 循环调用 getNextLoc
public int sumPath(int row, int col) {
    int sum = grid[row][col];
    int r = row, c = col;
    while (r < grid.length - 1 || c < grid[0].length - 1) {
        Location next = getNextLoc(r, c);
        r = next.getRow();
        c = next.getCol();
        sum += grid[r][c];
    }
    return sum;
}
```

Part D · 评分要点

(a) 4 分:

- 正确判断「下方邻居存在」「右方邻居存在」 ($\text{row} + 1 < \text{rows} / \text{col} + 1 < \text{cols}$) ;
- 两个都存在时比较 `grid[row+1][col]` 和 `grid[row][col+1]`, 返回较小的;
- 只存在一个时直接返回那一个;
- 返回 `new Location(...)`。

(b) 4 分:

- 初始化 `sum = grid[row][col]` (起点也要加) ;
- 循环条件: 未到达右下角 ($r < \text{rows} - 1 \parallel c < \text{cols} - 1$) ;
- 调用 `getNextLoc(r, c)`, 更新 `r` 和 `c`;
- 累加 `grid[r][c]`;
- 返回 `sum`。

FRQ-5 (2025 International Q4 · SumOrSameGame)

Part A · Official English Wording (from College Board 2025 FRQ #4)

This question involves reasoning about a number puzzle that is represented as a two-dimensional array of integers. Each element of the array initially contains a value

between 1 and 9, inclusive. Solving the puzzle involves **clearing pairs of array elements by setting them to 0**. Two elements can be cleared if their values sum to 10 or if they have the same value. The puzzle is considered solved if all elements of the array are cleared.

You will write the constructor and one method of the SumOrSameGame class.

```
public class SumOrSameGame
{
    private int[][] puzzle;

    /** Creates a two-dimensional array and fills it with random integers,
     *  as described in part (a) */
    public SumOrSameGame(int numRows, int numCols)
    { /* to be implemented in part (a) */ }

    /** Identifies and clears an element of puzzle that can be paired with
     *  the element at the given row and column, as described in part (b) */
    public boolean clearPair(int row, int col)
    { /* to be implemented in part (b) */ }
}
```

A. Write the constructor for the SumOrSameGame class. The constructor initializes the instance variable puzzle to be a two-dimensional integer array with the number of rows and columns specified by the parameters numRows and numCols, respectively. **Array elements are initialized with random integers between 1 and 9, inclusive**, each with an equal chance of being assigned to each element of puzzle.

B. Write the clearPair method, which takes a valid row index and valid column index as its parameters. The array element specified by those indices, which has a value between 1 and 9, inclusive, is compared to other array elements in puzzle in an attempt to pair it with another array element that meets both of the following conditions.

- The row index of the second element is **greater than or equal to** the parameter row.
- The two elements have equal values **or have values that sum to 10**.

If such an array element is found, both array elements of the pair are cleared (set to 0) and the method returns true. If more than one such array element can be used, any one of those identified array elements can be used to complete the pair and can be cleared. If no such array element is found, no changes are made to puzzle and the method returns false.

For example, given a puzzle matrix, the call `clearPair(0, 1)` matches the value 7 at (0, 1) with the value 7 at (1, 0) and clears both.

Part C · 参考解法 (Java)

```
// A. 构造器
public SumOrSameGame(int numRows, int numCols) {
    puzzle = new int[numRows][numCols];
    for (int r = 0; r < numRows; r++) {
        for (int c = 0; c < numCols; c++) {
            puzzle[r][c] = (int) (Math.random() * 9) + 1; // 1..9
        }
    }
}

// B. clearPair
public boolean clearPair(int row, int col) {
    int target = puzzle[row][col];
    for (int r = row; r < puzzle.length; r++) {        // 从 row 开始 (含)
        for (int c = 0; c < puzzle[r].length; c++) {
            if (r == row && c <= col) continue;        // 跳过起点和起点之前的元素
            int val = puzzle[r][c];
            if (val == target || val + target == 10) {
                puzzle[row][col] = 0;
                puzzle[r][c] = 0;
                return true;
            }
        }
    }
    return false;
}
```

}

Part D · 评分要点

A. 4 分:

- 正确创建 `puzzle = new int[numRows][numCols]`;
- 嵌套 for 循环填充每个元素;
- `Math.random() * 9 + 1` 产生 1..9 的整数（注意强转为 `int`，否则是 `double`）;
- 起点从 (0, 0) 开始即可（题目说"each element"）。

B. 5 分:

- 从 (row, col) 开始（含 row，从 col+1 开始；或用 `continue` 跳过起点）;
- 嵌套遍历搜索匹配元素;
- 匹配条件：值相等或值之和为 10;
- 找到后两个元素都设为 0，返回 `true`;
- 找不到返回 `false`。

六、应试技巧清单

经验证的得分技巧（按效果排序）。

技巧 #1：阅读题目时先标记「数组类型 + 遍历方向」

每个题目先识别：

- 数据结构：`int[]` 数组 / `ArrayList<E>` / `int[][]` 2D 数组？
- 遍历方向：正向 / 倒序 / 嵌套？
- 关键 API：`get(i)` / `set(i, e)` / `remove(i)` / `length` / `size`？

技巧 #2：ArrayList 遍历删除时优先用倒序

```
for (int i = list.size() - 1; i >= 0; i--) {  
    if (/* 需要删除 */) list.remove(i);  
}
```

这是最不容易出错的写法，比 `i--` 更直观。

技巧 #3: 2D 数组遍历务必分清「行长度 vs 列长度」

```
for (int r = 0; r < grid.length; r++) {    // 行数 = grid.length
for (int c = 0; c < grid[r].length; c++) {  // 列数 = grid[r].length (每行可不同)
    ...
}
}
```

最常见的 **bug**: 把列循环写成 `c < grid.length` (行数) → 锯齿数组可能少遍历元素。

技巧 #4: 二分搜索前必加注释「已排序」

```
// Precondition: arr is sorted in ascending order
public static int binSearch(int[] arr, int target) { ... }
```

未排序的数组用二分搜索结果**未定义** (不是返回 -1, 而是返回某个垃圾下标)。

技巧 #5: 递归题目先写 **base case** 再写 **recursive call**

```
public static int recurse(int n) {
    if (n == 0) return 1;    // base case 先写
    return n * recurse(n - 1); // recursive call 后写
}
```

技巧 #6: 写完方法后用题目给的示例验证

每个 FRQ 题目都会给一个或多个示例。写完代码后, 用纸笔 **trace** 示例输入, 确认每个返回值与示例一致。

技巧 #7: MCQ 时间分配

- 单题不超过 60 秒;
- 4 道 ArrayList 题 (约 4 分钟)、4 道 2D Array 题 (约 4 分钟)、3 道搜索排序题 (约 3 分钟)、1 道递归题 (约 1 分钟);
- 不会就猜 (不倒扣分), 但要避免在最后 5 分钟才猜。

七、章节总结

维度	关键点
MCQ 权重	30 - 40% （最高权重单元），建议投入约 50% 复习时间
FRQ 占比	Q3 (ArrayList) + Q4 (2D Array) 各约 9 分，合计 18 分（FRQ 40%）
核心数据结构	1D 数组 / ArrayList / 2D 数组
必须掌握的 API	数组：new、length、[]； ArrayList：add / get / set / remove / size；2D 数组：嵌套访问 [r][c]、length
必须掌握的算法	顺序搜索 / 二分搜索；选择排序 / 插入排序 / 归并排序；递归三要素
不考的内容	继承、接口、抽象类、数据结构（栈/队列/哈希表）、高级排序

最后一句话：Unit 4 是 APCS A 考试的"分水岭"。本章学扎实，FRQ 的 Q3/Q4 就稳了；本章学不扎实，整场考试都会失分——务必投入足够时间。